MONOGRÁFICO FORMATIVO

Fundamentos de IA Conversacional Neuro-Simbólica

«A system that cannot do what it needs to do cannot be accountable for what it does.»

Eje A — El problema: IA conversacional y accountability

Unidad 3 de 12

Research Sprint ACAI · Proyecto $(AI)^2$:TDG · 2026

Eje A — El problema

Prerequisitos: UD1, UD2 · Eje A — El problema

UNIDAD DIDÁCTICA 3

El ecosistema operativo: componentes, capacidades y límites de gobernanza

Contenidos de la unidad

3.2.1 El patrón y lo que resuelve 4
3.2.2 El análisis de gobernanza 5
3.2.3 El concepto de RAG gobernado 7
3.3.1 El paradigma agéntico y lo que aporta 8
3.3.2 El análisis de gobernanza 9
3.3.3 El concepto de capacidad agéntica gobernada 10
3.4.1 El estado del arte en alineamiento 11
3.4.2 El análisis de gobernanza 12
3.4.3 El concepto de alineamiento formalizado 13
3.5.1 El contexto en los LLMs: ventana, degradación y opacidad 14
3.5.2 La tradición del *information state update* 15
3.5.3 El *information state* en la arquitectura ACAI 16
3.6.1 El problema de la coordinación 17
3.6.2 La gobernanza formal unificada 17
3.6.3 Lo que la gobernanza unificada no resuelve 19

¿Por qué necesito esto para ACAI?

La UD 2 estableció que la conversación es un fenómeno multidimensional que las arquitecturas actuales generan sin representar. Esta unidad completa el diagnóstico desde la perspectiva tecnológica. El investigador que se incorpora a ACAI probablemente conoce RAG, ha usado frameworks agénticos, y quizás ha experimentado con Constitutional AI. Es probable que piense que combinando estas piezas se resuelve el problema de la accountability. Esta unidad desmonta esa intuición con rigor — sin caricaturizar las soluciones existentes, pero mostrando con precisión dónde falla cada una en términos de gobernanza. Al terminar, el investigador entenderá por qué ACAI no propone prescindir de estos componentes sino *gobernarlos* bajo un modelo formal unificado.

Objetivos de aprendizaje

Al completar esta unidad, el investigador será capaz de:

1. Describir los patrones RAG, Agentic AI y Constitutional AI con precisión técnica suficiente para evaluar sus propiedades de gobernanza.
2. Identificar, para cada patrón, qué resuelve genuinamente y dónde falla en términos de accountability: qué decisiones toma, quién las toma, y por qué no son auditables.
3. Argumentar por qué la combinación de patrones existentes no produce un sistema gobernado, incluso cuando cada componente funciona correctamente.
4. Conceptualizar la diferencia entre un componente autónomo y un componente gobernado por conocimiento formal.
5. Explicar el concepto de *information state* y su diferencia con el contexto implícito del LLM.
6. Articular la necesidad de gobernanza formal unificada como contribución diferencial de ACAI.

3.1 El argumento de la completitud

La UD 2 concluyó con un diagnóstico preciso: las arquitecturas conversacionales actuales generan comportamiento sin representarlo, lo que hace la accountability estructuralmente inalcanzable. La respuesta natural a este diagnóstico es proponer exactamente lo que ACAI propone: introducir representaciones formales explícitas y un motor simbólico que opere sobre ellas. Pero esta respuesta, por sí sola, es insuficiente.

Un sistema formalmente correcto pero operativamente incompleto no es accountable en la práctica.

Considérese un motor conversacional que opera sobre reglas formales perfectamente especificadas para el dominio asegurador. El motor sabe que una cancelación de póliza requiere verificar la identidad del cliente, consultar las condiciones de penalización, informar al cliente, y obtener confirmación explícita. Cada decisión es determinista y trazable. Pero en el momento en que el motor necesita consultar las condiciones de penalización, se encuentra con que esas condiciones están en un documento PDF de 200 páginas que se actualiza trimestralmente. Y cuando necesita verificar la identidad, debe invocar un servicio externo de autenticación. Y cuando genera la respuesta, debe ajustarse a las políticas de comunicación de la organización. Y a lo largo de toda la interacción, debe mantener un modelo coherente de qué se ha dicho, qué se ha entendido, y qué compromisos se han adquirido.

Cada una de estas necesidades requiere un componente que va más allá del motor simbólico: acceso a información documental, capacidad de ejecutar acciones, alineamiento con políticas, y gestión de contexto. Sin estos componentes, el motor es formalmente elegante e inútil en la práctica. Esta es la sustancia del principio P4: la accountability requiere completitud operativa.

El ecosistema tecnológico actual ofrece patrones maduros para cada una de estas necesidades: RAG para el acceso a información, arquitecturas agénticas para la ejecución de acciones, Constitutional AI para el alineamiento, y la ventana de contexto del LLM para la gestión conversacional. Lo que esta unidad demuestra es que estos patrones, tal como están formulados, aportan las capacidades que ACAI necesita pero carecen de las propiedades de gobernanza que ACAI exige. La innovación de ACAI no está en reemplazar estos componentes sino en *gobernarlos*: someter sus decisiones al mismo modelo formal que gobierna la conversación.

3.2 Retrieval-Augmented Generation: del patrón al problema de gobernanza

Sede principal de este concepto.

3.2.1 El patrón y lo que resuelve

Retrieval-Augmented Generation (RAG), tal como lo formularon Lewis et al. (2020), es un patrón arquitectónico que combina un módulo de recuperación de información con un modelo generativo. Ante una consulta del usuario, el sistema primero recupera documentos relevantes de un corpus externo y luego los inyecta en el contexto del modelo

Lewis et al. (2020) — el artículo fundacional de RAG.

generativo para que produzca una respuesta informada por esas fuentes. La formulación original proponía un entrenamiento end-to-end donde el retriever y el generador se optimizaban conjuntamente, pero en la práctica industrial el patrón se ha simplificado en un pipeline de tres etapas: indexación (los documentos se segmentan y se convierten en representaciones vectoriales), recuperación (ante una consulta se buscan los segmentos más similares semánticamente), y generación (los segmentos recuperados se incluyen en el prompt del LLM junto con la consulta).

El valor de RAG es genuino y sustancial. Resuelve uno de los problemas más persistentes de los LLMs: la desactualización del conocimiento. Un LLM entrenado con datos hasta una fecha determinada no puede responder sobre eventos o cambios posteriores; RAG permite acceder a información actualizada en tiempo real. Reduce además las alucinaciones factuales al anclar la generación en fuentes concretas: cuando el LLM tiene acceso a un documento que contiene la respuesta, la probabilidad de que invente información disminuye significativamente, como han documentado Shuster et al. (2021) en su trabajo sobre modelos con acceso a conocimiento externo.

El patrón ha evolucionado considerablemente desde la formulación original. Gao et al. (2024), en su survey comprensivo, distinguen entre RAG *naive* (el pipeline básico de tres etapas), RAG *advanced* (que incorpora técnicas de pre-recuperación como query rewriting y post-recuperación como reranking), y RAG *modular* (donde los componentes del pipeline se configuran de forma flexible). Las técnicas de chunking — cómo segmentar los documentos para la indexación — han generado una subliteratura propia, con propuestas que van desde la segmentación por tamaño fijo hasta el chunking semántico que respeta las fronteras de significado (Kamradt, 2023). El reranking — reordenar los resultados de la recuperación usando un modelo más preciso antes de inyectarlos en el contexto — se ha convertido en práctica estándar, con modelos como los de Nogueira y Cho (2020) y sus sucesores.

Gao et al. (2024): taxonomía de RAG naive, advanced y modular.

Estas mejoras son reales y relevantes. Pero todas operan dentro de un marco donde las decisiones fundamentales de gobernanza permanecen sin resolver.

3.2.2 El análisis de gobernanza

Para entender dónde falla RAG en términos de accountability, es necesario descomponer el patrón en las decisiones que toma y analizar quién las toma y con qué garantías.

En un RAG estándar, cinco decisiones críticas las toma el LLM o el pipeline de forma opaca e inauditable.

La primera decisión es **cuándo recuperar**. En un sistema conversacional, no toda pregunta del usuario requiere consultar documentos

externos. Algunas preguntas pueden responderse con la información ya disponible en el estado del diálogo; otras requieren acceder a documentación; otras requieren ejecutar una acción, no buscar información. En la implementación estándar, esta decisión la toma el LLM — sea porque toda consulta se enruta al retriever, sea porque el LLM decide invocarlo como herramienta. En ningún caso la decisión es trazable: no hay un registro formal de por qué se decidió buscar (o no buscar) en una situación concreta.

La segunda decisión es **qué buscar**. La consulta que se envía al retriever determina los resultados, y esa consulta puede ser la pregunta literal del usuario, una reformulación generada por el LLM, o una query expandida mediante técnicas de query rewriting. En todos los casos, la transformación de la necesidad informativa del usuario en una consulta de recuperación es un proceso opaco. Si el usuario pregunta «¿Pierdo la bonificación si cancelo?», la consulta efectiva al retriever podría ser «bonificación cancelación póliza», «condiciones pérdida bonificación seguro hogar», o cualquier otra reformulación. Cuál se elige, y por qué, no queda registrado.

La tercera decisión es **qué es relevante**. El retriever devuelve los k documentos más similares semánticamente a la consulta. Pero similitud semántica no es equivalente a relevancia para la tarea. Un documento puede ser semánticamente cercano pero referirse a un producto diferente, a una versión normativa obsoleta, o a condiciones que no aplican al caso concreto. Barnett et al. (2024), en su análisis de los siete puntos de fallo de RAG, documentaron que la recuperación de documentos irrelevantes o parcialmente relevantes es la causa más frecuente de respuestas incorrectas en sistemas RAG desplegados, con tasas que oscilan entre el 15 % y el 35 % según el dominio y la configuración.

La cuarta decisión es **cómo integrar la información**. Una vez que los documentos recuperados se inyectan en el contexto del LLM, el modelo debe decidir qué información usar, cómo combinarla con lo que ya sabe, y qué peso atribuir a cada fuente. Esta integración es enteramente opaca: el LLM procesa los documentos como tokens en su contexto y genera una respuesta que puede usar, ignorar, o malinterpretar la información recuperada. No hay mecanismo para verificar que el contenido factual de la respuesta corresponde efectivamente a lo que dicen los documentos.

Barnett et al. (2024) — los siete puntos de fallo de RAG.

La quinta decisión es **qué nivel de confianza atribuir**. No toda información documental tiene el mismo estatus: un artículo del contrato tiene fuerza legal; una FAQ tiene valor orientativo; una nota interna puede estar desactualizada. En un RAG estándar no hay mecanismo para distinguir estos niveles ni para ajustar la respuesta en consecuencia. El LLM trata todos los segmentos recuperados como contexto indistinto.

Cada una de estas cinco decisiones es, en un RAG estándar, opaca, no-trazable y no-verificable. La implicación para la accountability es directa: cuando el sistema proporciona información incorrecta al usuario, no se puede determinar si el fallo fue en la decisión de buscar, en la consulta, en la selección de documentos, en la integración, o en la atribución de confianza. La cadena de responsabilidad se pierde en el pipeline.

3.2.3 El concepto de RAG gobernado

RAG gobernado

En un **RAG gobernado**, las cinco decisiones del pipeline de recuperación están determinadas por el modelo formal de conocimiento, no por el LLM ni por heurísticas del pipeline. El modelo formal especifica: bajo qué condiciones del diálogo se debe buscar información documental; en qué fuentes buscar y con qué criterios de relevancia; cómo evaluar la pertinencia de los resultados para el caso concreto; cómo integrar la información en la respuesta; y qué nivel de confianza atribuir según el tipo de fuente. La traza de ejecución incluye todas estas decisiones.

La diferencia entre un RAG estándar y un RAG gobernado no es de rendimiento sino de arquitectura. Un RAG estándar puede funcionar mejor en términos de calidad de respuesta en ciertos escenarios; un RAG gobernado puede funcionar peor si las reglas no están bien diseñadas. Lo que el RAG gobernado garantiza es que cada decisión del pipeline es *auditable*: se puede reconstruir por qué se buscó, qué se encontró, por qué se consideró relevante, y cómo influyó en la respuesta. Esta auditabilidad es lo que la accountability en sectores regulados requiere.

RAG estándar vs. RAG gobernado

Contexto: El usuario ha solicitado cancelar su seguro de hogar y pregunta por la bonificación.
RAG estándar: El LLM decide buscar → envía la consulta «bonificación cancelación seguro hogar» al retriever → recupera 3 fragmentos de las condiciones generales → genera una respuesta que integra los fragmentos. Si la respuesta es incorrecta, no se puede determinar en qué punto del pipeline se produjo el error.
RAG gobernado: El motor simbólico determina que el estado del diálogo requiere consultar las condiciones de bonificación → la regla `bonus_query_01` especifica: buscar en *Condiciones Generales v2025.2*, sección *Bonificaciones por ausencia de siniestros* → el retriever busca en esa fuente específica → el motor evalúa la pertinencia del resultado contra

las restricciones formales (producto correcto, versión vigente) → el LLM genera la respuesta usando la información validada → la traza registra: regla aplicada, fuente consultada, fragmento recuperado, validación, respuesta.

3.3 Arquitecturas agénticas: de la autonomía a la gobernanza

Sede principal de este concepto.

3.3.1 El paradigma agéntico y lo que aporta

La capacidad de un sistema conversacional para ejecutar acciones sobre el mundo — consultar bases de datos, iniciar transacciones, invocar APIs, escalar a un operador humano — es una necesidad operativa ineludible. Un asistente que puede informar sobre las condiciones de cancelación pero no puede ejecutar la cancelación es útil pero incompleto. El paradigma agéntico, que ha experimentado una expansión notable desde 2023, proporciona esta capacidad.

El concepto de *tool use* — la capacidad de un LLM para invocar herramientas externas durante la generación — fue articulado formalmente por Schick et al. (2024) en Toolformer, donde demostraron que un modelo de lenguaje puede aprender a insertar llamadas a APIs en su proceso de generación. Sin embargo, la práctica industrial ha evolucionado hacia un mecanismo más estructurado: el *function calling*, donde el modelo selecciona entre un conjunto predefinido de funciones con esquemas tipados y devuelve los argumentos necesarios para invocarlas. Este mecanismo, implementado por OpenAI, Anthropic y Google en sus APIs, proporciona una interfaz más controlada que la inserción libre de llamadas a herramientas.

Schick et al. (2024) — Toolformer: enseñar al LLM a usar herramientas.

Sobre esta base, han surgido frameworks de orquestación que permiten construir flujos agénticos más complejos. El patrón ReAct (Yao et al., 2023) propuso intercalar razonamiento y acción: el LLM genera un paso de razonamiento (*thought*), ejecuta una acción (*action*), observa el resultado (*observation*), y repite hasta completar la tarea. Frameworks como LangChain (Chase, 2022), LlamaIndex, y CrewAI han operacionalizado estos patrones, proporcionando abstracciones para construir agentes que planifican, ejecutan, y se recuperan de errores.

Wang et al. (2024), en su survey comprensivo sobre agentes basados en LLM, identifican cuatro capacidades fundamentales: planificación (descomponer una tarea compleja en pasos), uso de herramientas (in-

Yao et al. (2023): ReAct — razonamiento intercalado con acción.

vocar APIs y servicios), memoria (mantener información a lo largo de la interacción), y reflexión (evaluar el resultado de acciones y ajustar el plan). Xi et al. (2023) amplían esta taxonomía para incluir arquitecturas multi-agente donde múltiples LLMs especializados colaboran en la resolución de tareas, cada uno con herramientas y roles diferenciados.

Estas capacidades son genuinamente valiosas para un sistema conversacional en un dominio real. Sin ellas, el sistema es un interlocutor pasivo incapaz de actuar. Pero el análisis de gobernanza revela problemas fundamentales que la entusiasta literatura sobre agentes frecuentemente minimiza.

3.3.2 El análisis de gobernanza

El problema de gobernanza en las arquitecturas agénticas es, en cierto sentido, más grave que en RAG, porque las acciones tienen consecuencias directas en el mundo real. Un documento irrelevante recuperado por RAG produce, en el peor caso, una respuesta incorrecta. Una acción ejecutada incorrectamente por un agente puede producir una transacción financiera errónea, una cancelación no autorizada, o una escalación inapropiada.

Un agente autónomo es, por definición, un agente cuyas decisiones no están gobernadas externamente.

La primera cuestión de gobernanza es la **selección de herramientas**. Cuando el LLM tiene acceso a un conjunto de funciones — por ejemplo, `consultar_póliza`, `cancelar_póliza`, `escalar_a_operador`, `enviar_email` — la decisión de cuál invocar la toma el modelo basándose en su interpretación de la situación. Esta decisión es probabilística e influenciable. Patil et al. (2023), en su trabajo sobre Gorilla, documentaron que los LLMs cometen errores de selección de herramienta en un 10–20 % de los casos, incluso con descripciones claras de cada función. En situaciones ambiguas — que son las más comunes en diálogos reales — la tasa de error puede ser significativamente mayor.

La segunda cuestión es el **orden de ejecución**. En tareas complejas que requieren múltiples acciones, el orden importa. Verificar la identidad antes de consultar datos es un requisito de seguridad. Informar sobre las consecuencias antes de ejecutar una cancelación es un requisito regulatorio. Un agente autónomo determina el orden de ejecución según su "razonamiento", que es en realidad una generación probabilística de pasos. Ruan et al. (2024), en su análisis de riesgos de agentes basados en LLM, documentaron casos donde agentes ejecutan acciones en orden incorrecto, omiten pasos de verificación, o ejecutan acciones redundantes, con consecuencias que van desde la ineficiencia hasta el daño operativo real.

Patil et al. (2023) — Gorilla: errores de selección de herramienta en LLMs.

La tercera cuestión es la **verificación de precondiciones**. Antes de ejecutar una acción con consecuencias, un sistema responsable debe

Ruan et al. (2024) — riesgos y fallos de agentes basados en LLM.

verificar que se cumplen las condiciones necesarias. ¿Tiene el usuario autoridad para esta operación? ¿Se ha proporcionado toda la información requerida? ¿Se ha obtenido la confirmación necesaria? En un framework agéntico estándar, estas verificaciones pueden estar codificadas como instrucciones en el prompt del agente, pero su cumplimiento no está garantizado — el LLM puede "olvidar" una precondición, especialmente en contextos largos o complejos.

La cuarta cuestión es la **gestión de errores**. Cuando una acción falla — el servicio externo no responde, los datos no son válidos, el resultado es inesperado — el agente debe decidir cómo proceder. En un framework estándar, esta decisión la toma el LLM, que puede reintentar, intentar una alternativa, escalar, o simplemente reportar el error al usuario. La estrategia de recuperación no está formalizada y no es predecible.

La consecuencia de estas cuatro carencias es que un sistema agéntico basado en LLM puede, en cualquier momento, ejecutar una acción que no corresponde, en un orden que no es correcto, sin haber verificado las precondiciones, y con una gestión de errores impredecible. Cada una de estas situaciones es un fallo de accountability: una acción no auditable, una decisión no trazable, un proceso no verificable.

3.3.3 El concepto de capacidad agéntica gobernada

> **Capacidad agéntica gobernada**
>
> En un sistema con **capacidad agéntica gobernada**, las acciones disponibles, las precondiciones para su ejecución, el orden en que deben ejecutarse, y las verificaciones post-ejecución están formalizadas en el modelo de conocimiento. El motor simbólico determina qué acción corresponde según las reglas del dominio y el estado del diálogo; el componente agéntico la ejecuta. El agente no decide autónomamente — ejecuta lo que el motor determina.

Phase 1
La gobernanza agéntica se implementa como reglas formales que el motor simbólico evalúa.

La diferencia es análoga a la distinción entre un empleado con autonomía total y un empleado que opera según procedimientos definidos. El primero puede ser más eficiente en situaciones imprevistas; el segundo ofrece garantías de que los procedimientos se cumplen. En un sector regulado, la segunda opción no es una preferencia sino un requisito. Un agente gobernado puede ser menos "creativo" en la resolución de problemas, pero su comportamiento es predecible, trazable y auditable.

Esto no significa que el LLM no tenga ningún papel en la capacidad agéntica. Puede asistir en la determinación de parámetros (interpretar que "el seguro de hogar" se refiere a la póliza HG-2019-4421), en la formulación de respuestas ante errores, y en la adaptación del lenguaje

al contexto. Lo que no hace es *decidir* qué acción ejecutar ni en qué orden. Esa decisión la toma el motor simbólico sobre la base de reglas formales.

3.4 Alineamiento y políticas: de la constitución al modelo formal

Sede principal de este concepto.

3.4.1 El estado del arte en alineamiento

El problema del alineamiento — lograr que un sistema de IA se comporte de acuerdo con valores, normas y objetivos humanos — ha generado una literatura extensa y un debate profundo que excede con mucho los límites de esta unidad. Lo que aquí interesa es un aspecto específico: cómo se gestionan las políticas de comportamiento en sistemas conversacionales, y por qué las aproximaciones actuales son insuficientes para la accountability.

La técnica más influyente ha sido RLHF (*Reinforcement Learning from Human Feedback*), introducida por Christiano et al. (2017) y llevada a escala industrial por Ouyang et al. (2022) con InstructGPT. La idea es entrenar un modelo de recompensa a partir de preferencias humanas y luego usar ese modelo para ajustar el LLM mediante aprendizaje por refuerzo. El resultado es un modelo que, estadísticamente, produce respuestas más alineadas con las preferencias expresadas. Variantes como DPO (*Direct Preference Optimization*, Rafailov et al., 2023) simplifican el proceso eliminando el modelo de recompensa explícito.

Ouyang et al. (2022) — InstructGPT y RLHF como técnica de alineamiento.

Constitutional AI (Bai et al., 2022) propuso un enfoque complementario: definir un conjunto de principios — una "constitución" — que el modelo usa para evaluarse y corregirse a sí mismo. El proceso consiste en generar respuestas, evaluarlas contra los principios constitucionales usando el propio modelo, y reentrenar con las revisiones como datos de preferencia. La innovación conceptual es significativa: explicitar los principios en lugar de derivarlos implícitamente de las preferencias de los anotadores.

A nivel de despliegue, estas técnicas de entrenamiento se complementan con mecanismos de runtime: system prompts que codifican las políticas del sistema, guardrails que filtran entradas y salidas, y clasificadores que detectan contenido que viola las políticas. Frameworks como Llama Guard (Inan et al., 2023) y NeMo Guardrails (Rebedea et al., 2023) proporcionan infraestructura para implementar estos mecanismos de forma modular.

Bai et al. (2022): Constitutional AI — principios explícitos para el alineamiento.

El resultado combinado de estas técnicas es innegablemente positivo.

Los LLMs alineados son más seguros, más útiles y más respetuosos que sus versiones base. Las políticas de comportamiento se cumplen con mayor frecuencia. La experiencia del usuario mejora sustancialmente.

3.4.2 El análisis de gobernanza

Pero "con mayor frecuencia" no es "siempre", y en sectores regulados la diferencia entre ambos es la diferencia entre conformidad e incumplimiento.

El problema fundamental de todas las aproximaciones de alineamiento actuales, desde RLHF hasta Constitutional AI, es que operan sobre las *distribuciones de salida* del modelo, no sobre las *representaciones de decisión*. RLHF modifica qué respuestas genera el modelo con mayor probabilidad. Constitutional AI modifica las preferencias del modelo durante el entrenamiento. Los system prompts proporcionan instrucciones que el modelo intenta seguir. Pero en ningún caso se introducen representaciones formales de las políticas que puedan verificarse antes del despliegue, que se ejecuten de forma determinista durante la operación, o que se auditen después.

Las constituciones son instrucciones que el modelo interpreta probabilísticamente — directrices, no reglas verificables.

Casper et al. (2023), en su análisis de los problemas abiertos del alineamiento basado en feedback humano, identificaron varias limitaciones fundamentales. El feedback humano es inconsistente: distintos anotadores expresan preferencias diferentes ante los mismos pares de respuestas. El modelo de recompensa es una aproximación: captura tendencias estadísticas, no reglas deterministas. Y la optimización puede producir *reward hacking*: el modelo aprende a satisfacer el modelo de recompensa sin alinearse genuinamente con los valores subyacentes.

Wolf et al. (2023) y Zou et al. (2023) demostraron que incluso los modelos más cuidadosamente alineados pueden ser inducidos a violar sus políticas mediante ataques adversariales — *jailbreaks* — que explotan la naturaleza probabilística de la generación para eludir las restricciones aprendidas. Aunque la robustez ante estos ataques ha mejorado con cada generación de modelos, la existencia misma de la vulnerabilidad demuestra que las restricciones no son formales sino estadísticas: se cumplen con alta probabilidad pero no con certeza.

Casper et al. (2023) — problemas abiertos del alineamiento basado en feedback.

Para un sistema conversacional en un sector regulado, las consecuencias son concretas. Una política que dice «no proporcionar asesoramiento financiero personalizado» puede codificarse en un system prompt, reforzarse con RLHF, y monitorizarse con guardrails. Pero si un usuario formula su solicitud de forma suficientemente indirecta, si el contexto conversacional es suficientemente largo, o si la formulación del prompt es suficientemente ambigua, el sistema puede violar la política. Y cuando eso ocurre, no hay mecanismo formal para determinar *por qué* la viola-

ción se produjo, ni para garantizar que no se repetirá en circunstancias similares.

3.4.3 El concepto de alineamiento formalizado

Alineamiento formalizado

En un sistema con **alineamiento formalizado**, las políticas de comportamiento — restricciones éticas, legales y organizacionales — están representadas en el mismo modelo formal de conocimiento que las reglas de negocio y la lógica del diálogo. Se verifican del mismo modo, se ejecutan determinísticamente, y se auditan con los mismos mecanismos. Las políticas no son instrucciones que el modelo intenta seguir sino restricciones formales que el motor simbólico impone.

La propuesta de ACAI no es que las técnicas de alineamiento actuales sean inútiles. RLHF, Constitutional AI y guardrails aportan valor real y ACAI puede incorporarlos como capas adicionales de seguridad. Lo que propone es que, para las políticas críticas — las que tienen consecuencias regulatorias, legales o éticas —, la formalización en el modelo de conocimiento ofrece garantías que las técnicas probabilísticas no pueden proporcionar.

La analogía es precisa: en un sistema legal, las leyes no son "directrices que los ciudadanos intentan seguir" sino reglas formales con condiciones de aplicación, excepciones codificadas, y mecanismos de verificación. Que los ciudadanos las cumplan voluntariamente (alineamiento) es deseable; que exista un sistema formal que las haga cumplir (gobernanza) es imprescindible. ACAI propone construir ese sistema formal para las políticas del dominio conversacional.

Políticas de generación como objeto formalizable

No todas las políticas son igualmente formalizables. Una restricción como «no ejecutar cancelaciones sin confirmación explícita» es perfectamente formalizable: es una precondición determinista. Una directriz como «ser empático cuando el usuario expresa frustración» es más difícil de formalizar porque involucra detección de estados emocionales y estrategias comunicativas graduadas. ACAI reconoce esta gradación: las políticas con consecuencias operativas directas se formalizan como reglas; las políticas de tono y estilo se gestionan mediante instrucciones al LLM gobernadas por el modelo formal (por ejemplo, el modelo formal determina

> que en este punto del diálogo se requiere tono empático, y configura el prompt del LLM en consecuencia). La frontera entre lo formalizado y lo instruido es, de nuevo, una decisión de diseño que la **UD 8** analizará en profundidad.

3.5 Gestión de contexto: del buffer de tokens al *information state*

Sede principal de este concepto.

3.5.1 El contexto en los LLMs: ventana, degradación y opacidad

Un sistema conversacional multi-turno necesita "recordar" lo que se ha dicho a lo largo de la interacción: qué ha solicitado el usuario, qué información se ha proporcionado, qué compromisos se han adquirido, qué preguntas están pendientes. En los LLMs, este "recuerdo" se implementa mediante la ventana de contexto: la secuencia de tokens — tanto del usuario como del sistema — que el modelo procesa en cada paso de generación.

La ventana de contexto ha crecido espectacularmente en los últimos años: de los 2.048 tokens de GPT-3 a los 128.000 de GPT-4 Turbo y los 200.000 de Claude 3, con propuestas experimentales que alcanzan millones de tokens. Este crecimiento ha generado la percepción de que el problema de la gestión de contexto está resuelto: si el modelo puede procesar toda la conversación, "recuerda" todo.

Una ventana de contexto más grande no es una representación más rica — es un buffer más largo con las mismas limitaciones estructurales.

Sin embargo, esta percepción confunde *acceso* con *representación*. El modelo tiene *acceso* a todos los tokens de la conversación, pero no tiene una *representación* del estado del diálogo. No distingue entre información mencionada y confirmada. No registra qué compromisos se han adquirido ni cuáles se han cumplido. No modela las subtareas abiertas ni las cerradas. No diferencia entre lo que el usuario pidió inicialmente y lo que pidió después de cambiar de opinión. Todos estos estados existen implícitamente en la secuencia de tokens, y el modelo puede inferirlos probabilísticamente con razonable fiabilidad en contextos cortos. Pero la inferencia es falible, y su fiabilidad se degrada con la longitud.

Liu et al. (2023) demostraron experimentalmente el efecto "lost in the middle": la capacidad de los LLMs para usar información del contexto decae significativamente para información situada en posiciones intermedias de contextos largos, mientras que la información al principio y al final se procesa con mayor fiabilidad. Levy et al. (2024) extendieron este análisis y mostraron que la degradación afecta especialmente a la

recuperación de hechos específicos y a la consistencia de razonamiento en contextos que exceden los 20.000 tokens.

Para un sistema conversacional accountable, las consecuencias son directas. Si un usuario proporciona su número de póliza en el turno 3, cambia su solicitud en el turno 7, y confirma la operación en el turno 15, el sistema necesita mantener un registro preciso de estos tres hechos y de sus relaciones. Un LLM con ventana de contexto suficiente probablemente los "recuerde", pero no puede garantizarlo, y la traza de cómo estos hechos influyeron en la decisión final no es inspectable.

Liu et al. (2023): "lost in the middle". Levy et al. (2024): degradación del razonamiento en contextos largos.

3.5.2 La tradición del *information state update*

La investigación en gestión de diálogo produjo, décadas antes de los LLMs, una respuesta formal a este problema. El paradigma de **information state update** (ISU), articulado por Larsson y Traum (2000) en el marco del proyecto TRINDI y formalizado en el toolkit TrindiKit, propone que un sistema de diálogo mantenga un modelo explícito del estado informacional del diálogo — el **information state** — que se actualiza de forma controlada con cada turno de la conversación.

Larsson & Traum (2000) — TrindiKit y el paradigma de *information state update*.

El information state, tal como lo formularon Larsson y Traum, no es simplemente una variable que almacena datos. Es una estructura formal que incluye varios componentes. Un componente de *agenda* contiene los objetivos pendientes del sistema y del usuario, organizados por prioridad. Un componente de *shared commitments* registra los compromisos mutuos adquiridos durante el diálogo y su estado (propuesto, aceptado, cumplido, retirado). Un componente de *questions under discussion* (QUD) mantiene las preguntas que están abiertas y que el diálogo está intentando resolver. Un componente de *private beliefs* registra lo que el sistema cree saber sobre el usuario y la situación.

Las reglas de actualización definen cómo se transforma el information state con cada acción del diálogo. Cuando el usuario proporciona información, se actualiza el componente de creencias privadas. Cuando el sistema confirma un dato, pasa de creencia privada a compromiso compartido. Cuando se abre una subtarea, se añade a la agenda. Cuando se cierra, se retira. Estas reglas son formales, deterministas, e inspectables.

QUD: *Questions Under Discussion* (Ginzburg, 2012) — las preguntas abiertas que organizan el diálogo.

Ginzburg (2012), en su trabajo sobre la semántica del diálogo, extendió este marco con una formalización más profunda basada en *Type Theory with Records* (TTR), que permite representar el contexto conversacional con la precisión de un sistema de tipos formales. Aunque la implementación completa de TTR excede lo que ACAI necesita en la Fase 1, la idea central — el contexto conversacional como estructura formal con semántica definida — es directamente relevante.

3.5.3 El *information state* en la arquitectura ACAI

La arquitectura ACAI adopta el concepto de information state como componente central del motor simbólico. En cada turno de la conversación, el information state se actualiza de forma explícita según reglas formalizadas en el modelo de conocimiento. El LLM no tiene acceso directo al information state — interactúa con él a través del motor simbólico: recibe del motor las instrucciones sobre qué interpretar y cómo generar, y el motor actualiza el information state según lo que el LLM ha interpretado.

> **_Information state_ en ACAI**
>
> El **information state** de ACAI es una estructura formal, mantenida y actualizada por el motor simbólico, que representa el estado completo del diálogo en un momento dado. Incluye, como mínimo: los datos proporcionados por el usuario y su estado de confirmación; los compromisos adquiridos por el sistema; las subtareas abiertas y cerradas; las condiciones de *grounding* pendientes; y los resultados de las acciones ejecutadas. Se actualiza determinísticamente con cada turno y es completamente inspectable.

La ventaja respecto a la ventana de contexto del LLM es categórica en tres aspectos. En *persistencia*: el information state no se degrada con la longitud de la conversación; un dato confirmado en el turno 3 tiene el mismo estatus en el turno 30. En *estructura*: el information state distingue entre tipos de información (datos, compromisos, preguntas abiertas, subtareas) mientras que la ventana de contexto los mezcla como tokens indiferenciados. Y en *inspección*: el information state es una estructura que puede imprimirse, visualizarse y auditarse en cualquier momento de la conversación; la ventana de contexto es una secuencia de tokens cuyo "significado" para el modelo es inaccesible.

> **El *information state* como pilar de la trazabilidad**
>
> El information state no es un componente auxiliar de la arquitectura ACAI sino su pilar central. Si ACAI promete trazabilidad — la capacidad de reconstruir por qué el sistema tomó cada decisión —, el information state es el objeto que hace posible esa reconstrucción. Cada decisión del motor simbólico opera sobre el information state vigente en ese momento. La traza de decisión es, literalmente, la secuencia de information states y las reglas que produjeron cada transición.

3.6 Síntesis: por qué la combinación no basta

3.6.1 El problema de la coordinación

El análisis de las secciones anteriores podría sugerir que los problemas de gobernanza son específicos de cada componente y que, por tanto, podrían resolverse mejorando cada componente por separado: un RAG más preciso, un agente más fiable, un alineamiento más robusto, un contexto más largo. Pero hay un problema adicional que emerge cuando los componentes se combinan y que no es reducible a las limitaciones de ninguno de ellos por separado: el problema de la coordinación.

La combinación de componentes no gobernados no produce un sistema gobernado — produce un sistema con múltiples fuentes de no-gobernabilidad.

En un sistema conversacional que combina RAG, agentes y guardrails, cada componente toma decisiones de forma independiente. El módulo RAG decide qué documentos recuperar. El agente decide qué herramientas invocar. El guardrail decide qué filtrar. El LLM integra todo y genera una respuesta. Pero no existe un modelo compartido que coordine estas decisiones. El RAG puede recuperar un documento que contradice una acción que el agente está a punto de ejecutar. El agente puede invocar una herramienta que el guardrail luego bloqueará, produciendo un flujo incoherente. El LLM puede generar una respuesta que integra información del RAG de una forma que viola una política del guardrail.

La literatura sobre sistemas multi-componente ha documentado extensamente este problema bajo diversas etiquetas. En ingeniería de software, se conoce como el problema de la *composición emergente*: el comportamiento del sistema compuesto no es predecible a partir del comportamiento de los componentes individuales (Ghezzi, Jazayeri y Mandrioli, 2003). En sistemas multiagente, se conoce como el problema de la *coordinación*: garantizar que agentes autónomos actúen de forma coherente requiere mecanismos explícitos que los agentes autónomos, por definición, no proporcionan.

Para un sistema conversacional accountable, el problema de la coordinación tiene una consecuencia directa: la trazabilidad end-to-end es imposible cuando los componentes no comparten un modelo formal de lo que están haciendo. Si el RAG decide buscar información, el agente decide ejecutar una acción, y el LLM decide cómo responder, y cada decisión se toma de forma independiente, no existe un punto de vista privilegiado desde el cual reconstruir la cadena completa de razonamiento que produjo la respuesta.

3.6.2 La gobernanza formal unificada

Phase 1 e I5
Gobernanza formal del ecosistema operativo.

Gobernanza formal unificada

La **gobernanza formal unificada** es el principio según el cual todos los componentes del ecosistema operativo — recuperación de información, ejecución de acciones, aplicación de políticas, y gestión de contexto — están gobernados por el mismo modelo formal de conocimiento y coordinados por el mismo motor simbólico. La traza de ejecución es completa y coherente porque refleja las decisiones de un único proceso formal, no la combinación de decisiones independientes.

En la arquitectura ACAI, el motor simbólico es el componente que implementa la gobernanza unificada. Ante cada turno de la conversación, el motor consulta el modelo de conocimiento y el information state, y determina qué debe ocurrir: ¿se necesita información documental? El motor determina qué buscar y en qué fuentes (RAG gobernado). ¿Se debe ejecutar una acción? El motor determina cuál, verifica las precondiciones, y ordena la ejecución (capacidad agéntica gobernada). ¿Hay restricciones de política que aplican? El motor las evalúa formalmente (alineamiento formalizado). ¿Cómo se actualiza el estado del diálogo? El motor aplica las reglas de actualización (information state).

El LLM participa en este proceso en dos puntos: al principio, interpretando el input del usuario y mapeándolo a conceptos del modelo formal; y al final, generando la respuesta en lenguaje natural conforme a lo que el motor ha determinado. Entre ambos puntos, las decisiones son formales, deterministas y trazables.

Gobernanza unificada en acción

Turno: El usuario ha solicitado cancelar su póliza de hogar y pregunta por la bonificación.
Motor simbólico: Consulta el information state → la solicitud de cancelación está registrada pero pendiente de confirmación. La pregunta sobre bonificación abre una subtarea. El motor evalúa:
Paso 1: ¿Se necesita información documental? *Sí* — la regla `bonus_info_required` determina que hay que consultar las condiciones de bonificación en las Condiciones Generales vigentes (RAG gobernado).
Paso 2: ¿Se necesita ejecutar una acción? *Sí* — la regla `bonus_check_action` determina que hay que consultar el historial de siniestros del cliente en el sistema de gestión (capacidad agéntica gobernada). Precondición: identidad verificada → verificada en turno anterior → proceder.
Paso 3: ¿Aplican restricciones de política? *Sí* — la política `retention_opportunity` indica que ante una cancelación se debe informar proactivamente de las opciones de retención (alineamiento formalizado).
Paso 4: Actualizar information state: subtarea

`bonificación_consulta` abierta, datos de bonificación obtenidos, opciones de retención pendientes de comunicar.
LLM (NLG): Genera la respuesta integrando: información de bonificación, datos de historial del cliente, y mención de opciones de retención, con tono según la política de comunicación.
Traza: Cada paso registra la regla aplicada, los datos consultados, las fuentes accedidas, las precondiciones verificadas, las políticas evaluadas, y el information state resultante.

Este escenario ilustra la diferencia fundamental entre un sistema que combina componentes autónomos y un sistema con gobernanza unificada. En el primer caso, cada componente actúa según su propia lógica y la coherencia del resultado es accidental. En el segundo, un único modelo formal coordina todos los componentes y la coherencia es garantizada. La traza del primer caso es fragmentaria (el log de cada componente por separado); la traza del segundo es completa (la secuencia de decisiones formales con sus justificaciones).

3.6.3 Lo que la gobernanza unificada no resuelve

Límites de la gobernanza unificada

La gobernanza formal unificada no hace que el sistema sea perfecto. Hace que sea *auditable*. Si las reglas están mal diseñadas, el sistema tomará decisiones incorrectas — pero la traza permitirá identificar qué regla causó el error y corregirla. Si el LLM interpreta incorrectamente un input, la respuesta será incorrecta — pero la traza mostrará dónde se produjo el error de interpretación. Si un servicio externo falla, la acción no se completará — pero el motor gestionará el fallo según las reglas formalizadas. La gobernanza unificada transforma los errores de *opacos* en *diagnosticables*. No los elimina; los hace gobernables.

Conexiones con otras unidades

→ **Prerequisito: UD 1** (Mapa conceptual ACAI) para los principios P4 (completitud operativa) y los componentes E1–E4. **UD 2** (La conversación como fenómeno) para el diagnóstico de la brecha de representación que esta unidad complementa.

⇄ **Complementa: UD 4** (Accountability y marco regulatorio) fundamenta desde la perspectiva regulatoria los requisitos de trazabilidad y verificabilidad que aquí se analizan desde la perspectiva

tecnológica. UD3 muestra que los componentes actuales no satisfacen los requisitos; UD4 muestra qué exigen exactamente esos requisitos.

⇄ Complementa: UD 5 (Fundamentos neuro-simbólicos) proporciona el marco teórico para la integración motor-LLM que aquí se plantea como solución. UD3 establece qué necesita gobernarse; UD5 explica cómo el paradigma neuro-simbólico fundamenta esa gobernanza.

➔ Aplicación: UD 8 (Dimensiones del conocimiento conversacional) toma el ecosistema operativo como una de las dimensiones que el formalismo debe gobernar: las reglas sobre cuándo buscar, qué acciones ejecutar, y qué políticas aplicar son parte del conocimiento conversacional que se formaliza.

➔ Aplicación: UD 9 (Formalismos) evalúa los formalismos candidatos considerando, entre otros criterios, su capacidad para representar las reglas de gobernanza del ecosistema operativo: reglas de recuperación, precondiciones de acciones, restricciones de políticas.

Ejercicios

1. [Análisis técnico] Seleccione un sistema conversacional comercial con el que haya interactuado (un chatbot de atención al cliente, un asistente de banca, o similar). Diseñe una secuencia de 5–7 turnos que involucre acceso a información documental, ejecución de una acción, y una situación donde una política debería aplicarse. Para cada turno, identifique: (a) qué componente del ecosistema operativo se necesitaría, (b) qué decisión toma ese componente, (c) quién toma esa decisión en el sistema actual (¿el LLM?, ¿un pipeline?, ¿una regla?), y (d) si esa decisión es auditable.

2. [Diseño] Para el escenario de la aseguradora que recorre el monográfico, diseñe las reglas de un RAG gobernado para la consulta de condiciones de bonificación. Especifique: (a) bajo qué condiciones del diálogo se activa la recuperación, (b) en qué fuentes se busca y con qué criterios, (c) cómo se valida la pertinencia del resultado, (d) cómo se atribuye el nivel de confianza, y (e) qué se registra en la traza. Presente las reglas en pseudocódigo o en una notación semiformal de su elección.

3. [Análisis comparativo] Compare la gestión de contexto en tres paradigmas: (a) un sistema de diálogo con information state explícito

al estilo de TrindiKit, (b) un LLM con ventana de contexto de 128k tokens, y (c) un LLM con RAG sobre el historial de conversaciones. Para cada paradigma, analice: cómo se representa un compromiso adquirido (por ejemplo, "el sistema se comprometió a no ejecutar la cancelación sin confirmación"), cómo se garantiza su persistencia a lo largo del diálogo, y cómo se verifica su cumplimiento.

4. [Reflexión crítica] El concepto de "agente gobernado" podría parecer una contradicción en los términos: un agente que no decide autónomamente no es un agente. Evalúe este argumento. ¿Es la autonomía una propiedad esencial de la agencia? ¿Puede un sistema tener capacidad de acción sin tener autonomía de decisión? ¿Cuáles son los trade-offs de restringir la autonomía del agente? Construya su argumento con referencia a la literatura sobre agentes (por ejemplo, Wooldridge y Jennings, 1995, sobre las propiedades de los agentes) y a los requisitos de accountability en sectores regulados.

5. [Aplicación a ACAI] Diseñe el flujo completo de un turno conversacional en la arquitectura ACAI para el siguiente escenario: el usuario dice «Mi vecino me ha dicho que si cancelo antes de fin de mes no pago penalización, ¿es verdad?». El flujo debe especificar: (a) cómo interpreta el LLM este input (incluyendo la referencia a información externa no verificada), (b) qué decide el motor simbólico, (c) qué componente del ecosistema se activa y por qué, (d) qué políticas aplican (incluyendo la política de no proporcionar asesoramiento basado en información de terceros), (e) cómo se genera la respuesta, y (f) qué registra la traza. Use el formato de escenario de esta unidad.

Referencias y lecturas

Lectura de entrada

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J. & Wang, H. (2024). Retrieval-Augmented Generation for Large Language Models: A Survey. *ArXiv preprint arXiv:2312.10997v5*. El survey más completo y actualizado sobre RAG, con una taxonomía clara de paradigmas (naive, advanced, modular) y un análisis de técnicas en cada etapa del pipeline. Lectura de entrada recomendada para situar el estado del arte antes de abordar el análisis de gobernanza de esta unidad.

Lecturas esenciales

- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S. & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*. — El artículo fundacional de RAG. Imprescindible para entender la formulación original y su motivación.
- Larsson, S. & Traum, D. (2000). Information state and dialogue management in the TRINDI dialogue move engine toolkit. *Natural Language Engineering*, 6(3–4), 323–340. — El artículo de referencia para el paradigma de information state update. Base directa del concepto de information state en ACAI. Lectura imprescindible.
- Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C. et al. (2022). Constitutional AI: Harmlessness from AI Feedback. *Anthropic Technical Report*. — La formulación de Constitutional AI. Esencial para entender el patrón de alineamiento que ACAI propone formalizar.
- Casper, S., Davies, X., Shi, C., Gilbert, T. K., Scheurer, J., Rando, J., Freedman, R., Korbak, T., Lindner, D., Freire, P., Wang, T., Marks, S., Segerie, C.-R., Carroll, M., Peng, A., Christoffersen, P., Damani, M., Shlegeris, B., Greenblatt, N., Baumli, K., McIlhenny, L., Russell, S. & Abbeel, P. (2023). Open Problems and Fundamental Limitations of Reinforcement Learning from Human Feedback. *Transactions on Machine Learning Research*. — El análisis más riguroso de las limitaciones del alineamiento basado en feedback. Esencial para fundamentar la necesidad de formalización.

Lecturas complementarias

- Barnett, S., Kurniawan, S., Thudumu, S., Brber, Z. & Vasa, R. (2024). Seven Failure Points When Engineering a Retrieval Augmented Generation System. *IEEE/ACM International Conference on AI Engineering*. — Análisis empírico de los puntos de fallo de RAG en producción. Directamente relevante para la argumentación sobre gobernanza.
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z. & Wen, J. (2024). A Survey on Large Language Model based Autonomous Agents. *Frontiers of Computer Science*, 18(6). — Survey comprensivo sobre agentes LLM. Referencia para la taxonomía de capacidades agénticas.
- Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E. et al. (2023). The Rise and Potential of Large Language Model Based Agents: A Survey. *ArXiv preprint arXiv:2309.07864*. — Otro survey influyente sobre agentes. Complementa a Wang et al. con más énfasis en arquitecturas multi-agente.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR 2023*. — El patrón de razonamiento-acción intercalado. Referencia para entender la aproximación agéntica actual.

- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N. & Scialom, T. (2024). Toolformer: Language Models Can Teach Themselves to Use Tools. *NeurIPS 2023*. — El trabajo que formalizó el concepto de tool use para LLMs.
- Patil, S. G., Zhang, T., Wang, X. & Gonzalez, J. E. (2023). Gorilla: Large Language Model Connected with Massive APIs. *ArXiv preprint arXiv:2305.15334*. — Análisis de errores en selección de herramientas por LLMs. Evidencia empírica para la argumentación sobre gobernanza agéntica.
- Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., Dubois, Y., Maddison, C. J. & Hashimoto, T. (2024). Identifying the Risks of LM Agents with an LM-Emulated Sandbox. *ICLR 2024*. — Análisis de riesgos de agentes autónomos. Evidencia empírica directa.
- Shuster, K., Poff, S., Chen, M., Kiela, D. & Weston, J. (2021). Retrieval Augmentation Reduces Hallucination in Conversation. *EMNLP 2021 (Findings)*. — Evidencia de la reducción de alucinaciones con RAG.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A. et al. (2022). Training language models to follow instructions with human feedback. *NeurIPS 2022*. — InstructGPT y RLHF a escala.
- Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D. & Finn, C. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. *NeurIPS 2023*. — DPO como alternativa a RLHF.
- Rebedea, T., Dinu, R., Sreedhar, M., Parisien, C. & Cohen, J. (2023). NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. *EMNLP 2023 (demo)*. — Framework de guardrails programables.
- Inan, H., Upasani, K., Chi, J., Rungta, R., Iyer, K., Mao, Y., Tontchev, M., Hu, Q., Fuller, B., Testuggine, D. & Khabsa, M. (2023). Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. *ArXiv preprint arXiv:2312.06674*. — Clasificador de seguridad como guardrail.
- Wolf, Y., Hadar, N., Bitton, I., Berber, S. & Elhadad, N. (2023). Fundamental Limitations of Alignment in Large Language Models. *ArXiv preprint arXiv:2304.11082*. — Análisis teórico de las limitaciones del alineamiento.
- Zou, A., Wang, Z., Carlini, N., Nasr, M., Kolter, J. Z. & Fredrikson, M. (2023). Universal and Transferable Adversarial Attacks on Aligned Language Models. *ArXiv preprint arXiv:2307.15043*. — Ataques adversariales contra modelos alineados.
- Ginzburg, J. (2012). *The Interactive Stance: Meaning for Conversation*. Oxford University Press. — Formalización del contexto conversacional con TTR. Referencia avanzada para la fundamentación teórica del information state.
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F. & Liang, P. (2023). Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the ACL*. — Evidencia de degradación del uso de contexto.

- Levy, M., Jacoby, A. & Goldberg, Y. (2024). Same Task, More Tokens: the Impact of Input Length on the Reasoning Performance of Large Language Models. *ACL 2024*. — Extensión del análisis de degradación en contextos largos.
- Ghezzi, C., Jazayeri, M. & Mandrioli, D. (2003). *Fundamentals of Software Engineering* (2.ª ed.). Prentice Hall. — Referencia para el problema de la composición emergente en sistemas de software.

MONOGRÁFICO FORMATIVO

Fundamentos de IA Conversacional Neuro-Simbólica

«An explanation that does not correspond to the actual decision process is not an explanation — it is a story.»

Eje A — El problema: IA conversacional y accountability

Unidad 4 de 12

Research Sprint ACAI · Proyecto $(AI)^2$:TDG · 2026

Eje A — El problema

Prerequisitos: UD1, UD2 · Eje A — El problema

UNIDAD DIDÁCTICA 4

Accountability, explicabilidad y marco regulatorio

Contenidos de la unidad

4.1.1 El problema terminológico 3
4.1.2 Transparencia 4
4.1.3 Interpretabilidad 5
4.1.4 Explicabilidad 5
4.1.5 Accountability 6
4.1.6 Relaciones entre los conceptos 7
4.2.1 El paradigma post-hoc 8
4.2.2 Las explicaciones de los LLMs: racionalizaciones, no explicaciones 9
4.2.3 El paradigma constitutivo 10
4.2.4 El argumento de Rudin y su extensión a la IA conversacional 11
4.3.1 De la regulación a las especificaciones técnicas 12
4.3.2 EU AI Act: requisitos para sistemas de alto riesgo 12
4.3.3 DORA: resiliencia operativa en el sector financiero 13
4.3.4 GDPR: el derecho a explicación 14
4.3.5 Síntesis: la tabla de requisitos técnicos 14
4.4.1 Dos estrategias de gobernanza 15
4.4.2 La analogía estructural 16
4.5.1 La filosofía del proyecto marco 16
4.5.2 ACAI como instanciación de la filosofía 17

¿Por qué necesito esto para ACAI?

Las unidades anteriores han establecido el diagnóstico: la conversación es un fenómeno multidimensional que las arquitecturas actuales generan sin representar (UD 2), y los componentes tecnológicos existentes carecen de las propiedades de gobernanza necesarias (UD 3). Pero este diagnóstico está incompleto sin una respuesta precisa a la pregunta fundamental: ¿qué significa, exactamente, que un sistema conversacional sea *accountable*? Y, más concretamente: ¿qué exige el marco regulatorio actual? Esta unidad proporciona ambas respuestas. Al terminar, el investigador tendrá una definición operativa de accountability, distinguirá con precisión entre explicabilidad post-hoc y constitutiva, y podrá traducir los requisitos regulatorios en especificaciones técnicas para la arquitectura ACAI.

Objetivos de aprendizaje

Al completar esta unidad, el investigador será capaz de:

1. Distinguir con precisión entre transparencia, interpretabilidad, explicabilidad y accountability, identificando las relaciones de dependencia entre estos conceptos.
2. Argumentar la diferencia entre explicabilidad post-hoc y constitutiva, y evaluar por qué la segunda es necesaria para la accountability en sectores regulados.
3. Traducir los requisitos del EU AI Act, DORA y GDPR en especificaciones técnicas concretas para un sistema conversacional.
4. Distinguir entre gobernanza superpuesta y gobernanza constitutiva, y explicar por qué ACAI adopta la segunda.
5. Conectar la propuesta de ACAI con la filosofía del proyecto (AI)^2:TDG sobre explicabilidad fundamentada en estructuras formales.

4.1 La constelación de conceptos: de la transparencia a la accountability

Sede principal de este concepto.

4.1.1 El problema terminológico

La literatura sobre IA explicable y responsable utiliza una constelación de términos — transparencia, interpretabilidad, explicabilidad,

Seis términos, usados como sinónimos en la industria, designan propiedades técnicas distintas con requisitos de implementación diferentes.

auditabilidad, accountability, responsabilidad — que en la práctica industrial y en buena parte de la divulgación se emplean de forma intercambiable. Esta confusión terminológica no es un problema menor: cada término designa una propiedad técnica diferente, con requisitos de implementación diferentes, y un sistema puede satisfacer unos sin satisfacer otros. ACAI necesita una definición operativa precisa de su objetivo, y esa definición requiere desambiguar la constelación.

Arrieta et al. (2020), en su survey comprensivo sobre Explainable AI, propusieron una taxonomía que ha servido como referencia para buena parte del trabajo posterior. Lipton (2018), en un trabajo anterior e igualmente influyente, había advertido que el término «interpretabilidad» se usaba para designar al menos tres propiedades diferentes, y que esta ambigüedad generaba confusión tanto en la investigación como en la práctica. Doshi-Velez y Kim (2017) intentaron una formalización más rigurosa, proponiendo niveles de evaluación de la interpretabilidad. Más recientemente, Schwalbe y Finzel (2023) han revisado la taxonomía a la luz de los desarrollos en LLMs y modelos generativos.

Arrieta et al. (2020) — taxonomía comprensiva de XAI.

Lo que sigue no es un catálogo de definiciones sino una construcción progresiva: cada concepto se apoya en el anterior y añade una exigencia adicional. Esta estructura no es solo pedagógica — refleja una relación de dependencia real entre las propiedades.

4.1.2 Transparencia

La **transparencia** es la propiedad más básica de la cadena. Un sistema es transparente cuando su estructura interna y su proceso de operación son accesibles para su inspección. Lipton (2018) distinguió tres niveles de transparencia: la transparencia del modelo completo (*simulability* — un humano puede simular mentalmente el proceso completo), la transparencia de los componentes (*decomposability* — cada componente tiene una interpretación inteligible), y la transparencia algorítmica (el algoritmo de entrenamiento produce modelos cuyo comportamiento es comprensible).

Un árbol de decisión con pocas ramas es transparente en los tres niveles. Un LLM no es transparente en ninguno: su estructura interna (miles de millones de parámetros) no es simulable, sus componentes (capas de atención, matrices de pesos) no tienen interpretación semántica individual, y el proceso de entrenamiento no produce modelos cuyo comportamiento sea comprensible a priori.

Lipton (2018): tres niveles de transparencia — simulability, decomposability, transparencia algorítmica.

La transparencia es condición necesaria pero no suficiente para la accountability. Un sistema puede ser transparente sin ser explicable: que los parámetros sean accesibles no significa que sean comprensibles. Y un sistema puede ser parcialmente explicable sin ser transparente:

las técnicas post-hoc generan explicaciones de sistemas opacos. Pero la accountability, como veremos, requiere una forma de transparencia que solo las representaciones explícitas pueden proporcionar.

4.1.3 Interpretabilidad

La **interpretabilidad** añade a la transparencia el requisito de comprensibilidad humana. Un sistema es interpretable cuando un humano puede comprender cómo produce sus decisiones. La distinción con la transparencia es sutil pero importante: la transparencia es una propiedad del sistema (está abierto a inspección), mientras que la interpretabilidad es una propiedad de la relación entre el sistema y un observador humano (el observador puede comprender lo que inspecciona).

Doshi-Velez y Kim (2017) propusieron evaluar la interpretabilidad en tres niveles: evaluación a nivel de aplicación (¿es útil la explicación para un usuario final en su tarea?), evaluación a nivel humano (¿es comprensible la explicación para un evaluador no experto?), y evaluación a nivel de función (¿produce la explicación resultados correctos según métricas formales?). Estos niveles son directamente relevantes para ACAI, porque el lenguaje de representación debe satisfacer la restricción de legibilidad (I1), que es esencialmente un requisito de interpretabilidad: un experto de dominio no técnico debe poder comprender las reglas formalizadas.

Molnar (2022), en su libro de referencia sobre interpretable machine learning, ofrece una distinción práctica que es útil para ACAI: los modelos *intrínsecamente* interpretables (modelos lineales, árboles de decisión, sistemas de reglas) versus los modelos que requieren técnicas de interpretabilidad *post-hoc* (redes neuronales profundas, LLMs). La arquitectura neuro-simbólica de ACAI combina ambos: el motor simbólico opera sobre reglas que son intrínsecamente interpretables; el LLM es opaco pero sus decisiones no son las que deben ser accountable — solo proporciona capacidad lingüística.

📖 Doshi-Velez & Kim (2017) — niveles de evaluación de la interpretabilidad.

4.1.4 Explicabilidad

La **explicabilidad** añade a la interpretabilidad el requisito de articulación causal: no solo se puede comprender qué hizo el sistema, sino *por qué* lo hizo. Arrieta et al. (2020) definen la explicabilidad como la capacidad de producir detalles o razones que hagan claro el funcionamiento del modelo a un observador humano, incluyendo las relaciones causales entre las entradas, los estados internos, y las salidas.

La distinción con la interpretabilidad es que un sistema puede ser interpretable (puedo comprender las reglas) sin ser explicable (no puedo

reconstruir por qué, en este caso concreto, se aplicó esta regla y no otra). La explicabilidad requiere que el sistema pueda articular la cadena causal que conecta la entrada con la salida, pasando por las decisiones intermedias y las reglas aplicadas.

Para un sistema conversacional en un sector regulado, la explicabilidad tiene un destinatario concreto que no es el usuario final sino el auditor: un profesional técnico o regulatorio que necesita verificar que el sistema operó correctamente en un caso específico. El auditor no pregunta «¿qué hizo el sistema?» (interpretabilidad) sino «¿por qué hizo lo que hizo, y era correcto hacerlo según las normas aplicables?» (explicabilidad). Esta pregunta requiere acceso a la cadena causal completa: qué input recibió, cómo lo interpretó, qué reglas aplicó, qué información consultó, qué precondiciones verificó, y por qué produjo esta respuesta y no otra.

4.1.5 Accountability

Accountability

La **accountability** es la propiedad de un sistema cuyas decisiones son explicables (se puede articular por qué se tomaron), verificables (se puede comprobar que cumplen una especificación formal), y atribuibles (se puede identificar qué componente, regla o agente es responsable de cada decisión). La accountability presupone transparencia, interpretabilidad y explicabilidad, y añade los requisitos de verificabilidad formal y atribución de responsabilidad.

La accountability es explicabilidad + verificabilidad + atribución de responsabilidad. Es el concepto más exigente de la cadena.

La verificabilidad distingue a la accountability de la mera explicabilidad. Un sistema puede ser explicable sin ser verificable: puede producir una explicación plausible que, sin embargo, no se puede contrastar contra una especificación formal. Un LLM que genera una explicación de su propia decisión produce un artefacto lingüístico que puede ser coherente y comprensible, pero que no se puede verificar contra reglas formales porque esas reglas no existen en el sistema. La accountability exige que exista una especificación — las reglas formalizadas en el modelo de conocimiento — y que la traza de decisión pueda contrastarse contra ella.

La atribución de responsabilidad distingue a la accountability de la verificabilidad abstracta. En un sistema complejo, no basta con verificar que el comportamiento cumple la especificación; es necesario poder determinar, en caso de fallo, *qué* falló: ¿fue la regla (está mal diseñada)? ¿Fue la interpretación del LLM (malinterpretó al usuario)? ¿Fue la información (el documento recuperado estaba desactualizado)?

¿Fue la acción (el servicio externo respondió incorrectamente)? Sin esta capacidad de atribución, la corrección de errores es ciega.

Referencia contextual. Este concepto se desarrolla en profundidad en **UD 12**. Las métricas operativas de accountability — consistencia, trazabilidad, verificabilidad, gobernabilidad — y las metodologías para medirlas se desarrollan en profundidad en la unidad de evaluación.

4.1.6 Relaciones entre los conceptos

La construcción progresiva revela una cadena de dependencias que puede sintetizarse así. La transparencia (acceso a la estructura interna) es condición necesaria para la interpretabilidad (comprensión de la estructura). La interpretabilidad es condición necesaria para la explicabilidad (articulación causal). La explicabilidad es condición necesaria para la accountability, que además requiere verificabilidad (contrastación con especificación) y atribución de responsabilidad.

Las propiedades como niveles de exigencia

Cada nivel de la cadena impone requisitos técnicos adicionales sobre el sistema. La transparencia requiere que la estructura sea accesible. La interpretabilidad requiere que sea comprensible por un humano. La explicabilidad requiere que la cadena causal sea articulable. La accountability requiere además que exista una especificación formal contra la cual verificar, y un mecanismo de atribución de responsabilidad.

Un sistema basado en LLM puro no satisface ningún nivel de la cadena para las decisiones operativas (aunque el texto generado pueda ser comprensible, el proceso de decisión no lo es). Un sistema con guardrails satisface parcialmente la transparencia (las reglas del guardrail son inspeccionables) pero no la accountability (las decisiones del LLM siguen siendo opacas).

La arquitectura ACAI está diseñada para satisfacer la accountability completa para las decisiones operativas: el motor simbólico opera sobre reglas transparentes, interpretables, que producen trazas explicables, verificables contra la especificación formal, y atribuibles a reglas y componentes concretos.

Implicaciones de la taxonomía

La taxonomía establece que la accountability no es un sinónimo elegante de explicabilidad. Es un concepto estrictamente más exigente que requiere verificabilidad formal y atribución. Esto tiene

una consecuencia directa para ACAI: no basta con que el sistema pueda explicar lo que hace (lo que un LLM puede hacer, al menos superficialmente). Es necesario que las decisiones puedan verificarse contra una especificación formal y que los fallos puedan atribuirse a componentes concretos. Estas dos propiedades adicionales son las que exigen representaciones formales explícitas.

4.2 Explicabilidad post-hoc vs. constitutiva

Sede principal de este concepto.

4.2.1 El paradigma post-hoc

La gran mayoría del trabajo en IA explicable ha operado bajo un paradigma que puede denominarse *post-hoc*: dado un modelo opaco que ya ha tomado una decisión, generar una explicación de esa decisión. Las técnicas más influyentes de este paradigma son bien conocidas. LIME (Ribeiro, Singh y Guestrin, 2016) genera explicaciones locales perturbando la entrada y observando cómo cambia la salida, construyendo un modelo interpretable que aproxima el comportamiento del modelo opaco en el entorno de la instancia explicada. SHAP (Lundberg y Lee, 2017) usa valores de Shapley de la teoría de juegos cooperativos para asignar a cada *feature* de la entrada una contribución a la predicción. Los métodos basados en atención (Vaswani et al., 2017; Jain y Wallace, 2019) usan los pesos de atención del transformer como indicadores de qué partes de la entrada influyeron más en la salida.

Ribeiro et al. (2016) — LIME. Lundberg & Lee (2017) — SHAP.

Estas técnicas han producido avances reales en la comprensión de modelos de machine learning, y siguen siendo herramientas valiosas para el análisis y la depuración de modelos. Pero como fundamento para la accountability, tienen una limitación que no es técnica sino epistemológica: la explicación que producen no es una descripción del proceso de decisión real del modelo sino una *aproximación interpretable* de ese proceso.

Esta limitación ha sido documentada con rigor creciente. Rudin (2019), en un artículo que ha tenido enorme influencia bajo el título programático «Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead», argumentó que las explicaciones post-hoc son inherentemente no fiables para decisiones de alto riesgo. Su argumento central es que una explicación post-hoc no tiene por qué corresponder al proceso real de decisión: es una reconstrucción que se ajusta a los datos observables (entradas y

Una explicación post-hoc no describe el proceso real de decisión — construye una narrativa plausible sobre él.

salidas) pero que puede ser consistente con múltiples procesos internos diferentes. Dos explicaciones post-hoc diferentes pueden ser igualmente compatibles con el comportamiento observado, y no hay forma de determinar cuál corresponde al proceso real.

Jain y Wallace (2019) demostraron experimentalmente que los pesos de atención no son explicaciones fiables: modelos con distribuciones de atención muy diferentes pueden producir las mismas predicciones, lo que significa que la atención no es una ventana al proceso de decisión sino un artefacto del mecanismo computacional. Adebayo et al. (2018) mostraron que varios métodos de explicación basados en saliencia producen mapas visualmente similares para modelos entrenados y para modelos con pesos aleatorios, cuestionando profundamente su validez como explicaciones.

Rudin (2019) — el argumento contra las explicaciones post-hoc para decisiones de alto riesgo.

4.2.2 Las explicaciones de los LLMs: racionalizaciones, no explicaciones

La aparición de los LLMs ha añadido una nueva modalidad de explicación post-hoc: el modelo genera, en lenguaje natural, una explicación de su propia respuesta. Cuando se le pide a un LLM que explique por qué ha dado una respuesta concreta, produce un texto coherente, articulado, y aparentemente razonable. La técnica de *chain-of-thought prompting* (Wei et al., 2022) explota esta capacidad pidiendo al modelo que razone paso a paso antes de producir la respuesta final.

Sin embargo, una serie de trabajos recientes ha demostrado que estas explicaciones son *racionalizaciones*: narrativas plausibles construidas para justificar la respuesta, no descripciones del proceso que la produjo.

Turpin et al. (2024) demostraron que los modelos producen cadenas de razonamiento que son sistemáticamente infieles cuando se introducen sesgos en el input. Los modelos son influenciados por los sesgos (cambian su respuesta) pero sus cadenas de razonamiento no mencionan los sesgos — construyen justificaciones que parecen razonables pero que no reflejan el proceso real. Lanham et al. (2023), en un estudio complementario, mostraron que modificar o truncar las cadenas de razonamiento intermedias frecuentemente no afecta a la respuesta final, lo que indica que el modelo no "usa" realmente esas cadenas para razonar sino que las genera como un artefacto lingüístico paralelo.

Turpin et al. (2024): el chain-of-thought puede ser sistemáticamente infiel al proceso real.

Para la accountability, las consecuencias son severas. Un sistema conversacional que, ante una auditoría, produce una explicación generada por el LLM está proporcionando una racionalización, no una explicación. La racionalización puede ser completamente coherente y completamente incorrecta como descripción del proceso real. Si el auditor pregunta «¿por qué el sistema rechazó la solicitud de cancelación?», una explica-

Lanham et al. (2023) — infidelidad de las cadenas de razonamiento.

ción post-hoc generada por el LLM dirá algo como «porque la póliza tiene menos de 30 días de antigüedad». Pero esta explicación puede ser una reconstrucción plausible que no corresponde al proceso real: quizás el modelo rechazó la solicitud por una combinación opaca de patrones en sus datos de entrenamiento, y la regla de los 30 días es simplemente la justificación más plausible que el modelo puede construir.

4.2.3 El paradigma constitutivo

Explicabilidad constitutiva

Un sistema exhibe **explicabilidad constitutiva** cuando su proceso de decisión opera sobre representaciones explícitas cuya traza es inherentemente explicativa. La explicación no se genera *después* de la decisión como una reconstrucción; *es* la traza del proceso de decisión. La correspondencia entre la explicación y el proceso real está garantizada por construcción, no por aproximación.

La diferencia entre ambos paradigmas puede ilustrarse con un ejemplo concreto.

En la explicabilidad constitutiva, la traza del proceso *es* la explicación. La correspondencia está garantizada por construcción.

Explicabilidad post-hoc vs. constitutiva

Situación: El sistema ha rechazado la solicitud de cancelación de un usuario. Un auditor pide la explicación.

Explicación post-hoc (LLM): «La solicitud fue rechazada porque la póliza tiene una antigüedad inferior a 30 días, lo que según las condiciones generales impide la cancelación sin penalización. El usuario no aceptó la penalización propuesta.»

Esta explicación es coherente, comprensible, y plausible. Pero no se puede verificar que corresponda al proceso real. Quizás el modelo rechazó la solicitud por otra razón y construyó esta justificación porque es la más coherente con los datos visibles.

Explicación constitutiva (ACAI): Traza de ejecución:

1. Input: «Quiero cancelar mi seguro» → NLU: acto `solicitud_cancelación`, producto `seguro_hogar_HG-2019-4421`.
2. Motor: regla `cancel_eligibility_01` → precondición: antigüedad $\geq$ 30 días → consulta: antigüedad = 18 días → precondición no satisfecha.
3. Motor: regla `cancel_penalty_01` → cancelación posible con penalización de 45 € → acción: informar y solicitar aceptación.
4. Input: «No, entonces prefiero esperar» → NLU: acto `rechazo_condición` → Motor: actualizar IS, solicitud can-

celada por el usuario. No ejecutar cancelación.

Esta traza no es una reconstrucción — es el registro literal de lo que ocurrió. Cada paso muestra la regla aplicada, los datos consultados, y la decisión tomada. La correspondencia con el proceso real es total porque la traza *es* el proceso.

La distinción entre ambos paradigmas no es de grado sino de naturaleza. La explicación post-hoc es una hipótesis sobre el proceso; la explicación constitutiva es un registro del proceso. La primera puede ser correcta o incorrecta; la segunda es correcta por construcción. La primera es útil para la comprensión; la segunda es necesaria para la accountability.

4.2.4 El argumento de Rudin y su extensión a la IA conversacional

El argumento de Rudin (2019) fue formulado para modelos de machine learning clásico — clasificadores, modelos de regresión, redes neuronales de clasificación — y proponía usar modelos intrínsecamente interpretables (modelos lineales dispersos, listas de reglas, modelos aditivos generalizados) en lugar de modelos opacos con explicaciones post-hoc. La extensión de este argumento a la IA conversacional requiere una adaptación que reconozca la especificidad del dominio.

En un sistema conversacional, no se puede simplemente reemplazar el LLM por un modelo interpretable: la variabilidad lingüística del lenguaje natural no es tratable con modelos lineales ni listas de reglas. El P1 de ACAI (separación de competencias) reconoce esta realidad: la capacidad lingüística requiere un LLM. Lo que el argumento de Rudin implica para ACAI es que las *decisiones operativas* — aquellas que tienen consecuencias de negocio, regulatorias o éticas — deben tomarse mediante un proceso intrínsecamente interpretable. Las decisiones lingüísticas (cómo interpretar, cómo generar) pueden delegarse al LLM; las decisiones operativas (qué hacer, qué reglas aplicar, qué información proporcionar) deben operar sobre representaciones explícitas con trazas constitutivas.

La explicabilidad constitutiva como requisito de diseño

La distinción post-hoc/constitutiva transforma la explicabilidad de un problema de análisis (dado un modelo opaco, generar una explicación) en un problema de diseño (diseñar el sistema de modo que las decisiones sean inherentemente explicables). Para ACAI, esto significa que la arquitectura no debe diseñarse primero y hacerse explicable después, sino que la explicabilidad debe ser

una propiedad constitutiva del diseño desde el origen. Esta es la razón por la que la tesis raíz dice que la accountability es un problema de *representación*: las representaciones formales son el mecanismo que hace posible la explicabilidad constitutiva.

4.3 El marco regulatorio como requisito de ingeniería

4.3.1 De la regulación a las especificaciones técnicas

El tratamiento del marco regulatorio en esta unidad no pretende ser un análisis jurídico sino una *traducción*: convertir los requisitos legales en especificaciones técnicas que el diseño de ACAI debe satisfacer. El investigador no necesita ser experto en derecho europeo, pero sí necesita entender qué le exige la regulación a un sistema conversacional de IA desplegado en un sector regulado.

Tres marcos regulatorios son especialmente relevantes para ACAI: el EU AI Act, que regula los sistemas de IA de forma general con requisitos específicos para sistemas de alto riesgo; DORA (*Digital Operational Resilience Act*), que impone requisitos de resiliencia digital al sector financiero; y el GDPR, que incluye disposiciones sobre decisiones automatizadas que afectan a individuos. Para cada uno, extraeremos los requisitos técnicos concretos.

La regulación no pide que los sistemas "intenten ser explicables" — exige trazabilidad, verificabilidad y supervisión humana como propiedades demostrables.

4.3.2 EU AI Act: requisitos para sistemas de alto riesgo

El EU AI Act, aprobado en 2024 y con entrada en vigor progresiva hasta 2027, clasifica los sistemas de IA en categorías de riesgo y establece requisitos proporcionales. Los sistemas de IA conversacional desplegados en sectores como seguros, banca, salud o administración pública pueden caer en la categoría de alto riesgo, dependiendo de su función y contexto de despliegue.

EU AI Act (2024) — Reglamento (UE) 2024/1689.

Para los sistemas de alto riesgo, los artículos 9 a 15 del Reglamento establecen requisitos que pueden traducirse en especificaciones técnicas concretas. Hacker et al. (2023), en su análisis de las implicaciones técnicas del AI Act, realizaron una primera traducción sistemática que resulta directamente útil para el diseño de ACAI.

Art. 9–15 del EU AI Act: requisitos para sistemas de alto riesgo.

El artículo 9 exige un **sistema de gestión de riesgos** que incluya la identificación, estimación y evaluación de los riesgos conocidos y previsibles. Para un sistema conversacional, esto se traduce en la necesidad de documentar formalmente qué puede salir mal (riesgos de interpretación incorrecta, de ejecución de acciones erróneas, de violación de políticas) y cómo se mitiga cada riesgo. Un sistema ACAI puede

satisfacer este requisito porque los modos de fallo están ligados a reglas formales identificables.

El artículo 11 exige **documentación técnica** que incluya una descripción del diseño, los datos de entrenamiento, el proceso de decisión, y los mecanismos de supervisión. El artículo 12 exige que el sistema tenga capacidades de **registro automático** (*logging*) que permitan la trazabilidad de su funcionamiento a lo largo del ciclo de vida. Para un sistema conversacional, esto se traduce en la necesidad de que cada turno del diálogo produzca una traza que incluya el input recibido, la interpretación, las reglas aplicadas, las acciones ejecutadas, y la respuesta generada. La arquitectura ACAI produce esta traza constitutivamente: no es un log añadido sino el registro natural del proceso de decisión del motor simbólico.

Hacker et al. (2023) — implementación técnica del AI Act.

El artículo 13 exige **transparencia** y la provisión de información suficiente para que los usuarios comprendan las capacidades y limitaciones del sistema. El artículo 14 exige **supervisión humana** efectiva, que incluye la capacidad de un humano de intervenir en el funcionamiento del sistema. Para un sistema conversacional, esto se traduce en interfaces de inspección del information state y mecanismos de intervención (escalación a humano) integrados en el flujo formal.

4.3.3 DORA: resiliencia operativa en el sector financiero

DORA (*Digital Operational Resilience Act*) impone requisitos adicionales al sector financiero que son relevantes cuando el sistema conversacional opera en el ámbito de banca o seguros. Los requisitos más pertinentes para ACAI son la obligación de realizar pruebas de resiliencia operativa digital, incluyendo pruebas de escenarios adversos; la exigencia de que los sistemas terceros (incluidos los servicios de IA) estén sujetos a gestión de riesgos de terceras partes; y la obligación de reportar incidentes relacionados con las TIC, incluyendo fallos de sistemas de IA que afecten al servicio.

DORA: Reglamento (UE) 2022/2554, en vigor desde enero de 2025.

Para un sistema conversacional basado en LLM, DORA plantea un desafío adicional: la naturaleza probabilística del LLM hace que las pruebas de resiliencia no puedan producir resultados deterministas. El mismo escenario de prueba puede producir resultados diferentes en ejecuciones diferentes. Un sistema ACAI mitiga este problema porque las decisiones operativas son deterministas (el motor simbólico produce el mismo resultado ante el mismo information state y las mismas reglas), y los componentes no deterministas (NLU, NLG) operan bajo supervisión formal.

4.3.4 GDPR: el derecho a explicación

El artículo 22 del GDPR, junto con los considerandos 63 y 71, establece que las personas tienen derecho a no ser objeto de decisiones basadas únicamente en el tratamiento automatizado que produzcan efectos jurídicos o significativos. Cuando tales decisiones se toman, el interesado tiene derecho a obtener una explicación de la lógica subyacente.

Veale y Zuiderveen Borgesius (2021) analizaron las ambigüedades de este derecho y concluyeron que, pese a los debates jurídicos sobre su alcance exacto, la tendencia regulatoria es inequívocamente hacia la exigencia de explicaciones significativas. Edwards y Veale (2017) señalaron que la mayoría de las técnicas de XAI no producen explicaciones que satisfagan el estándar legal, porque proporcionan información sobre el modelo en general pero no sobre la decisión específica que afecta al individuo.

Veale & Zuiderveen Borgesius (2021) — el derecho a explicación y sus ambigüedades.

Para un sistema conversacional ACAI, el derecho a explicación se satisface constitutivamente: la traza del motor simbólico para una conversación específica documenta exactamente qué reglas se aplicaron, qué datos se consultaron, y qué decisiones se tomaron en ese caso concreto. La explicación no es genérica («el modelo suele considerar la antigüedad de la póliza») sino específica («en su caso, la regla `cancel_penalty_01` se aplicó porque su póliza tiene 18 días de antigüedad, lo que requiere una penalización de 45 € según el artículo 23.2 de las Condiciones Generales»).

4.3.5 Síntesis: la tabla de requisitos técnicos

Requisitos técnicos derivados del marco regulatorio

El análisis de los tres marcos regulatorios converge en un conjunto de requisitos técnicos concretos que el sistema conversacional debe satisfacer:

Trazabilidad por turno: cada turno del diálogo debe producir una traza que incluya input, interpretación, reglas aplicadas, datos consultados, acciones ejecutadas, y respuesta generada (EU AI Act art. 12).

Verificabilidad: la traza debe poder contrastarse contra una especificación formal (las reglas del modelo de conocimiento) para determinar si el comportamiento fue correcto (EU AI Act art. 9).

Explicación específica: el sistema debe poder producir una explicación de una decisión concreta que afecta a un individuo concreto, no una explicación genérica del modelo (GDPR art. 22).

Supervisión humana: el sistema debe proporcionar mecanismos

para que un humano inspeccione el estado del diálogo e intervenga si es necesario (EU AI Act art. 14).
Determinismo operativo: las decisiones con consecuencias de negocio deben ser deterministas, de modo que las pruebas de resiliencia produzcan resultados reproducibles (DORA).
Documentación formal: el modelo de conocimiento, las reglas, y los procedimientos deben estar documentados de forma que un auditor pueda comprenderlos y evaluarlos (EU AI Act art. 11).
Estos seis requisitos son satisfechos constitutivamente por la arquitectura ACAI: no son capas añadidas sino consecuencias directas del diseño basado en representaciones formales con motor simbólico.

4.4 Gobernanza constitutiva vs. superpuesta

4.4.1 Dos estrategias de gobernanza

El análisis de los mecanismos de accountability actuales (guardrails, alineamiento, explicaciones post-hoc) y de los requisitos regulatorios (trazabilidad, verificabilidad, explicación específica) converge en una distinción que generaliza la oposición post-hoc/constitutiva a nivel arquitectónico completo.

Gobernanza superpuesta: controlar desde fuera. Gobernanza constitutiva: diseñar desde dentro.

La **gobernanza superpuesta** añade mecanismos de control y explicabilidad sobre un sistema que opera de forma autónoma. El sistema base (el LLM) toma sus decisiones; las capas superpuestas (guardrails, monitores, generadores de explicaciones) intentan controlar, observar y explicar esas decisiones. Esta estrategia es la dominante en la industria actual y produce resultados valiosos: reduce los comportamientos indeseados, proporciona mecanismos de seguridad, y genera explicaciones que satisfacen a muchos usuarios.

La **gobernanza constitutiva** diseña el sistema de modo que las propiedades deseadas — trazabilidad, verificabilidad, explicabilidad, determinismo — sean consecuencias de la estructura interna del sistema, no de las capas añadidas. Las decisiones se toman sobre representaciones formales; la traza de esas decisiones es inherentemente explicativa; la especificación formal permite la verificación; la estructura del motor permite la atribución.

La diferencia puede formalizarse en términos de garantías. La gobernanza superpuesta proporciona garantías *estadísticas*: reduce la probabilidad de comportamientos indeseados. La gobernanza constitutiva proporciona garantías *formales*: para las decisiones que operan sobre

el modelo formal, las propiedades de accountability se cumplen por construcción. Las garantías estadísticas son valiosas y tienen su lugar; las garantías formales son lo que los sectores regulados requieren.

4.4.2 La analogía estructural

La analogía más precisa para la diferencia entre ambas estrategias proviene de la ingeniería civil. Un edificio puede protegerse contra incendios de dos formas complementarias. La protección superpuesta consiste en instalar alarmas, rociadores, extintores y planes de evacuación. La protección constitutiva consiste en construir el edificio con materiales resistentes al fuego, diseñar las rutas de evacuación como parte de la estructura, y separar los compartimentos de forma que el fuego no pueda propagarse. Ambas estrategias son necesarias; pero solo la segunda ofrece la garantía de que un fuego en la cocina no colapsará el edificio. Los códigos de edificación exigen ambas, pero la constitutiva es el fundamento sobre el que la superpuesta opera.

ACAI adopta una posición análoga. La gobernanza constitutiva — el motor simbólico operando sobre representaciones formales — es el fundamento. La gobernanza superpuesta — guardrails adicionales, monitorización, alertas — puede añadirse como capa de seguridad adicional. Pero el diseño base debe garantizar las propiedades de accountability independientemente de las capas superpuestas, del mismo modo que la estructura del edificio debe resistir el fuego independientemente de que los rociadores funcionen.

4.5 Accountability y estructuras formales: la conexión con (AI)^2:TDG

4.5.1 La filosofía del proyecto marco

El proyecto (AI)^2:TDG (*Accountable and Interpretable AI through Topology and Discrete Geometry*) se funda en una tesis que puede formularse así: la explicabilidad robusta de los sistemas de IA requiere fundamentarse en *estructuras matemáticas formales* — topológicas, geométricas, algebraicas — no en aproximaciones numéricas ni en narrativas post-hoc. Las herramientas de la topología y la geometría discreta proporcionan invariantes, descomposiciones y relaciones estructurales que capturan propiedades de los sistemas de forma rigurosa y verificable.

Esta filosofía se manifiesta en las líneas de investigación del proyecto. RL1 explora cómo la topología algebraica y la geometría discreta pueden revelar estructura en datos de alta dimensionalidad y en los es-

pacios de representación de modelos de IA. RL2 investiga la integración neuro-simbólica y la representación del conocimiento como vehículos de accountability. RL3 aplica estas herramientas a dominios concretos, incluyendo los sistemas conversacionales verificables.

4.5.2 ACAI como instanciación de la filosofía

ACAI instancia la filosofía de (AI)^2:TDG en el dominio conversacional de una forma específica y coherente. El modelo de conocimiento formalizado es una estructura discreta — un grafo de reglas, estados, transiciones y restricciones — sobre el cual las propiedades de accountability son verificables formalmente. Las trazas conversacionales son trayectorias en un espacio de estados discreto cuyas propiedades — cobertura, convergencia, consistencia — son analizables con herramientas formales.

Las trazas conversacionales de ACAI como trayectorias en un espacio de estados discreto — un objeto de estudio natural para la topología.

La conexión es más profunda de lo que la analogía sugiere. Un modelo de conocimiento conversacional define un espacio de estados (el conjunto de information states posibles) y un conjunto de transiciones (las reglas que determinan cómo se transita entre estados). Las conversaciones son caminos en este espacio. Propiedades como «¿puede el sistema llegar a un estado de cancelación sin pasar por un estado de confirmación?» son propiedades de accesibilidad del grafo de estados. Propiedades como «¿hay conversaciones que no terminan?» son propiedades de convergencia. Propiedades como «¿las reglas son consistentes entre sí?» son propiedades de satisfacibilidad de la especificación. Todas son formalmente verificables sobre la estructura discreta del modelo.

Las herramientas topológicas de RL1 entran como extensión natural cuando el análisis requiere ir más allá de las propiedades discretas elementales. La topología del espacio conversacional — su forma, sus componentes conexas, sus "agujeros" — puede revelar patrones estructurales no capturados por las reglas individuales. Un "agujero" en el espacio conversacional (una región de information states que el sistema no puede alcanzar ni desde la cual puede salir) es un problema de diseño que el análisis topológico puede detectar. Esta conexión no se explota en el sprint ACAI pero fundamenta la continuación de la investigación dentro de (AI)^2:TDG.

> **ACAI como puente entre accountability práctica e investigación fundamental**
>
> ACAI opera en dos planos simultáneamente. En el plano práctico, produce un sistema conversacional con propiedades de accountability verificables. En el plano de investigación, genera las estruc-

turas formales — modelos de conocimiento, espacios de estados, trazas — sobre las cuales las herramientas de (AI)^2:TDG pueden operar. La Phase 1 construye el plano práctico; la Phase 2 y la investigación posterior explotan el plano fundamental.

Conexiones con otras unidades

→ Prerequisito: UD 1 (Mapa conceptual) para la tesis raíz y los principios fundacionales. **UD 2** (La conversación como fenómeno) para el diagnóstico de la brecha de representación.

⇄ Complementa: UD 3 (Ecosistema operativo) analiza los componentes tecnológicos y sus límites de gobernanza. UD4 proporciona el marco conceptual y regulatorio que explica *por qué* esos límites son inaceptables. Juntas, UD3 y UD4 cierran el diagnóstico completo del Eje A.

Aplicación: UD 12 (Evaluación) toma directamente la taxonomía de accountability de esta unidad y la traduce en métricas operativas. Las propiedades definidas aquí — trazabilidad, verificabilidad, explicabilidad, atribución — se convierten en dimensiones medibles en UD12.

Aplicación: UD 9 (Formalismos) evalúa los formalismos candidatos considerando, entre otros criterios, su capacidad para satisfacer los requisitos regulatorios derivados aquí: ¿permite el formalismo producir trazas verificables? ¿Son las reglas legibles para un auditor?

⇄ Complementa: UD 5 (Fundamentos neuro-simbólicos) proporciona el marco teórico que hace posible la explicabilidad constitutiva: la integración neuro-simbólica es el paradigma que permite tener simultáneamente capacidad lingüística y proceso de decisión formal.

Ejercicios

1. [Taxonomía aplicada] Seleccione un sistema conversacional comercial (chatbot de banca, asistente de atención al cliente, o similar) y evalúe qué niveles de la cadena transparencia → interpretabilidad → explicabilidad → accountability satisface. Para cada nivel, proporcione evidencia concreta de si se satisface o no. Si el sistema proporciona algún tipo de explicación, clasifíquela como post-hoc o constitutiva y justifique su clasificación.

2. [Análisis regulatorio] Seleccione tres artículos del EU AI Act (puede usar los artículos 9, 12 y 14 discutidos en esta unidad o elegir otros) y, para cada uno, formule: (a) el requisito legal en lenguaje simplificado, (b) la traducción a especificación técnica concreta para un sistema conversacional, y (c) cómo la arquitectura ACAI satisface ese requisito y cómo fallaría un sistema basado exclusivamente en LLM.

3. [Diseño de traza] Para el escenario de la aseguradora, diseñe el formato de una traza de accountability para un turno conversacional completo. La traza debe incluir toda la información necesaria para que un auditor pueda verificar que la decisión fue correcta, explicar por qué se tomó, y atribuir la responsabilidad en caso de error. Especifique qué campos incluiría, qué tipo de dato contiene cada campo, y quién (qué componente) produce cada dato.

4. [Reflexión crítica] El argumento de Rudin (2019) contra las explicaciones post-hoc ha sido influyente pero también criticado. Algunos autores argumentan que en muchos dominios prácticos, las explicaciones post-hoc son «suficientemente buenas» y que la insistencia en modelos intrínsecamente interpretables limita innecesariamente la capacidad del sistema. Evalúe este contraargumento en el contexto específico de ACAI: ¿hay situaciones en un sistema conversacional regulado donde la explicabilidad post-hoc sería suficiente? ¿Dónde estaría la frontera?

5. [Aplicación a ACAI] Considere el siguiente escenario: un regulador financiero realiza una auditoría del sistema conversacional de una aseguradora y solicita evidencia de que el sistema cumple con el artículo 12 del EU AI Act (capacidades de registro). Compare la evidencia que podría proporcionar: (a) un sistema basado en LLM con guardrails y logging de inputs/outputs, (b) un sistema basado en LLM con RAG y un sistema de explicaciones post-hoc, y (c) un sistema ACAI con motor simbólico y trazas constitutivas. Para cada caso, evalúe si la evidencia satisface el requisito legal y argumente su evaluación.

Referencias y lecturas

Lectura de entrada

Rudin, C. (2019). Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead. *Nature Machine Intelligence*, 1(5), 206–215. El artículo más

influyente sobre la insuficiencia de las explicaciones post-hoc para decisiones de alto riesgo. Argumentación clara, accesible, y directamente relevante para la distinción post-hoc/constitutiva que fundamenta ACAI. Lectura de entrada imprescindible.

Lecturas esenciales

- Arrieta, A. B., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., García, S., Gil-López, S., Molina, D., Benjamins, R., Chatila, R. & Herrera, F. (2020). Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion*, 58, 82–115. — El survey de referencia para la taxonomía conceptual. Extenso pero con una estructura clara que permite lectura selectiva.
- Lipton, Z. C. (2018). The Mythos of Model Interpretability. *Queue*, 16(3), 31–57. — El análisis que desambiguó el concepto de interpretabilidad. Breve, incisivo, y fundacional para la taxonomía de esta unidad.
- Hacker, P., Engel, A. & Mauer, M. (2023). Regulating ChatGPT and other Large Generative AI Models. *FAccT 2023*. — La traducción más rigurosa de los requisitos del AI Act a especificaciones técnicas para modelos generativos. Directamente útil para derivar requisitos de ACAI.
- Turpin, M., Michael, J., Perez, E. & Bowman, S. R. (2024). Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting. *NeurIPS 2023*. — La demostración empírica de que las cadenas de razonamiento de los LLMs son infieles. Evidencia central para el argumento contra las explicaciones post-hoc generadas por LLMs.

Lecturas complementarias

- Doshi-Velez, F. & Kim, B. (2017). Towards A Rigorous Science of Interpretable Machine Learning. *ArXiv preprint arXiv:1702.08608*. — La propuesta de formalización de la interpretabilidad con niveles de evaluación.
- Molnar, C. (2022). *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable* (2.ª ed.). Leanpub. — Libro de referencia sobre técnicas de interpretabilidad. Disponible online gratuitamente. Útil como referencia para las técnicas post-hoc.
- Ribeiro, M. T., Singh, S. & Guestrin, C. (2016). "Why Should I Trust You?": Explaining the Predictions of Any Classifier. *KDD 2016*. — LIME: la técnica de explicación local más influyente.
- Lundberg, S. M. & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *NeurIPS 2017*. — SHAP: valores de Shapley para explicación de modelos.
- Jain, S. & Wallace, B. C. (2019). Attention is not Explanation. *NAACL 2019*. — La demostración de que los pesos de atención no son explicaciones fiables.
- Adebayo, J., Gilmer, J., Muelly, M., Goodfellow, I., Hardt, M. & Kim, B. (2018). Sanity Checks for Saliency Maps. *NeurIPS 2018*. — Cuestionamiento de la validez de explicaciones basadas en saliencia.

- Lanham, T., Chen, A., Radhakrishnan, A., Steiner, B., Denison, C., Hernandez, D., Li, D., Durmus, E., Hubinger, E., Kernion, J., Lukošiūtė, K., Nguyen, K., Cheng, N., Joseph, N., Schiefer, N., Rausch, O., Larson, R., McCandlish, S., Kundu, S., Kadavath, S., Yang, S., Henighan, T., Maxwell, T., Telleen-Lawton, T., Hume, T., Hatfield-Dodds, Z., Kaplan, J., Brauner, J., Bowman, S. R. & Perez, E. (2023). Measuring Faithfulness in Chain-of-Thought Reasoning. *ArXiv preprint arXiv:2307.13702*. — Evidencia complementaria sobre la infidelidad de las cadenas de razonamiento.
- Schwalbe, G. & Finzel, B. (2023). A Comprehensive Taxonomy for Explainable Artificial Intelligence: A Systematic Survey of Surveys on Methods and Concepts. *Data Mining and Knowledge Discovery*, 38, 3183–3233. — Taxonomía actualizada de XAI con cobertura de LLMs.
- Holzinger, A., Saranti, A., Molnar, C., Biecek, P. & Samek, W. (2022). Explainable AI Methods — A Brief Overview. *xxAI — Beyond Explainable AI*, Lecture Notes in Computer Science 13200, 13–38. — Panorámica concisa y actualizada de métodos de XAI.
- Veale, M. & Zuiderveen Borgesius, F. (2021). Demystifying the Draft EU Artificial Intelligence Act. *Computer Law & Security Review*, 41, 105581. — Análisis accesible del AI Act con perspectiva técnica.
- Edwards, L. & Veale, M. (2017). Slave to the Algorithm? Why a "Right to an Explanation" Is Probably Not the Remedy You Are Looking For. *Duke Law & Technology Review*, 16(1), 18–84. — Análisis crítico del derecho a explicación en el GDPR.
- EU AI Act (2024). Reglamento (UE) 2024/1689 del Parlamento Europeo y del Consejo por el que se establecen normas armonizadas en materia de inteligencia artificial. — Texto legal oficial.
- DORA (2022). Reglamento (UE) 2022/2554 sobre la resiliencia operativa digital del sector financiero. — Texto legal oficial.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. & Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *NeurIPS 2022*. — El artículo que popularizó chain-of-thought. Relevante como contexto para la discusión sobre racionalización.

MONOGRÁFICO FORMATIVO

Fundamentos de IA Conversacional Neuro-Simbólica

«Symbols without grounding are fragile.
Learning without structure is blind.
The future belongs to their integration.»

Eje B — El paradigma neuro-simbólico

Unidad 5 de 12

Research Sprint ACAI · Proyecto $(AI)^2$:TDG · 2026

Eje B — El paradigma neuro-simbólico

Prerequisito: UD1 · Eje B — El paradigma neuro-simbólico

UNIDAD DIDÁCTICA 5

Fundamentos del paradigma neuro-simbólico

Contenidos de la unidad

5.1.1 Dos intuiciones sobre la inteligencia 3
5.1.2 Lo que cada paradigma captura y lo que pierde 5
Las fortalezas del razonamiento simbólico 5
Las debilidades del razonamiento simbólico 6
Las fortalezas del procesamiento neuronal 6
Las debilidades del procesamiento neuronal 7
5.1.3 La complementariedad como argumento para la integración 7
5.2.1 El programa de Garcez y Lamb 8
5.2.2 El estado de la tercera ola 9
5.3.1 El marco de referencia 10
5.3.2 Nivel 1: Symbolic Neuro Symbolic 10
5.3.3 Nivel 2: Symbolic[Neuro] 11
5.3.4 Nivel 3: Neuro : Symbolic ⇌ Neuro 12
5.3.5 Niveles 4 y 5: horizontes de investigación 13
5.3.6 El espectro como mapa de ruta 14
5.4.1 Coordenadas del posicionamiento 15
5.4.2 Las especificidades de ACAI 16
5.4.3 Lo que ACAI toma y lo que aporta 16

¿Por qué necesito esto para ACAI?

ACAI propone una arquitectura neuro-simbólica: un motor simbólico que gobierna las decisiones operativas, un LLM que maneja la variabilidad lingüística, y una integración progresiva entre ambos. Pero esta propuesta no surge del vacío. Se inscribe en un programa de investigación con décadas de historia, un marco teórico articulado, y una taxonomía de niveles de integración que estructura directamente las fases del sprint. Esta unidad proporciona ese fundamento. Al terminar, el investigador sabrá de dónde viene el paradigma neuro-simbólico, qué ofrece, cómo se clasifica la integración, y dónde se sitúa ACAI en ese paisaje. Sin esta comprensión, las decisiones arquitectónicas del sprint parecerán arbitrarias; con ella, se revelan como consecuencias de un análisis riguroso.

Objetivos de aprendizaje

Al completar esta unidad, el investigador será capaz de:

1. Caracterizar las fortalezas y debilidades de los paradigmas simbólico y conexionista con precisión técnica, no como caricaturas.
2. Explicar el programa de las tres olas de la IA neuro-simbólica y situar ACAI en la tercera ola.
3. Describir los cinco niveles de la taxonomía de Kautz, identificar qué capacidades ofrece cada nivel, y justificar por qué ACAI progresa del nivel 1–2 al 2–3.
4. Argumentar por qué la integración neuro-simbólica es una necesidad técnica — no una preferencia estética — para un sistema conversacional accountable.
5. Identificar el posicionamiento de ACAI en el paisaje neuro-simbólico y su contribución específica al programa de investigación.

5.1 La tensión fundacional de la inteligencia artificial

Sede principal de este concepto.

5.1.1 Dos intuiciones sobre la inteligencia

La inteligencia artificial nació, en la segunda mitad del siglo XX, con dos intuiciones simultáneas e incompatibles sobre cómo producir comportamiento inteligente en máquinas. Ambas intuiciones captaban

algo real sobre la naturaleza de la cognición; ambas perdían algo esencial. La tensión entre ellas ha definido el campo durante siete décadas y sigue siendo, en una forma transformada, la tensión que ACAI propone resolver en el dominio conversacional.

La primera intuición — la **intuición simbólica** — fue articulada con máxima precisión por Newell y Simon (1976) en su hipótesis del sistema de símbolos físicos: «Un sistema de símbolos físicos tiene los medios necesarios y suficientes para la acción inteligente general.» La inteligencia, según esta hipótesis, consiste en la capacidad de manipular representaciones simbólicas según reglas formales. El conocimiento se codifica en estructuras discretas con semántica definida — proposiciones, reglas, marcos, redes semánticas — y el razonamiento es un proceso de transformación formal de esas estructuras. La tradición que emergió de esta intuición produjo los sistemas expertos, la lógica computacional, los planificadores automáticos, las ontologías, y los lenguajes de representación del conocimiento que siguen siendo herramientas fundamentales de la ingeniería del conocimiento.

Dos intuiciones fundacionales: la inteligencia como manipulación de símbolos vs. la inteligencia como aprendizaje estadístico.

La segunda intuición — la **intuición conexionista** — tiene raíces en los trabajos de McCulloch y Pitts (1943) sobre neuronas artificiales y de Rosenblatt (1958) sobre el perceptrón, pero alcanzó madurez computacional con el redescubrimiento de la retropropagación por Rumelhart, Hinton y Williams (1986). La inteligencia, según esta intuición, no reside en reglas explícitas sino que emerge de la interacción entre grandes números de unidades simples conectadas, cuyos pesos se ajustan mediante aprendizaje a partir de datos. El conocimiento no se codifica explícitamente sino que se distribuye en los patrones de activación de la red. El razonamiento no es transformación formal de símbolos sino propagación de activación a través de capas de conexiones ponderadas.

Newell & Simon (1976) — la hipótesis del sistema de símbolos físicos.

Es importante resistir la tentación de tratar estas intuiciones como posiciones extremas que nadie sostiene realmente. En su forma más sofisticada, ambas tradiciones han producido resultados extraordinarios. La tradición simbólica produjo, entre otros logros, los sistemas de verificación formal que garantizan la corrección de software crítico, las bases de conocimiento que sustentan la web semántica, y los planificadores que controlan misiones espaciales. La tradición conexionista produjo, en su forma contemporánea, los modelos de lenguaje que comprenden y generan texto con fluidez sin precedentes, los sistemas de visión que superan la capacidad humana en tareas de clasificación, y los modelos generativos que producen imágenes, música y código. Ninguna de estas tradiciones es un error — cada una captura un aspecto genuino de lo que necesitamos para construir sistemas inteligentes.

Rumelhart, Hinton & Williams (1986): la retropropagación como fundamento del conexionismo moderno.

5.1.2 Lo que cada paradigma captura y lo que pierde

La tensión entre ambos paradigmas no es una cuestión de preferencia sino de *propiedades complementarias*. Cada paradigma ofrece capacidades que el otro no puede proporcionar, y cada uno tiene limitaciones que el otro no sufre. Caracterizar estas complementariedades con precisión es fundamental para entender por qué la integración neuro-simbólica es una necesidad técnica, no una preferencia estética.

Las fortalezas del razonamiento simbólico

El razonamiento simbólico posee cuatro propiedades que son directamente relevantes para la accountability y que el procesamiento neuronal no puede proporcionar.

Lo simbólico aporta estructura, composicionalidad, verificabilidad, y eficiencia en conocimiento — exactamente lo que ACAI necesita para la accountability.

La primera es la **composicionalidad**. Los sistemas simbólicos operan sobre representaciones que se componen de partes con significado propio, y el significado del todo se deriva del significado de las partes y de las reglas de composición. Una regla como «si el cliente solicita cancelación Y la antigüedad es menor a 30 días, ENTONCES aplicar penalización» se compone de condiciones y acciones que tienen significado independiente, y el significado de la regla completa se deriva de la semántica de la conjunción y del condicional. Esta propiedad permite construir sistemas cuyo comportamiento es analizable en términos de sus partes constituyentes. Fodor y Pylyshyn (1988), en su crítica clásica del conexionismo, argumentaron que la composicionalidad es una propiedad irrenunciable para cualquier sistema que aspire a la sistematicidad del pensamiento.

Fodor & Pylyshyn (1988) — composicionalidad y sistematicidad como propiedades irrenunciables.

La segunda es la **verificabilidad**. Un sistema que opera sobre reglas formales puede ser verificado contra una especificación: se puede comprobar que las reglas son consistentes entre sí, que cubren todos los casos requeridos, que no producen estados no deseados. Las técnicas de verificación formal — model checking, theorem proving, análisis estático — llevan décadas siendo aplicadas con éxito a sistemas simbólicos. Esta capacidad de verificación es exactamente la propiedad que la **UD 4** identificó como componente esencial de la accountability y que los requisitos regulatorios exigen.

La tercera es la **interpretabilidad intrínseca**. Las representaciones simbólicas tienen semántica definida: cada símbolo denota algo, cada regla tiene una lectura interpretable. La traza de un razonamiento simbólico es una secuencia de pasos con significado, no una secuencia de multiplicaciones de matrices. Esta propiedad es lo que permite la explicabilidad constitutiva introducida en la **UD 4**: la traza del proceso de decisión *es* la explicación.

La cuarta es la **eficiencia en conocimiento**. Un sistema simbólico

puede incorporar conocimiento de dominio directamente, sin necesidad de aprender de miles de ejemplos. Una regla de negocio se codifica una vez y se aplica correctamente en todos los casos. Esta eficiencia es crucial en dominios donde los datos etiquetados son escasos pero el conocimiento experto es abundante — exactamente la situación de muchos dominios regulados.

Las debilidades del razonamiento simbólico

Pero el razonamiento simbólico tiene debilidades igualmente fundamentales que han limitado históricamente su aplicabilidad.

La más conocida es la **fragilidad** ante la variabilidad del mundo real. Un sistema de reglas que espera la entrada «cancelar póliza» fallará ante «me gustaría dejar de tener el seguro» o «¿cómo hago para darme de baja?». La variabilidad lingüística, perceptual y situacional del mundo real es intratable con reglas escritas a mano. Es exactamente lo que la **UD 2** documentó como la limitación fatal de los sistemas de diálogo basados en reglas.

La segunda debilidad es el **cuello de botella de la ingeniería del conocimiento**. Codificar todo el conocimiento necesario para un dominio complejo en representaciones formales es un proceso costoso, lento, y propenso a errores y omisiones. Este problema, identificado ya por Feigenbaum (1977) como la «limitación fundamental» de los sistemas expertos, nunca fue resuelto satisfactoriamente por la tradición simbólica. ACAI lo aborda directamente con I2 (LLM como meta-herramienta para la formalización asistida), que es precisamente un intento de romper este cuello de botella usando capacidades generativas.

Feigenbaum (1977): el cuello de botella de la adquisición de conocimiento como limitación fundamental de la IA simbólica.

La tercera debilidad es la **dificultad para manejar la incertidumbre y la gradualidad**. El mundo real no es nítido: los conceptos tienen fronteras difusas, la información es incompleta, las categorías se solapan. Los sistemas simbólicos clásicos operan en lógica bivalente (verdadero o falso) y manejan la incertidumbre mediante extensiones (lógica difusa, redes bayesianas, factores de certeza) que añaden complejidad sin resolver el problema de fondo: la representación discreta tiene dificultades intrínsecas para capturar la continuidad del mundo.

Las fortalezas del procesamiento neuronal

Las fortalezas del procesamiento neuronal son, en buena medida, la imagen especular de las debilidades del simbólico.

La **robustez ante la variabilidad** es quizás la fortaleza más relevante para ACAI. Un modelo entrenado sobre datos lingüísticos reconoce formulaciones nunca vistas, maneja sinónimos, comprende elipsis, y tolera errores gramaticales. Esta capacidad de generalización es lo que los

LLMs aportan a la arquitectura ACAI: manejar la variabilidad lingüística que las reglas formales no pueden cubrir.

La **capacidad de aprendizaje desde datos** permite adquirir patrones complejos sin codificación manual. Un LLM aprende las convenciones del lenguaje natural — pragmática, implicatura, cortesía, registro — a partir de su exposición a cantidades masivas de texto, sin que nadie tenga que formular reglas explícitas para cada convención.

La **representación distribuida** permite capturar similitudes y relaciones graduales entre conceptos. En el espacio de representación de un modelo neuronal, «cancelar», «rescindir», «dar de baja» y «anular» están próximos; «renovar» y «ampliar» están próximos entre sí pero lejos de los anteriores. Esta geometría semántica emerge del entrenamiento sin necesidad de codificación manual.

Las debilidades del procesamiento neuronal

Las debilidades del procesamiento neuronal son, simétricamente, la imagen especular de las fortalezas del simbólico. La **opacidad**: las decisiones son el resultado de operaciones numéricas distribuidas que carecen de interpretación semántica accesible. La **inconsistencia**: el mismo input procesado dos veces puede producir resultados diferentes debido a la naturaleza estocástica de la generación. La **imposibilidad de verificación formal**: no hay especificación contra la cual verificar ni proceso cuya conformidad se pueda comprobar. Y el **hambre de datos**: los modelos requieren cantidades masivas de ejemplos para aprender patrones que una sola regla simbólica podría capturar.

Marcus (2020): catálogo sistemático de las limitaciones del deep learning — razonamiento, composicionalidad, robustez.

Marcus (2020), en su análisis de las limitaciones del aprendizaje profundo, documentó estas debilidades con extensos ejemplos y argumentó que son consecuencias estructurales de la arquitectura, no defectos que versiones futuras corregirán. Bengio (2019), desde dentro de la tradición conexionista, reconoció la necesidad de incorporar composicionalidad y razonamiento sistemático en los modelos neuronales, en lo que denominó una agenda para la "consciencia computacional" (*System 2 deep learning*). Lake et al. (2017) propusieron que la capacidad de construir modelos causales composicionales es lo que distingue la cognición humana de las capacidades actuales del aprendizaje profundo.

Lake et al. (2017) — *Building Machines That Learn and Think Like People*.

5.1.3 La complementariedad como argumento para la integración

La simetría entre fortalezas y debilidades de ambos paradigmas no es accidental. Refleja una complementariedad profunda que puede formularse así: **lo que cada paradigma necesita es exactamente lo que el otro ofrece**.

El razonamiento simbólico necesita un mecanismo para manejar la variabilidad del mundo real — el procesamiento neuronal lo proporciona. El procesamiento neuronal necesita estructura, composicionalidad y verificabilidad — el razonamiento simbólico los proporciona. El simbólico necesita reducir el coste de la adquisición de conocimiento — los modelos generativos pueden asistir en la formalización. El neuronal necesita que sus decisiones sean auditables — las representaciones formales proporcionan el marco de auditoría.

La integración como necesidad técnica

Para ACAI, la integración neuro-simbólica no es una opción de diseño entre varias posibles. Es una *consecuencia directa* del análisis de requisitos. La accountability exige propiedades que solo el razonamiento simbólico puede proporcionar (verificabilidad, trazabilidad, determinismo). La operación en lenguaje natural exige capacidades que solo el procesamiento neuronal puede proporcionar (comprensión de formulaciones variables, generación de texto natural). Ningún paradigma por separado satisface ambos conjuntos de requisitos. La integración es la única solución que los satisface simultáneamente.

5.2 Las tres olas de la IA neuro-simbólica

5.2.1 El programa de Garcez y Lamb

Garcez y Lamb (2023), en su artículo programático *Neurosymbolic AI: The 3rd Wave*, propusieron una narrativa del campo de la inteligencia artificial organizada en tres olas que corresponden a tres respuestas diferentes a la tensión fundacional. Esta narrativa no es una periodización histórica estricta — las tres olas coexisten temporalmente — sino un marco conceptual que identifica tres *paradigmas* con propiedades y limitaciones diferentes.

Garcez & Lamb (2023) — *Neurosymbolic AI: The 3rd Wave.*

La **primera ola** corresponde a la IA simbólica en sus diversas manifestaciones: sistemas expertos, planificación automática, representación del conocimiento, razonamiento lógico. El conocimiento se representa explícitamente y el comportamiento se deriva formalmente de esas representaciones. Las fortalezas de la primera ola son la interpretabilidad, la verificabilidad, y la eficiencia en dominios bien definidos. Su limitación fundamental es la fragilidad: la incapacidad de manejar la variabilidad y la incertidumbre del mundo real, y el coste prohibitivo de la adquisición manual de conocimiento.

Primera ola: representación sin aprendizaje. Segunda ola: aprendizaje sin representación. Tercera ola: la síntesis.

La **segunda ola** corresponde al aprendizaje profundo en sus diversas

manifestaciones: redes convolucionales, modelos de secuencia, transformers, modelos generativos. El conocimiento se adquiere automáticamente desde los datos y se almacena en los parámetros del modelo. Las fortalezas de la segunda ola son la capacidad de aprendizaje, la robustez ante la variabilidad, y el rendimiento en tareas perceptuales y lingüísticas. Su limitación fundamental es la opacidad: la incapacidad de proporcionar garantías formales sobre el comportamiento, y la dificultad para incorporar conocimiento de dominio de forma controlada.

La **tercera ola** es el programa de integración neuro-simbólica: la construcción de sistemas que combinen las fortalezas de ambos paradigmas, usando representaciones simbólicas para la estructura y el razonamiento, y redes neuronales para la percepción, el aprendizaje y la generación. Garcez y Lamb argumentan que esta tercera ola no es simplemente la suma de las dos anteriores sino un paradigma genuinamente nuevo que requiere nuevas arquitecturas, nuevos formalismos, y nuevas metodologías de evaluación.

5.2.2 El estado de la tercera ola

Es necesario ser honesto sobre el estado de la tercera ola. Como señalan los propios Garcez y Lamb, la integración neuro-simbólica es más un programa de investigación en desarrollo que un paradigma consolidado. No existe una arquitectura neuro-simbólica canónica del mismo modo que existe la arquitectura transformer para el procesamiento neuronal del lenguaje. Lo que existe es un conjunto creciente de resultados que demuestran que la integración es posible y productiva en dominios específicos, y un consenso emergente sobre los principios que deben guiar esa integración.

Hitzler et al. (2022): volumen colectivo que documenta el estado del arte neuro-simbólico con 25 contribuciones.

Hitzler et al. (2022), en su volumen colectivo que reúne 25 contribuciones de los principales grupos de investigación, documentan avances concretos en la integración de lógica y redes neuronales, en el aprendizaje de representaciones simbólicas a partir de datos, en el razonamiento neuronal sobre grafos de conocimiento, y en la generación de explicaciones combinando análisis simbólico y neuronal. Estos avances son genuinos pero parciales: cada uno resuelve un aspecto de la integración en un dominio específico, y la síntesis general sigue siendo un objetivo de investigación.

ACAI se sitúa en este paisaje como una contribución a la tercera ola en un dominio específico (IA conversacional) con un objetivo específico (accountability). No pretende resolver el problema general de la integración neuro-simbólica sino demostrar que, en el dominio conversacional, una integración cuidadosamente diseñada produce propiedades de accountability que ningún paradigma por separado puede proporcionar.

Esta modestia de alcance es una fortaleza: permite diseños concretos donde el programa general requiere abstracciones.

> **La tercera ola como programa, no como logro**
>
> El investigador debe entender que la IA neuro-simbólica de tercera ola es un programa de investigación activo, no una tecnología madura con soluciones establecidas. ACAI contribuye a este programa; no lo hereda como un kit de herramientas listo para usar. Las decisiones de diseño del sprint — qué nivel de integración intentar, qué formalismos usar, cómo evaluar la integración — son decisiones de investigación genuinas, no aplicaciones de recetas conocidas.

5.3 La taxonomía de Kautz: cinco niveles de integración

Sede principal de este concepto.

5.3.1 El marco de referencia

Kautz (2022) — AAAI Robert S. Engelmore Memorial Lecture. Taxonomía fundacional.

Kautz (2022), en su AAAI Robert S. Engelmore Memorial Lecture titulada *The Third AI Summer*, propuso una taxonomía de cinco niveles de integración neuro-simbólica que ha sido adoptada ampliamente como marco de referencia. La taxonomía describe un espectro que va desde la composición secuencial más simple hasta la fusión completa de representaciones neuronales y simbólicas. Cada nivel ofrece capacidades que el anterior no tiene, pero exige cimientos que el anterior no requiere.

Esta taxonomía es directamente relevante para ACAI porque la propuesta ejecutiva define explícitamente las fases del sprint en sus términos: la Phase 1 opera en los niveles 1–2, y la Phase 2 avanza hacia los niveles 2–3. Entender qué significa cada nivel, qué capacidades ofrece, y qué exige, es imprescindible para comprender las decisiones arquitectónicas del sprint.

5.3.2 Nivel 1: Symbolic Neuro Symbolic

> **Nivel 1 — Symbolic Neuro Symbolic**
>
> En el nivel 1, un sistema simbólico usa un módulo neuronal como componente encapsulado, tratándolo como una caja negra con una interfaz definida. El sistema simbólico invoca al módulo neuronal para una tarea específica (por ejemplo, clasificación o generación

de texto), usa el resultado, y continúa su razonamiento simbólico. La dirección de control va del componente simbólico al neuronal.

Nivel 1: el componente simbólico orquesta; el neuronal es un servicio.

El ejemplo más directo de nivel 1 en el contexto de ACAI es el pipeline básico de la Phase 1: el motor simbólico invoca al LLM como servicio de NLU («interpreta este input del usuario») y como servicio de NLG («genera una respuesta que comunique esta decisión»). Entre ambas invocaciones, el motor opera de forma puramente simbólica: consulta las reglas, evalúa las condiciones, actualiza el information state. El LLM no tiene conocimiento del razonamiento simbólico; es un servicio lingüístico invocado con una interfaz definida.

Las ventajas del nivel 1 son la simplicidad y la claridad de la separación de competencias. Las decisiones simbólicas son completamente trazables porque el componente neuronal no interviene en ellas. La desventaja principal es la rigidez de la interfaz: el motor simbólico no puede condicionar cómo el LLM procesa su tarea (qué buscar en la interpretación, qué matices capturar), y el LLM no puede informar al motor sobre incertidumbres o matices que no encajan en la interfaz predefinida.

La versión dual — Neuro Symbolic Neuro, donde un sistema neuronal usa un componente simbólico — también existe y es de hecho el patrón más común en la industria actual: un LLM que invoca herramientas (calculadoras, bases de datos, APIs) tratándolas como servicios. Pero esta versión dual no produce accountability porque las decisiones de cuándo y cómo invocar el componente simbólico las toma el LLM de forma opaca. ACAI elige deliberadamente la dirección de control simbólico → neuronal.

5.3.3 Nivel 2: Symbolic[Neuro]

Nivel 2 — Symbolic[Neuro]

En el nivel 2, el componente neuronal opera *dentro* del marco definido por el componente simbólico. El sistema simbólico no solo invoca al neuronal como servicio sino que le proporciona contexto estructurado que condiciona su operación: restricciones sobre la salida esperada, vocabulario permitido, esquema de la respuesta, o distribución de probabilidad condicionada por el estado simbólico.

Nivel 2: el componente simbólico no solo invoca al neuronal — lo condiciona.

La diferencia entre los niveles 1 y 2 es sutil pero importante. En el nivel 1, el motor simbólico pide al LLM «interpreta este input» y recibe una respuesta. En el nivel 2, el motor pide «interpreta este input sabiendo que

estamos en un diálogo de cancelación, que el usuario ya ha proporcionado su número de póliza, y que los únicos actos comunicativos posibles en este punto son `confirmar`, `rechazar`, `preguntar_condiciones` o `proporcionar_dato`». Esta información estructurada reduce el espacio de interpretación del LLM y aumenta la fiabilidad de su respuesta.

Phase 1
Opera en los niveles 1–2.

La Phase 1 de ACAI incluye elementos de nivel 2 cuando el motor simbólico proporciona al LLM información del information state como contexto para la interpretación y la generación. Las técnicas de *structured prompting* — donde el prompt del LLM incluye el esquema esperado de la respuesta, las entidades relevantes del dominio, y las restricciones del estado del diálogo — son implementaciones prácticas del nivel 2.

Las ventajas del nivel 2 son una mayor fiabilidad de la interfaz neuro-simbólica (el LLM comete menos errores cuando está condicionado por contexto estructurado) y una integración más rica (el estado simbólico influye en el procesamiento neuronal). La desventaja es que el condicionamiento opera a nivel de prompt, no de computación: el LLM recibe instrucciones más precisas pero sigue interpretándolas probabilísticamente.

5.3.4 Nivel 3: Neuro : Symbolic ⇌ Neuro

Nivel 3 — Integración bidireccional

En el nivel 3, la interacción entre los componentes neuronal y simbólico es bidireccional y opera a un nivel más profundo que el intercambio de información por interfaz. El estado simbólico condiciona la computación neuronal (no solo su entrada sino su proceso), y los resultados de la computación neuronal alimentan la evolución del conocimiento simbólico. Ambos componentes se influyen mutuamente durante la operación del sistema.

El nivel 3 introduce una diferencia cualitativa respecto a los anteriores. En los niveles 1 y 2, la frontera entre lo neuronal y lo simbólico es una *interfaz*: los componentes se comunican mediante representaciones intermedias pero operan internamente de forma independiente. En el nivel 3, la frontera se convierte en una *zona de interacción* donde los paradigmas se influyen mutuamente.

Phase 2
Avanza hacia los niveles 2–3.

La Phase 2 de ACAI explora tres mecanismos de nivel 3 que fueron presentados como frentes de innovación I3a–c en la **UD 1**. El primero es el **condicionamiento simbólico de la computación neuronal** (I3a): el estado del diálogo modifica no solo el prompt sino los mecanismos de atención o las restricciones de decodificación del LLM. Técnicas como el *constrained decoding* — donde la generación del LLM se fuerza a producir

tokens que satisfacen una gramática formal — son implementaciones concretas de este mecanismo.

El segundo es el **aprendizaje simbólico desde trazas** (I3b): las trazas conversacionales de la Phase 1 se analizan para detectar patrones no cubiertos por las reglas formales y proponer extensiones del modelo. El conocimiento simbólico evoluciona a partir de evidencia empírica, cerrando parcialmente el cuello de botella de la ingeniería del conocimiento.

El tercero es el **razonamiento abductivo compartido** (I3c): ante inputs ambiguos, el sistema genera hipótesis interpretativas combinando la distribución de probabilidad del LLM con las restricciones del estado simbólico, ordenadas por plausibilidad formal. La interpretación no es ni puramente neuronal (probabilística) ni puramente simbólica (determinista) sino una integración de ambas.

Nivel 2 vs. nivel 3 en la interpretación

Input del usuario: «Sí, pero espere un momento.»
Nivel 2: El motor informa al LLM que los actos posibles son `confirmar`, `rechazar`, `pedir_pausa`. El LLM interpreta: probablemente `confirmar` + `pedir_pausa`. El motor procesa ambos actos secuencialmente.
Nivel 3: El motor no solo informa los actos posibles sino que condiciona las probabilidades de decodificación según el estado del diálogo. Si el turno anterior fue una solicitud de confirmación de cancelación (operación irreversible), el sistema aumenta la probabilidad de interpretaciones que incluyan señales de vacilación o solicitud de tiempo. Además, el patrón «sí, pero» en contexto de decisión irreversible se detecta como señal de *incertidumbre pragmática*, lo que activa un protocolo de verificación reforzada en el motor simbólico. La interpretación integra evidencia neuronal (la distribución de probabilidad) con conocimiento simbólico (el patrón pragmático y el contexto de la operación).

5.3.5 Niveles 4 y 5: horizontes de investigación

Nivel 4 — Neuro:Symbolic

En el nivel 4, las representaciones neuronales y simbólicas se mapean entre sí de forma sistemática. Los conceptos simbólicos tienen correspondencias directas en el espacio de representación neuronal, y viceversa. Esto permite, por ejemplo, que el aprendizaje neuronal produzca directamente nuevos conceptos simbólicos, o que la estructura simbólica organice el espacio de representación neuronal.

Nivel 5 — Neuro≡Symbolic

En el nivel 5, las representaciones neuronales y simbólicas se fusionan en un sistema unificado donde la distinción entre ambos paradigmas pierde sentido. El sistema opera sobre representaciones que son simultáneamente distribuidas (como las neuronales) y estructuradas (como las simbólicas).

Los niveles 4 y 5 representan horizontes de investigación que van más allá del alcance del sprint ACAI. Trabajos como Logic Tensor Networks (Badreddine et al., 2022), que representan fórmulas lógicas como operaciones sobre tensores, y DeepProbLog (Manhaeve et al., 2021), que integra redes neuronales con programación probabilística lógica, son aproximaciones al nivel 4 en dominios específicos. La fusión completa del nivel 5 sigue siendo un objetivo de investigación fundamental.

Logic Tensor Networks (Badreddine et al., 2022), DeepProbLog (Manhaeve et al., 2021): aproximaciones al nivel 4.

Para ACAI, los niveles 4 y 5 son relevantes no como objetivo del sprint sino como horizonte que muestra la dirección del programa de investigación a largo plazo. Las representaciones formales que ACAI produce en la Phase 1 y los mecanismos de integración que explora en la Phase 2 sientan las bases para exploraciones más profundas que podrían, en proyectos futuros, acercarse a los niveles 4 y 5.

5.3.6 El espectro como mapa de ruta

La taxonomía de Kautz justifica directamente el P3 de ACAI (la integración como espectro) y la estructura en dos fases del sprint. Cada nivel de la taxonomía ofrece capacidades que el anterior no tiene, pero exige cimientos que el anterior no requiere.

P3: la integración es un espectro. Avanzar deliberadamente es más productivo que intentar la fusión sin los cimientos resueltos.

Para operar en el nivel 1, se necesita un motor simbólico funcional y un LLM accesible como servicio. Para el nivel 2, se necesita además un mecanismo de condicionamiento del LLM basado en el estado simbólico. Para el nivel 3, se necesitan además técnicas de constrained decoding, mecanismos de aprendizaje desde trazas, y un marco para el razonamiento abductivo. Intentar el nivel 3 sin haber consolidado las representaciones formales del nivel 1 es como intentar integrar dos sistemas que no están individualmente definidos.

La Phase 1 consolida los niveles 1–2: diseña el lenguaje de representación, construye el motor simbólico, establece las interfaces con el LLM, y demuestra que el pipeline gobernado produce propiedades de accountability medibles. La Phase 2 avanza hacia el nivel 3 sobre esos cimientos, explorando los tres mecanismos de integración bidireccional. La **métrica central** de la Phase 2 es comparativa: debe demostrar que la integración de nivel 2–3 mejora la de nivel 1–2 en dimensiones concretas

(robustez, manejo de ambigüedad, adaptación a cambios de dominio).

> **Progresión deliberada vs. ambición prematura**
>
> La progresión por niveles no es cautela sino rigor. Cada nivel añade complejidad arquitectónica y reduce la trazabilidad del sistema. En el nivel 1, las decisiones simbólicas son completamente independientes del procesamiento neuronal; su trazabilidad es total. En el nivel 2, las decisiones simbólicas están condicionadas por información del contexto neuronal; la trazabilidad se mantiene pero incluye la dependencia del condicionamiento. En el nivel 3, la interacción bidireccional hace que las decisiones de cada componente dependan del otro; la trazabilidad requiere registrar la interacción completa.
> Para ACAI, donde la trazabilidad es un objetivo central, esta progresión implica que cada avance en integración debe ir acompañado de una extensión del modelo de trazabilidad que lo documente. La evaluación comparativa entre fases es el mecanismo que garantiza que la integración más profunda produce beneficios que justifican la complejidad añadida.

5.4 Posicionamiento de ACAI en el paisaje neuro-simbólico

5.4.1 Coordenadas del posicionamiento

Los tres marcos presentados — la tensión fundacional (sección 5.1), las tres olas de Garcez y Lamb (sección 5.2), y la taxonomía de Kautz (sección 5.3) — permiten situar ACAI con precisión en el paisaje neuro-simbólico.

ACAI es un sistema de **tercera ola**: combina representación simbólica explícita con procesamiento neuronal, buscando la síntesis de las fortalezas de ambos paradigmas. Opera en los **niveles 1–3 de Kautz**: comienza con un pipeline gobernado (niveles 1–2) y explora la integración bidireccional (nivel 3). Explota la **complementariedad** de forma específica: asigna la variabilidad lingüística al LLM y la gobernabilidad al motor simbólico, con interfaces de complejidad creciente entre ambos.

Pero ACAI no es simplemente una instanciación genérica del programa neuro-simbólico. Tiene al menos tres especificidades que lo distinguen de otros sistemas neuro-simbólicos y que constituyen sus contribuciones propias al programa.

5.4.2 Las especificidades de ACAI

ACAI aporta al programa neuro-simbólico tres especificidades: la triple restricción, el LLM como meta-herramienta, y la accountability como métrica central.

La primera especificidad es la **triple restricción** (I1): el lenguaje de representación debe ser simultáneamente legible por un humano, procesable por un motor, y generable por un LLM. La tercera restricción — la generabilidad — es completamente nueva en el panorama neuro-simbólico. Los formalismos clásicos de representación del conocimiento nunca se diseñaron para ser producidos por un modelo generativo; los lenguajes formales del paradigma neuro-simbólico asumen que las representaciones se codifican manualmente o se aprenden por el propio sistema. Que un LLM pueda generar fiablemente representaciones formales correctas es un requisito que la era de los modelos generativos hace posible plantear y que ACAI coloca en el centro de su diseño.

🔗 **Referencia contextual.** Este concepto se desarrolla en profundidad en **UD 9** y **UD 11**. La evaluación de formalismos contra la triple restricción y el análisis de sus tensiones se desarrollan en profundidad.

La segunda especificidad es el uso del **LLM como meta-herramienta** (I2): el LLM no solo participa en la operación del sistema conversacional sino que construye las representaciones formales que luego lo gobiernan. Este meta-uso — el modelo genera la maquinaria que lo restringe — no tiene precedente directo en la literatura neuro-simbólica. Los sistemas neuro-simbólicos clásicos asumen que las representaciones simbólicas se proporcionan externamente; ACAI propone que el propio LLM las genere, bajo supervisión humana, a partir de descripciones en lenguaje natural.

🔗 **Referencia contextual.** Este concepto se desarrolla en profundidad en **UD 10**. La formalización asistida por LLM y los experimentos de generabilidad se desarrollan como unidad con entregable directo para la Fase 0.

La tercera especificidad es que la **accountability** es la métrica central de evaluación, no una propiedad secundaria. En la mayoría de los sistemas neuro-simbólicos, la métrica principal es el rendimiento en una tarea (accuracy, F1, BLEU). En ACAI, la métrica principal es la calidad de la accountability: ¿son las decisiones trazables? ¿Son verificables? ¿Son consistentes? El rendimiento en la tarea conversacional es importante pero subordinado: un sistema que conversa bien pero no es accountable fracasa en el objetivo de ACAI, mientras que un sistema que es plenamente accountable pero menos fluido cumple el objetivo y puede mejorarse iterativamente.

5.4.3 Lo que ACAI toma y lo que aporta

Es importante distinguir con claridad qué toma ACAI del programa neuro-simbólico y qué aporta a él. ACAI toma el marco conceptual ge-

neral (la complementariedad de paradigmas), la taxonomía de niveles (como guía de progresión), y un conjunto de técnicas concretas (constrained decoding, structured output, semantic parsing con restricciones) que la **UD 7** desarrollará en detalle.

Lo que ACAI aporta es la demostración de que, en el dominio específico de la IA conversacional, una integración neuro-simbólica diseñada para la accountability puede producir propiedades — trazabilidad, verificabilidad, consistencia — que ningún paradigma por separado puede ofrecer. Si esta demostración tiene éxito, contribuye al programa neuro-simbólico con un caso empírico detallado, con un lenguaje de representación nuevo (diseñado bajo la triple restricción), y con una metodología de evaluación centrada en la accountability que puede adaptarse a otros dominios.

ACAI como contribución al programa neuro-simbólico

ACAI no es un consumidor pasivo del paradigma neuro-simbólico sino un contribuyente activo. Si el sprint produce sus entregables, el campo neuro-simbólico tendrá: (a) un caso empírico de integración en niveles 1–3 de Kautz con evaluación comparativa entre niveles, (b) un lenguaje de representación diseñado bajo una restricción inédita (generabilidad por LLM), (c) un pipeline de formalización asistida que aborda el cuello de botella de la ingeniería del conocimiento con herramientas generativas, y (d) una metodología de evaluación centrada en accountability.

Conexiones con otras unidades

Prerequisito: UD 1 (Mapa conceptual ACAI) para los principios P1, P3 y los frentes de innovación I1–I3. La UD5 puede estudiarse en paralelo con el Eje A (UD2–UD4), pero las conexiones con el diagnóstico del problema son más ricas si se ha completado al menos la UD2.

Complementa: UD 2 (La conversación como fenómeno) proporcionó el diagnóstico de la brecha de representación desde la perspectiva del fenómeno conversacional. UD5 proporciona el paradigma que fundamenta la solución a esa brecha.

Complementa: UD 4 (Accountability y marco regulatorio) definió las propiedades que el sistema debe satisfacer. UD5 muestra que solo la integración neuro-simbólica puede satisfacerlas simultáneamente con la capacidad lingüística necesaria.

Aplicación: UD 6 (Arquitecturas y patrones) toma la taxonomía

de Kautz y la concreta en arquitecturas implementables. UD5 proporciona el marco; UD6 proporciona las piezas de construcción.

Aplicación: UD 7 (Técnicas neuro-simbólicas para NLP) toma las capacidades de nivel 2 y 3 y las materializa en técnicas concretas: constrained decoding, structured output, semantic parsing con restricciones. UD5 fundamenta la necesidad; UD7 proporciona las herramientas.

Ejercicios

1. [Análisis] Para el escenario de la aseguradora (cancelación de póliza con consulta de bonificación), describa cómo manejaría el diálogo completo un sistema puramente simbólico (primera ola) y un sistema puramente neuronal (segunda ola). Para cada caso, identifique con precisión: (a) qué hace bien, (b) dónde falla, y (c) qué propiedad del paradigma causa el fallo. Conecte su análisis con la tabla de fortalezas y debilidades de la sección 5.1.2.

2. [Clasificación] Para cada uno de los siguientes sistemas, determine en qué nivel de la taxonomía de Kautz opera y justifique su clasificación: (a) un chatbot basado en RASA con un modelo BERT para NLU y un gestor de diálogo basado en reglas, (b) un LLM con function calling que invoca una base de datos SQL, (c) un sistema que usa constrained decoding para forzar al LLM a generar respuestas en un formato JSON válido según un esquema de dominio, (d) un sistema donde las trazas conversacionales se usan para refinar las reglas del motor simbólico mediante aprendizaje supervisado.

3. [Diseño] Proponga un mecanismo concreto de nivel 3 (integración bidireccional) para el siguiente escenario: el usuario dice algo ambiguo («No estoy seguro de lo que me conviene») en un punto del diálogo donde hay múltiples interpretaciones posibles (¿pide información? ¿quiere cancelar el trámite? ¿solicita tiempo para pensar?). Describa cómo combinaría la distribución de probabilidad del LLM con las restricciones del estado simbólico para producir una interpretación ordenada por plausibilidad. Especifique qué información aporta cada componente y cómo se integran.

4. [Reflexión crítica] Marcus (2020) argumenta que el deep learning tiene limitaciones estructurales que no se resolverán con más datos ni con modelos más grandes. LeCun, Bengio y Hinton, en su respuesta, argumentan que las técnicas de aprendizaje profundo

seguirán avanzando y eventualmente resolverán los problemas que Marcus identifica. ¿Cómo afecta la posición que se adopte en este debate al diseño de ACAI? Si Marcus tiene razón, ¿qué implicaciones tiene para la integración neuro-simbólica? Si los defensores del deep learning tienen razón, ¿pierde ACAI su razón de ser?

5. [Aplicación a ACAI] La Phase 2 de ACAI debe demostrar que la integración de nivel 2–3 mejora respecto a la Phase 1 (nivel 1–2). Diseñe un experimento comparativo que permita medir esta mejora. Especifique: (a) qué dimensión de rendimiento o accountability mediría, (b) qué escenarios de prueba usaría, (c) qué resultado esperaría del sistema de nivel 1–2 y qué del sistema de nivel 2–3, y (d) qué resultado invalidaría la hipótesis de que la integración profunda aporta valor.

Referencias y lecturas

Lectura de entrada

Garcez, A. d'Avila & Lamb, L. C. (2023). Neurosymbolic AI: The 3rd Wave. *Artificial Intelligence Review*, 56, 12387–12406. El artículo programático que define las tres olas y establece la agenda de la IA neuro-simbólica. Accesible, bien argumentado, y directamente relevante para entender la posición de ACAI en el campo. Lectura de entrada imprescindible.

Lecturas esenciales

- Kautz, H. (2022). The Third AI Summer: AAAI Robert S. Engelmore Memorial Lecture. *AI Magazine*, 43(1), 93–104. — La taxonomía de cinco niveles que estructura las fases de ACAI. Breve, claro, y fundacional para esta unidad.
- Hitzler, P., Sarker, M. K. & Eberhart, A. (Eds.) (2022). *Neuro-Symbolic Artificial Intelligence: The State of the Art*. IOS Press. — El volumen colectivo de referencia. 25 contribuciones que cubren el paisaje completo. No es lectura de entrada sino de referencia: consultar las contribuciones específicas según necesidad.
- Lake, B. M., Ullman, T. D., Tenenbaum, J. B. & Gershman, S. J. (2017). Building Machines That Learn and Think Like People. *Behavioral and Brain Sciences*, 40, e253. — El argumento sobre la necesidad de composicionalidad y modelos causales. Relevante para entender las limitaciones del paradigma conexionista puro.
- Marcus, G. (2020). The Next Decade in AI: Four Steps Towards Robust Artificial Intelligence. *ArXiv preprint arXiv:2002.06177*. — El análisis más

articulado de las limitaciones del deep learning y la necesidad de integración con representaciones simbólicas.

Lecturas complementarias

- Newell, A. & Simon, H. A. (1976). Computer Science as Empirical Inquiry: Symbols and Search. *Communications of the ACM*, 19(3), 113–126. — La hipótesis del sistema de símbolos físicos. Lectura fundacional de valor histórico.
- Rumelhart, D. E., Hinton, G. E. & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536. — Retropropagación: el fundamento técnico del conexionismo moderno. Breve y fundacional.
- Fodor, J. A. & Pylyshyn, Z. W. (1988). Connectionism and cognitive architecture: A critical analysis. *Cognition*, 28(1–2), 3–71. — La crítica clásica del conexionismo desde la perspectiva de la composicionalidad. Relevante para P1.
- Bengio, Y. (2019). From System 1 Deep Learning to System 2 Deep Learning (conferencia invitada, NeurIPS 2019). — El reconocimiento desde dentro del conexionismo de la necesidad de composicionalidad y razonamiento. Relevante como complemento a Marcus (2020).
- Badreddine, S., Garcez, A. d'Avila, Serafini, L. & Spranger, M. (2022). Logic Tensor Networks. *Artificial Intelligence*, 303, 103649. — Representación de fórmulas lógicas como operaciones tensoriales. Ejemplo de integración en niveles 3–4.
- Manhaeve, R., Dumančić, S., Kimmig, A., Demeester, T. & De Raedt, L. (2021). Neural Probabilistic Logic Programming in DeepProbLog. *Artificial Intelligence*, 298, 103504. — Integración de redes neuronales con programación probabilística lógica. Ejemplo de integración en niveles 3–4.
- Hamilton, K., Nayak, A., Božić, B. & Longo, L. (2022). Is Neuro-Symbolic AI Meeting its Promises in Natural Language Processing? A Structured Review. *Semantic Web*, 13(6), 1047–1074. — Revisión del estado de la integración neuro-simbólica en NLP. Puente directo hacia la UD7.
- McCulloch, W. S. & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 5, 115–133. — Neuronas artificiales: el origen histórico del conexionismo. Valor arqueológico.
- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408. — El perceptrón. Valor histórico.
- Feigenbaum, E. A. (1977). The Art of Artificial Intelligence: Themes and Case Studies of Knowledge Engineering. *IJCAI-77*. — El cuello de botella de la adquisición de conocimiento. Relevante para I2.

MONOGRÁFICO FORMATIVO

Fundamentos de IA Conversacional Neuro-Simbólica

«Architecture is the allocation of decisions to the components best equipped to make them.»

Eje B — El paradigma neuro-simbólico

Unidad 6 de 12

Research Sprint ACAI · Proyecto $(AI)^2$:TDG · 2026

Eje B — El paradigma neuro-simbólico

Prerequisitos: UD1, UD5 · Eje B — El paradigma neuro-simbólico

UNIDAD DIDÁCTICA 6

Arquitecturas y patrones de integración neuro-simbólica

Contenidos de la unidad

6.1.1 Los tres ejes de decisión 3
Dirección de control 4
Granularidad de la interfaz 4
Timing de la integración 5
6.2.1 El pipeline gobernado 6
6.2.2 Neural con guardrails simbólicos 7
6.2.3 Symbolic con fallback neuronal 8
6.2.4 Structured prompting 9
6.3.1 Constrained decoding 10
6.3.2 El reasoning loop neuro-simbólico 12
6.3.3 Aprendizaje simbólico desde trazas 13
6.4.1 Una arquitectura, múltiples patrones 14
6.4.2 El information state como pilar integrador 15
6.5.1 Trazabilidad vs. flexibilidad 16
6.5.2 Latencia vs. profundidad de integración 16
6.5.3 Complejidad vs. verificabilidad 17

¿Por qué necesito esto para ACAI?

La UD 5 estableció el mapa teórico del paradigma neuro-simbólico: la tensión fundacional, las tres olas, la taxonomía de Kautz. Pero un mapa teórico no se construye: se necesitan piezas. Esta unidad proporciona las piezas de construcción — patrones arquitectónicos concretos que implementan la integración neuro-simbólica a diferentes niveles de profundidad. Al terminar, el investigador dispondrá de un repertorio de patrones evaluados contra las propiedades de accountability, entenderá cómo la arquitectura ACAI se compone a partir de esos patrones, y podrá analizar los trade-offs que toda decisión arquitectónica implica.

Objetivos de aprendizaje

Al completar esta unidad, el investigador será capaz de:

1. Identificar los tres ejes de decisión arquitectónica — dirección de control, granularidad de interfaz, timing de integración — y justificar las elecciones de ACAI en cada eje.
2. Describir los patrones arquitectónicos de nivel 1–2 (pipeline gobernado, neural con guardrails simbólicos, symbolic con fallback neuronal, structured prompting) y evaluar sus propiedades de accountability.
3. Describir los patrones de nivel 3 (constrained decoding, reasoning loop neuro-simbólico, aprendizaje simbólico desde trazas) y analizar qué ganan y qué pierden respecto a los patrones de nivel 1–2.
4. Explicar cómo la arquitectura ACAI se compone a partir de múltiples patrones que operan en diferentes momentos y niveles.
5. Evaluar los trade-offs de trazabilidad vs. flexibilidad, latencia vs. profundidad, y complejidad vs. verificabilidad en el diseño de sistemas neuro-simbólicos.

6.1 Principios de diseño para la integración

Sede principal de este concepto.

6.1.1 Los tres ejes de decisión

El diseño de una arquitectura neuro-simbólica no es un problema con una solución única. Es un espacio de decisiones donde diferentes

Toda arquitectura neuro-simbólica es el resultado de tres decisiones: quién orquesta, qué se comunican los componentes, y cuándo interactúan.

elecciones producen sistemas con propiedades radicalmente diferentes. Antes de presentar patrones concretos, es necesario mapear este espacio identificando los ejes de decisión que lo organizan. Garcez et al. (2019), en su trabajo sobre integración neuronal-simbólica, propusieron una taxonomía de dimensiones de diseño que puede refinarse, para los propósitos de ACAI, en tres ejes fundamentales.

Garcez et al. (2019) — dimensiones de diseño para la integración neuro-simbólica.

Dirección de control

El primer eje es la **dirección de control**: ¿qué componente orquesta el flujo de procesamiento? Esta decisión es la más consequencial para las propiedades del sistema porque determina dónde residen las decisiones operativas.

En un patrón *symbolic-in-the-loop*, el componente simbólico orquesta y el neuronal es un servicio invocado para tareas específicas. Las decisiones operativas las toma el motor simbólico; el LLM proporciona capacidad lingüística bajo demanda. La trazabilidad de las decisiones es directa porque el proceso de decisión es simbólico y, por tanto, inspectable.

En un patrón *neural-in-the-loop*, el componente neuronal orquesta y el simbólico es una herramienta invocada cuando se necesita. Es el patrón de los frameworks agénticos actuales: el LLM decide cuándo consultar una base de datos, cuándo aplicar una regla, cuándo invocar un servicio. Como analizó la **UD 3**, este patrón produce flexibilidad a costa de la gobernabilidad.

En un patrón de *co-orquestación*, ambos componentes tienen capacidad de iniciativa y se coordinan mediante un protocolo de interacción. Es el patrón más rico pero también el más complejo, y la trazabilidad requiere registrar las decisiones de coordinación.

ACAI elige deliberadamente *symbolic-in-the-loop* para las decisiones operativas.

ACAI elige *symbolic-in-the-loop* para las decisiones operativas como consecuencia directa del análisis de accountability. Si el objetivo es que las decisiones sean verificables contra una especificación formal, el componente que las toma debe operar sobre representaciones formales. Esto no significa que el LLM no tenga autonomía alguna: la tiene para la interpretación lingüística y la generación de texto. Lo que no tiene es autonomía para las decisiones de negocio — qué información proporcionar, qué acción ejecutar, qué política aplicar.

Granularidad de la interfaz

El segundo eje es la **granularidad de la interfaz**: ¿qué riqueza tienen las representaciones que fluyen entre los componentes? Este eje opera en un continuo que va desde interfaces mínimas hasta interfaces ricas.

Una *interfaz mínima* transmite strings: el motor envía un texto al LLM y recibe un texto como respuesta. Es la interfaz de nivel 1 en la taxonomía de Kautz. Su simplicidad es una ventaja (fácil de implementar, fácil de depurar) pero su pobreza es una limitación (no transmite incertidumbre, no especifica restricciones, no proporciona contexto estructurado).

Una *interfaz estructurada* transmite objetos tipados: el motor envía al LLM no solo un texto sino un objeto que incluye el texto del usuario, el estado relevante del diálogo (en formato estructurado), las restricciones sobre la respuesta esperada (esquema JSON, actos comunicativos válidos, entidades del dominio), y metadatos (nivel de confianza requerido, política de generación). El LLM responde con un objeto que incluye la interpretación estructurada, señales de confianza, y candidatos alternativos. Es la interfaz de nivel 2, y la que ACAI implementa en la Phase 1.

Una *interfaz computacional* permite que un componente intervenga en el proceso de cómputo del otro: no solo envía datos sino que modifica restricciones sobre cómo se procesan. El constrained decoding, donde el motor simbólico modifica las probabilidades de generación del LLM token a token, es una interfaz computacional. Es la interfaz de nivel 3.

La lección de la **UD 2** sobre los pipelines NLU–DM–NLG es directamente relevante aquí: aquellos sistemas fallaron porque la interfaz entre módulos — el par (intent, slots) — era demasiado pobre para transmitir la riqueza de la comprensión lingüística. ACAI debe diseñar interfaces suficientemente ricas para preservar la información necesaria sin perder la separación de competencias.

Timing de la integración

El tercer eje es el **timing de la integración**: ¿en qué momento del ciclo de vida del sistema interactúan los componentes?

La integración en *design time* ocurre durante la construcción del sistema, no durante su operación. El caso paradigmático en ACAI es la formalización asistida (I2): el LLM ayuda a construir las representaciones formales a partir de descripciones en lenguaje natural, pero una vez compiladas, las representaciones son puramente simbólicas y el LLM no participa en su ejecución. La ventaja es que las representaciones resultantes pueden verificarse antes del despliegue; la desventaja es que la adaptación a situaciones no previstas requiere un nuevo ciclo de design time.

La integración en *runtime* ocurre durante la operación del sistema, en cada turno de la conversación. El pipeline gobernado de ACAI opera en runtime: el LLM interpreta, el motor decide, el LLM genera. La ventaja es la adaptación en tiempo real; la desventaja es que la calidad depende

ACAI integra en design time (formalización asistida) y en runtime (pipeline gobernado), con diferentes propiedades.

de la fiabilidad del LLM en cada turno.

La integración en *learning time* ocurre cuando las trazas de operación se analizan para mejorar el sistema. Las trazas conversacionales de la Phase 1 pueden analizarse para detectar patrones no cubiertos y proponer extensiones del modelo formal. Es una integración que cierra el ciclo: la operación informa al diseño.

ACAI opera en los tres momentos, y esta es una de sus particularidades arquitectónicas: la mayoría de los sistemas neuro-simbólicos operan solo en uno o dos.

6.2 Patrones arquitectónicos de nivel 1–2

Sede principal de este concepto.

Los patrones de nivel 1–2 en la taxonomía de Kautz comparten una característica común: la frontera entre los componentes neuronal y simbólico es una *interfaz* bien definida. Los componentes se comunican mediante representaciones intermedias pero operan internamente de forma independiente. Esto produce una propiedad valiosa: las decisiones de cada componente pueden analizarse por separado. En contrapartida, la rigidez de la interfaz limita la riqueza de la integración.

6.2.1 El pipeline gobernado

Pipeline gobernado

El **pipeline gobernado** es un patrón de nivel 1–2 en el que el componente simbólico orquesta un flujo secuencial: interpretación (neuronal) → decisión (simbólico) → generación (neuronal). El motor simbólico determina qué se interpreta, cómo se decide, y qué se genera; el LLM ejecuta las tareas lingüísticas bajo las restricciones del motor.

Phase 1
Patrón central de la arquitectura.

El pipeline gobernado es el patrón central de la Phase 1 de ACAI y opera en dos modos complementarios.

El *modo de compilación* (design time) transforma conocimiento conversacional expresado en lenguaje natural en representaciones formales. Un experto de dominio describe, por ejemplo, las condiciones de cancelación de una póliza en prosa. El LLM, guiado por un prompt estructurado que incluye el esquema del formalismo ACAI, genera una representación formal de esas condiciones. Un validador simbólico verifica la corrección formal de la representación. Un experto humano revisa y aprueba. El resultado es un modelo formal verificado que no depende

del LLM para su ejecución. Este modo implementa I2 (LLM como meta-herramienta) y opera en nivel 2: el LLM no genera libremente sino que está condicionado por el esquema del formalismo.

El *modo de ejecución* (runtime) procesa cada turno de la conversación. El flujo puede describirse con precisión. Primero, el LLM recibe el input del usuario junto con el estado relevante del diálogo (interfaz estructurada de nivel 2) y produce una interpretación en el formato esperado por el motor: acto comunicativo, entidades, señales de *grounding*. Segundo, el motor simbólico recibe la interpretación, consulta el modelo de conocimiento, evalúa las reglas aplicables, actualiza el information state, y determina la acción del sistema: qué información proporcionar, qué preguntar, qué acción ejecutar, qué confirmar. Tercero, el LLM recibe la acción determinada por el motor junto con el contexto conversacional y las políticas de generación, y produce la respuesta en lenguaje natural. La traza registra cada paso con la regla aplicada, los datos consultados, y el information state resultante.

Las propiedades de accountability del pipeline gobernado son directas. La *trazabilidad* es completa para las decisiones simbólicas: cada decisión se rastrea hasta la regla que la produjo y el information state sobre el que operó. La *verificabilidad* es posible: las reglas pueden comprobarse formalmente antes del despliegue. La *consistencia* es garantizada para las decisiones operativas: el mismo information state y las mismas reglas producen la misma decisión.

Pan et al. (2023) — integración de LLMs con bases de conocimiento como pipeline gobernado.

Las limitaciones son igualmente claras. La calidad del sistema depende críticamente de la calidad de la interpretación del LLM en el paso 1: si el LLM malinterpreta un input, la decisión simbólica será correcta respecto a la interpretación errónea pero incorrecta respecto a la intención real del usuario. La interfaz entre NLU y motor es un cuello de botella: debe ser suficientemente rica para transmitir las dimensiones relevantes del acto comunicativo pero suficientemente estructurada para ser procesable por el motor. Y la rigidez del pipeline hace que las situaciones no previstas por las reglas (los *unknown unknowns*) produzcan fallos predecibles (no hay regla que aplicar) en lugar de degradaciones graduales.

6.2.2 Neural con guardrails simbólicos

Neural con guardrails simbólicos

En el patrón **neural con guardrails simbólicos**, un LLM opera como sistema principal y un componente simbólico filtra, valida o modifica su output según reglas formales. La dirección de control

es neural-in-the-loop: el LLM genera y lo simbólico restringe.

Este patrón es el más extendido en la industria actual y merece un análisis cuidadoso porque es la alternativa más visible a la arquitectura ACAI. NeMo Guardrails (Rebedea et al., 2023) implementa este patrón con un lenguaje de especificación (Colang) que define flujos conversacionales y restricciones que se aplican sobre la generación del LLM. Llama Guard (Inan et al., 2023) implementa una variante donde el filtro es él mismo un modelo neuronal entrenado sobre una taxonomía de categorías de seguridad. Guardrails AI implementa una versión programática donde las restricciones son funciones Python que validan el output.

Rebedea et al. (2023): NeMo Guardrails y Colang como ejemplo industrial del patrón.

El patrón tiene virtudes reales. Es fácil de desplegar: se añade sobre un sistema existente sin rediseñar la arquitectura. Es modular: los guardrails pueden añadirse, modificarse o eliminarse independientemente. Es efectivo para restricciones simples: filtrar contenido prohibido, validar formatos, detectar temas fuera de alcance.

Pero para la accountability, sus limitaciones son fundamentales, y se concentran en la dirección de control. El LLM toma las decisiones y el guardrail las filtra. Esto significa que el guardrail solo puede actuar sobre decisiones que el LLM ya ha tomado: puede rechazar una respuesta pero no puede determinar cuál sería la respuesta correcta. Puede detectar una violación de política pero no puede garantizar que la respuesta que no viola la política sea consistente con las decisiones anteriores. El guardrail opera sobre la *superficie* (el texto generado), no sobre el *proceso* (las razones que produjeron el texto). Y la traza que produce es una traza de filtrado (qué se bloqueó y por qué), no una traza de decisión (por qué se decidió lo que se decidió).

La diferencia con el pipeline gobernado de ACAI es arquitectónica, no de grado. En ACAI, el motor simbólico determina qué responder *antes* de que el LLM genere. En el patrón de guardrails, el LLM genera y el simbólico filtra *después*. El primero produce accountability constitutiva; el segundo produce control superficial.

6.2.3 Symbolic con fallback neuronal

Symbolic con fallback neuronal

En el patrón **symbolic con fallback neuronal**, un sistema de reglas opera como componente principal y recurre al LLM cuando ninguna regla cubre la situación actual. La dirección de control es symbolic-in-the-loop cuando las reglas aplican, y se pierde cuando el sistema recurre al fallback.

Este patrón es común en sistemas de diálogo híbridos como RASA con fallback a LLM, y en chatbots corporativos que combinan flujos predefinidos con una capacidad generativa para preguntas no anticipadas. Su atractivo es pragmático: permite construir un sistema que maneja los casos conocidos con reglas y los desconocidos con el LLM.

La limitación principal es que la transición al fallback es un punto de pérdida de gobernabilidad. Cuando el sistema activa el fallback, pasa de decisiones trazables a decisiones opacas, y la traza registra «fallback activado» sin más detalle. Para un auditor, esto es un vacío: sabe que el sistema tomó una decisión pero no puede verificar si fue correcta. Además, la decisión de *cuándo* activar el fallback — cuándo las reglas "no cubren" la situación — es en sí misma una decisión que puede ser incorrecta: el sistema puede activar el fallback cuando una regla aplicaba (pérdida de gobernabilidad innecesaria) o no activarlo cuando debería (respuesta incorrecta basada en reglas inadecuadas).

ACAI se distingue de este patrón en un aspecto importante: no tiene un "modo fallback" donde el LLM opere sin gobernanza. Cuando el motor simbólico no tiene reglas para una situación, puede escalar a un operador humano (una acción gobernada con traza), solicitar información adicional al usuario (una acción gobernada con traza), o informar de sus limitaciones (una acción gobernada con traza). Lo que no hace es delegar la decisión al LLM de forma no gobernada.

6.2.4 Structured prompting

Structured prompting

El **structured prompting** es un patrón de nivel 2 en el que el estado simbólico se inyecta en el prompt del LLM como contexto estructurado que condiciona su operación. El motor no solo invoca al LLM sino que configura su contexto de procesamiento con información estructurada: esquema de la respuesta esperada, entidades del dominio relevantes, restricciones del estado del diálogo, y políticas de generación.

El structured prompting ha evolucionado significativamente con la maduración de las APIs de LLMs. Las técnicas de *structured output* — donde la API del LLM garantiza que la respuesta se ajusta a un esquema JSON predefinido — representan una implementación industrializada del nivel 2. OpenAI (2024) introdujo Structured Outputs como una feature que garantiza conformidad con un *JSON Schema* proporcionado. Anthropic y Google ofrecen mecanismos análogos. Estas implementaciones resuelven un problema práctico del nivel 1: la necesidad de parsear

OpenAI (2024) — Structured Outputs: JSON mode garantizado por esquema.

texto libre para extraer información estructurada, que era una fuente frecuente de errores.

Para ACAI, el structured prompting es el mecanismo principal de la interfaz motor → LLM en la Phase 1. Cuando el motor invoca al LLM para NLU, le proporciona el esquema del acto comunicativo esperado, las entidades relevantes del dominio, el estado del diálogo, y las restricciones de interpretación. Cuando lo invoca para NLG, le proporciona la acción determinada, el contexto conversacional, la política de generación aplicable, y el formato de respuesta. En ambos casos, el LLM opera dentro de un espacio condicionado por el estado simbólico.

Structured Outputs: el LLM garantiza que la respuesta se ajusta a un esquema. Nivel 2 industrializado.

La ventaja del structured prompting respecto a la interfaz de strings del nivel 1 es sustancial: reduce significativamente los errores de interpretación (el LLM sabe qué buscar) y de generación (el LLM sabe qué formato producir). La limitación es que el condicionamiento opera a nivel de prompt, no de computación: el LLM recibe instrucciones más precisas pero sigue interpretándolas probabilísticamente. Un esquema JSON garantiza el formato pero no la corrección semántica del contenido.

6.3 Patrones arquitectónicos de nivel 3

Sede principal de este concepto.

Los patrones de nivel 3 se distinguen de los anteriores en un aspecto fundamental: la frontera entre componentes deja de ser una interfaz y se convierte en una zona de interacción. Los componentes no solo intercambian datos sino que se influyen mutuamente durante el procesamiento. Esto produce una integración más profunda con capacidades que los niveles anteriores no tienen, pero a costa de una complejidad significativamente mayor.

6.3.1 Constrained decoding

Constrained decoding

El **constrained decoding** es un patrón de nivel 3 en el que el componente simbólico interviene *durante* la generación del LLM, modificando las probabilidades de los tokens a generar para garantizar que la salida satisface restricciones formales. El componente simbólico no actúa antes (como en el structured prompting) ni después (como en los guardrails) sino *durante* la computación neuronal.

El mecanismo es técnicamente preciso. En cada paso de la generación autoregresiva, el LLM produce una distribución de probabilidad sobre el vocabulario completo de tokens. Antes de muestrear el siguiente token, una máscara derivada de una gramática formal o un conjunto de restricciones anula la probabilidad de los tokens que violarían las restricciones. El LLM solo puede generar tokens permitidos, pero dentro de esos tokens, la distribución de probabilidad refleja las preferencias aprendidas del modelo. El resultado es una salida que es simultáneamente natural (refleja los patrones del LLM) y formalmente correcta (satisface las restricciones simbólicas).

El constrained decoding es la implementación más directa de nivel 3: lo simbólico interviene durante la computación neuronal, no solo antes o después.

Willard y Louf (2023), en su trabajo sobre Outlines, demostraron que el constrained decoding con gramáticas formales es práctico y eficiente: el overhead computacional es moderado y la calidad de la generación se mantiene o incluso mejora (porque las restricciones eliminan salidas degeneradas). Su implementación permite definir restricciones como gramáticas context-free, expresiones regulares, o esquemas JSON, y las aplica durante la generación con un autómata que se ejecuta en paralelo con el modelo.

Willard & Louf (2023) — Outlines: constrained generation con gramáticas formales.

Zheng et al. (2023), con SGLang, propusieron un enfoque complementario que integra las restricciones de generación con el control del flujo del programa, permitiendo intercalar generación libre con generación restringida dentro de una misma secuencia. Esto es directamente relevante para ACAI: en un turno conversacional, algunas partes de la respuesta pueden estar fuertemente restringidas (datos factuales, compromisos formales) mientras que otras pueden ser más libres (expresiones de cortesía, transiciones discursivas).

Zheng et al. (2023): SGLang — intercalar generación libre y restringida en una misma secuencia.

Para la Phase 2 de ACAI, el constrained decoding ofrece una ganancia específica respecto al structured prompting de la Phase 1. En la Phase 1, el LLM genera una interpretación dentro de un esquema, pero la corrección semántica depende de que el LLM siga las instrucciones probabilísticamente. En la Phase 2 con constrained decoding, el LLM solo puede generar interpretaciones que satisfacen la gramática del formalismo ACAI. La corrección *sintáctica* está garantizada por construcción; la corrección *semántica* sigue dependiendo del LLM, pero el espacio de errores posibles se reduce drásticamente.

6.3.2 El reasoning loop neuro-simbólico

Reasoning loop neuro-simbólico

El **reasoning loop** es un patrón de nivel 3 en el que el LLM y el motor simbólico interactúan en múltiples ciclos dentro de un mismo turno conversacional. El LLM propone, el motor evalúa, y si la propuesta no satisface las restricciones, el motor proporciona feedback estructurado que condiciona la siguiente propuesta del LLM. El ciclo continúa hasta que se alcanza una propuesta válida o se agota un presupuesto de iteraciones.

El reasoning loop extiende el pipeline gobernado de un flujo lineal (NLU → decisión → NLG) a un flujo iterativo donde la interfaz neuronal-simbólica se cruza múltiples veces. La diferencia es análoga a la que existe entre un compilador de una pasada y un compilador con refinamiento iterativo: el segundo produce mejores resultados porque puede corregir errores y refinar aproximaciones.

Nye et al. (2021): *scratchpad reasoning* como proto-reasoning loop.

Nye et al. (2021), en su trabajo sobre *scratchpad reasoning*, demostraron que los modelos de lenguaje producen resultados significativamente mejores cuando pueden generar pasos intermedios que son evaluados y corregidos. Aunque su trabajo no usaba un motor simbólico explícito, la estructura es análoga: generación → evaluación → corrección → regeneración.

En el contexto de ACAI, el reasoning loop puede implementarse en al menos tres situaciones donde el pipeline lineal de la Phase 1 es insuficiente.

La primera situación es la *interpretación de inputs ambiguos*. Cuando el LLM produce una interpretación que el motor evalúa como ambigua (el acto comunicativo no se determina unívocamente), el motor puede devolver al LLM las hipótesis alternativas con sus restricciones respectivas y solicitar una desambiguación informada. El LLM no genera una segunda interpretación desde cero sino condicionada por el análisis simbólico de la ambigüedad.

La segunda situación es la *generación de respuestas complejas*. Cuando la respuesta del sistema debe integrar múltiples componentes — información factual, compromisos, opciones, solicitudes de confirmación —, el motor puede guiar la generación mediante un ciclo: generar un borrador, evaluar su completitud y corrección contra los requisitos simbólicos, proporcionar feedback sobre lo que falta o es incorrecto, regenerar.

La tercera situación es la *formalización asistida* (design time). El pipeline de compilación de la Phase 1 puede ser un reasoning loop: el LLM genera una primera formalización, el validador simbólico detecta erro-

res formales, el LLM corrige condicionado por el feedback del validador, y el ciclo continúa hasta obtener una representación válida.

Reasoning loop para interpretación ambigua

Input: «Me gustaría cambiar lo del seguro.»
Ciclo 1: LLM interpreta → acto: `solicitar_cambio`, objeto: `seguro` (sin especificar producto ni tipo de cambio). Motor evalúa: interpretación infraespecificada, múltiples lecturas compatibles (cambiar cobertura, cambiar titular, cancelar, modificar forma de pago). Feedback al LLM: «El input es compatible con los actos `modificar_cobertura`, `cambiar_titular`, `solicitar_cancelación`, `modificar_pago`. El usuario tiene dos pólizas activas: hogar (HG-2019-4421) y auto (AU-2021-0087). ¿Hay evidencia lingüística que discrimine entre estas interpretaciones?»
Ciclo 2: LLM reanaliza condicionado por las opciones concretas → la formulación «cambiar lo del seguro» con el uso de «lo del» sugiere referencia a un tema previo o a una situación conocida, no una operación técnica específica → clasificación: `solicitud_inespecífica` con alta incertidumbre. Motor decide: solicitar clarificación al usuario con las opciones pertinentes.
Resultado: El sistema no adivina la intención del usuario ni elige una interpretación arbitraria. La traza registra la ambigüedad, las hipótesis evaluadas, y la decisión de solicitar clarificación.

6.3.3 Aprendizaje simbólico desde trazas

Aprendizaje simbólico desde trazas

El **aprendizaje simbólico desde trazas** es un patrón de nivel 3 en el que las trazas de ejecución del sistema se analizan (usando técnicas neuronales) para detectar patrones no cubiertos por las reglas formales y proponer extensiones del modelo simbólico. La dirección de integración es inversa a la del pipeline gobernado: aquí es el componente neuronal el que alimenta la evolución del simbólico.

Este patrón implementa lo que la **UD 5** describió como I3b: la evolución del conocimiento simbólico a partir de evidencia empírica. A diferencia de los dos patrones anteriores, que operan en runtime, este opera en learning time — entre sesiones, no durante la conversación.

La mecánica puede describirse en cuatro pasos. Primero, el sistema acumula trazas conversacionales de la operación normal, incluyendo los casos donde el motor no encontró reglas aplicables, donde la interpretación del LLM fue corregida por el usuario, o donde el usuario expresó insatisfacción. Segundo, un proceso de análisis — que puede

usar el LLM como herramienta de procesamiento — clasifica estos casos e identifica patrones recurrentes que sugieren lagunas en el modelo formal. Tercero, el sistema propone extensiones o modificaciones del modelo formal: nuevas reglas, nuevas entidades, nuevos flujos. Cuarto, un experto humano evalúa las propuestas, las aprueba, modifica o descarta, y las incorpora al modelo formal.

Este patrón es especialmente relevante para ACAI porque aborda simultáneamente dos problemas. Por un lado, cierra parcialmente el cuello de botella de la ingeniería del conocimiento (el modelo formal se enriquece a partir de la operación real, no solo del diseño manual). Por otro, proporciona un mecanismo de mejora continua que es compatible con la accountability: las extensiones propuestas son reglas formales que se verifican antes de incorporarse, no ajustes opacos de parámetros.

El aprendizaje desde trazas cierra parcialmente el cuello de botella de la ingeniería del conocimiento.

6.4 La arquitectura ACAI como composición de patrones

6.4.1 Una arquitectura, múltiples patrones

El análisis de patrones de las secciones anteriores podría sugerir que ACAI debe elegir uno. La realidad es más rica: ACAI compone múltiples patrones que operan en diferentes momentos del ciclo de vida y a diferentes niveles de integración, coordinados por el information state como estructura central.

ACAI no es un patrón arquitectónico único sino una composición deliberada de patrones que operan en diferentes momentos y niveles.

En *design time*, ACAI usa el pipeline de formalización asistida: un reasoning loop (nivel 3) donde el LLM genera representaciones formales, el validador simbólico las evalúa, y el ciclo itera hasta obtener un modelo formal correcto que un experto humano aprueba. El resultado es un modelo de conocimiento compilado que incluye reglas de diálogo, reglas de negocio, políticas de generación, y configuración del ecosistema operativo (E1–E4).

En *runtime Phase 1*, ACAI usa el pipeline gobernado (nivel 1–2) con structured prompting: el motor simbólico orquesta, invocando al LLM con interfaces estructuradas para NLU y NLG, tomando las decisiones operativas sobre el modelo formal, y actualizando el information state con cada turno. La gobernanza del ecosistema operativo (RAG gobernado, capacidad agéntica gobernada, alineamiento formalizado) opera dentro de este pipeline.

En *runtime Phase 2*, ACAI añade al pipeline gobernado los mecanismos de nivel 3: constrained decoding para garantizar que las interpretaciones y generaciones satisfacen restricciones formales, y reasoning loops para manejar ambigüedad y complejidad. La frontera rígida de

la Phase 1 se convierte en una zona de interacción más fluida, pero el motor simbólico retiene la autoridad sobre las decisiones operativas.

En *learning time*, ACAI usa el aprendizaje simbólico desde trazas para proponer extensiones del modelo formal a partir de la operación real, cerrando el ciclo design → runtime → learning → design.

6.4.2 El information state como pilar integrador

En esta composición de patrones, el **information state** — introducido en la **UD 3** — funciona como la estructura que conecta todos los componentes y todos los momentos. Es el objeto sobre el que el motor simbólico opera, lo que el LLM recibe como contexto estructurado, lo que las trazas registran, y lo que el aprendizaje desde trazas analiza. Cada patrón de la composición lee del information state, escribe en él, o lo usa como referencia.

Flujo completo en la arquitectura compuesta

Contexto: El usuario dice «Quiero cancelar mi seguro de hogar, pero no sé si me conviene» en el turno 3 de la conversación. La identidad está verificada desde el turno 1.

NLU (structured prompting, nivel 2): El motor envía al LLM el input del usuario junto con el information state (identidad verificada, pólizas activas: hogar HG-2019-4421 y auto AU-2021-0087) y el esquema de interpretación (actos posibles, entidades del dominio). El LLM produce: acto principal `solicitar_cancelación` condicionado a acto secundario `consultar_conveniencia`, producto `seguro_hogar_HG-2019-4421`.

Decisión (motor simbólico): El motor recibe la interpretación estructurada. Evalúa: (1) regla `cancel_requires_confirmation` → marcar solicitud como pendiente de confirmación; (2) regla `conditional_request_handling` → la solicitud está condicionada a una consulta, priorizar la resolución de la consulta; (3) regla `bonus_info_required` → el usuario pregunta por conveniencia en contexto de cancelación, activar consulta de condiciones de bonificación. Information state actualizado: subtarea `consulta_bonificación` abierta, solicitud de cancelación en estado `condicionada_a_consulta`.

Ecosistema (RAG gobernado + capacidad agéntica gobernada): El motor determina que se necesitan dos datos: condiciones de bonificación (fuente: Condiciones Generales v2025.2, sección Bonificaciones) y datos de siniestros del cliente (fuente: sistema de gestión, API `siniestros_cliente`). Ambas consultas se ejecutan con trazas que registran fuente, consulta, y resultado.

NLG (structured prompting, nivel 2): El motor envía al LLM la acción determinada (informar sobre bonificación, datos del cliente, política de comunicación empática por tratarse de una cancelación potencial) y el

> LLM genera la respuesta.
> **Traza:** Cada paso registra: regla aplicada, datos consultados, fuentes accedidas, information state antes y después.

6.5 Trade-offs arquitectónicos

6.5.1 Trazabilidad vs. flexibilidad

El primero y más fundamental de los trade-offs es la tensión entre la trazabilidad de las decisiones y la flexibilidad del sistema para manejar situaciones imprevistas. En un extremo, un sistema puramente simbólico con reglas exhaustivas tiene trazabilidad perfecta pero no puede manejar ninguna situación que las reglas no anticipen. En el otro extremo, un LLM puro puede manejar cualquier situación pero sin trazabilidad alguna.

Cada incremento de integración gana flexibilidad y pierde simplicidad de verificación. El arte está en encontrar el balance correcto para cada dominio.

El pipeline gobernado de la Phase 1 ocupa una posición específica en este continuo: alta trazabilidad (todas las decisiones operativas son simbólicas) con flexibilidad limitada (las situaciones no cubiertas por las reglas requieren escalación o extensión del modelo). La adición de mecanismos de nivel 3 en la Phase 2 desplaza el balance: la flexibilidad aumenta (el reasoning loop puede manejar situaciones ambiguas, el constrained decoding permite generación más rica) pero la trazabilidad se hace más compleja (las interacciones entre componentes añaden pasos que deben registrarse).

Para ACAI, el balance se resuelve con un principio: *las decisiones con consecuencias operativas son siempre simbólicas y trazables; la flexibilidad neuronal opera dentro de límites definidos por el motor simbólico*. La Phase 2 no introduce decisiones neuronales autónomas; introduce mecanismos que enriquecen la interacción neuronal-simbólica sin transferir la autoridad de decisión al componente neuronal.

6.5.2 Latencia vs. profundidad de integración

El segundo trade-off es práctico y tiene impacto directo en la experiencia del usuario. Cada interacción entre componentes añade latencia al procesamiento de un turno. En el pipeline gobernado de la Phase 1, un turno requiere al menos dos llamadas al LLM (NLU y NLG) más el procesamiento simbólico. En la Phase 2 con reasoning loop, un turno puede requerir múltiples ciclos de interacción LLM-motor, cada uno con su latencia.

En un sistema conversacional, la latencia percibida por el usuario es un factor crítico de la experiencia. Estudios de diseño de interacción han

establecido que respuestas por encima de 2–3 segundos degradan significativamente la percepción de naturalidad del diálogo. El constrained decoding, por su parte, añade overhead computacional en cada paso de generación, aunque las implementaciones recientes como Outlines han reducido este overhead significativamente.

El trade-off de latencia no invalida los patrones de nivel 3 pero exige disciplina en su aplicación.

La resolución de este trade-off en ACAI es selectiva: los mecanismos de nivel 3 no se aplican a todos los turnos sino solo cuando el procesamiento de nivel 2 es insuficiente. El reasoning loop se activa cuando la interpretación inicial es ambigua, no para todos los inputs. El constrained decoding se aplica a las partes de la respuesta que requieren corrección formal, no a toda la generación. Esta aplicación selectiva mantiene la latencia promedio cercana a la del pipeline de la Phase 1, con aumentos puntuales cuando la complejidad del turno lo requiere.

6.5.3 Complejidad vs. verificabilidad

El tercer trade-off afecta directamente a la accountability y es, por tanto, especialmente relevante para ACAI. Los patrones de nivel 3 producen sistemas más capaces pero más complejos de verificar. En el pipeline lineal de la Phase 1, la verificación del sistema puede descomponerse: se verifican las reglas simbólicas por un lado y se evalúa la calidad del LLM por otro. En los patrones de nivel 3, las interacciones entre componentes crean caminos de ejecución adicionales que deben verificarse: ¿qué pasa si el reasoning loop no converge? ¿Qué pasa si el constrained decoding elimina todos los tokens salvo uno degenerado? ¿Cómo se verifica la consistencia de un sistema donde las restricciones simbólicas y las distribuciones neuronales interactúan?

Referencia contextual. Este concepto se desarrolla en profundidad en **UD 12**. Las métricas de evaluación deben adaptarse para capturar la complejidad adicional de los patrones de nivel 3, incluyendo métricas de convergencia del reasoning loop y de calidad de la generación restringida.

La resolución en ACAI sigue el principio de la progresión deliberada (P3): la Phase 2 debe demostrar que cada mecanismo de nivel 3 añade valor medible que justifica la complejidad adicional. Si el reasoning loop mejora la calidad de la interpretación en un 15 % pero hace la verificación cuatro veces más compleja, la decisión de adoptarlo depende del balance concreto en el dominio de despliegue. La métrica central de la Phase 2 es precisamente esta: comparación cuantitativa entre niveles con beneficios y costes explícitos.

Conexiones con otras unidades

→ Prerequisito: UD 5 (Fundamentos neuro-simbólicos) para la taxonomía de Kautz y las fortalezas complementarias. **UD 1** (Mapa conceptual) para P1, P3, E1–E4.

⇄ Complementa: UD 3 (Ecosistema operativo) presentó los componentes E1–E4 y sus límites de gobernanza. UD6 muestra cómo los patrones arquitectónicos implementan la gobernanza de esos componentes.

Aplicación: UD 7 (Técnicas neuro-simbólicas para NLP) concreta los patrones de nivel 3 en técnicas específicas: constrained decoding, semantic parsing, structured generation. UD6 proporciona la arquitectura; UD7 proporciona las técnicas.

Aplicación: UD 9 (Formalismos) necesita saber qué patrones arquitectónicos usará el sistema para evaluar los formalismos candidatos: ¿es el formalismo compatible con el constrained decoding? ¿Se puede usar en el reasoning loop?

Aplicación: UD 10 (Formalización asistida) implementa directamente el pipeline de compilación y el reasoning loop descritos aquí.

Ejercicios

1. [Clasificación] Para cada uno de los siguientes sistemas, identifique qué patrón arquitectónico usa (o qué combinación de patrones), en qué nivel de Kautz opera, y cuál es la dirección de control: (a) un chatbot empresarial basado en GPT-4 con un system prompt extenso y NeMo Guardrails; (b) un sistema RASA con un modelo BERT para NLU, un gestor de diálogo basado en reglas, y templates para NLG; (c) un sistema donde un LLM genera SQL a partir de preguntas en lenguaje natural y un validador verifica que el SQL es sintáctica y semánticamente correcto antes de ejecutarlo; (d) un sistema donde las reglas de un chatbot se revisan mensualmente a partir de un análisis de conversaciones fallidas realizado con LLM.

2. [Diseño de interfaz] Diseñe la interfaz estructurada (nivel 2) entre el motor simbólico y el LLM para la tarea de NLU en el escenario de la aseguradora. Especifique: (a) qué información envía el motor al LLM (qué campos, qué tipos, qué restricciones), (b) qué información devuelve el LLM al motor (qué estructura tiene la interpretación), y (c) cómo se señala la incertidumbre cuando la

interpretación no es unívoca. Justifique la granularidad elegida: ¿por qué estos campos y no otros?

3. [Análisis de trade-offs] Considere un escenario donde el sistema ACAI debe procesar 500 conversaciones simultáneas, cada una con una latencia máxima de 3 segundos por turno. Analice: (a) cuántas llamadas al LLM requiere un turno en el pipeline de Phase 1 (nivel 2), (b) cuántas llamadas requiere un turno con reasoning loop (nivel 3) en el peor caso, (c) qué estrategias de optimización podrían reducir la latencia sin sacrificar la profundidad de integración.

4. [Reflexión crítica] El patrón "neural con guardrails simbólicos" es significativamente más fácil de implementar que el pipeline gobernado de ACAI y produce resultados aceptables en muchos escenarios. ¿En qué condiciones concretas sería preferible el patrón de guardrails al pipeline gobernado? ¿Hay dominios o tipos de conversación donde la accountability plena no justifica la complejidad arquitectónica adicional? Argumente su posición.

5. [Aplicación a ACAI] Diseñe un reasoning loop para la formalización asistida (design time): un experto de dominio describe en lenguaje natural las condiciones de renovación automática de una póliza de hogar. Especifique: (a) el prompt inicial al LLM con el esquema del formalismo, (b) al menos dos tipos de error que el validador simbólico podría detectar, (c) cómo el feedback del validador se estructura para que el LLM corrija los errores, y (d) las condiciones de terminación del loop (cuándo se acepta el resultado, cuándo se pide intervención humana).

Referencias y lecturas

Lectura de entrada

Yu, D., Yang, B., Liu, C., Sun, F. & Gonzalez, J. E. (2023). Neuro-Symbolic Approaches for Text-Based Reasoning: A Survey. *ArXiv preprint*. Survey actualizado que cubre los patrones de integración neuro-simbólica en el contexto específico de NLP y razonamiento textual. Proporciona una visión panorámica de las arquitecturas disponibles con análisis de trade-offs. Lectura de entrada que complementa bien la perspectiva más general de Garcez y Lamb (2023).

Lecturas esenciales

- Garcez, A. d'Avila, Besold, T. R., De Raedt, L., Földiák, P., Hitzler, P., Icard, T., Kühnberger, K.-U., Lamb, L. C., Miikkulainen, R. & Silver, D. L. (2019). Neural-Symbolic Computing: An Effective Methodology for Principled Integration of Machine Learning and Reasoning. *Journal of Applied Logics*, 6(4), 611–632. — El marco de diseño para la integración neuro-simbólica. Define dimensiones de diseño que esta unidad refina para ACAI.
- Willard, B. T. & Louf, R. (2023). Efficient Guided Generation for Large Language Models. *ArXiv preprint arXiv:2307.09702*. — Outlines: constrained decoding con gramáticas formales. Referencia técnica esencial para el patrón de nivel 3 más directamente implementable.
- Pan, S., Luo, L., Wang, Y., Chen, C., Wang, J. & Wu, X. (2023). Unifying Large Language Models and Knowledge Graphs: A Roadmap. *IEEE Transactions on Knowledge and Data Engineering*. — Integración de LLMs con bases de conocimiento formales. Directamente relevante para el pipeline gobernado con RAG gobernado de ACAI.
- Larsson, S. & Traum, D. (2000). Information state and dialogue management in the TRINDI dialogue move engine toolkit. *Natural Language Engineering*, 6(3–4), 323–340. — Ya citado en UD3, pero aquí relevante como fundamento del information state como pilar integrador de la arquitectura.

Lecturas complementarias

- Zheng, L., Yin, L., Xie, Z., Huang, J., Sun, C., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C. & Sheng, Y. (2023). Efficiently Programming Large Language Models using SGLang. *ArXiv preprint arXiv:2312.07104*. — Control de flujo integrado con generación restringida. Complementa a Outlines con un enfoque de programación.
- Rebedea, T., Dinu, R., Sreedhar, M., Parisien, C. & Cohen, J. (2023). NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications. *EMNLP 2023 (demo)*. — El ejemplo industrial principal del patrón "neural con guardrails simbólicos".
- Inan, H. et al. (2023). Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. *ArXiv preprint*. — Variante del patrón de guardrails donde el filtro es neuronal con taxonomía simbólica.
- Nye, M., Andreassen, A. J., Gur-Ari, G., Michalewski, H., Austin, J., Biber, D., Dohan, D., Lewkowycz, A., Bosma, M., Luan, D., Sutton, C. & Odena, A. (2021). Show Your Work: Scratchpads for Intermediate Computation with Language Models. *ArXiv preprint arXiv:2112.00114*. — Razonamiento con pasos intermedios como precursor del reasoning loop.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR 2023*. — El patrón ReAct como referencia del patrón "neural-in-the-loop" (contrapunto a ACAI).
- OpenAI (2024). Structured Outputs. *OpenAI API Documentation*. — Implementación industrial del structured prompting de nivel 2.

- De Raedt, L., Dumančić, S., Manhaeve, R. & Marra, G. (2020). From Statistical Relational to Neuro-Symbolic Artificial Intelligence. *ArXiv preprint arXiv:2003.08316*. — Perspectiva desde la programación lógica probabilística. Relevante para la fundamentación teórica de la integración.
- Donadello, I., Serafini, L. & Garcez, A. d'Avila (2017). Logic Tensor Networks for Semantic Image Interpretation. *IJCAI 2017*. — Ejemplo de integración profunda (nivel 3–4) en el dominio de la visión. Relevante como referencia de lo posible más allá de ACAI.
- Poesia, G., Polozov, O., Le, V., Tiwari, A., Soares, G., Meek, C. & Gulwani, S. (2022). Synchromesh: Reliable Code Generation from Pre-trained Language Models. *ICLR 2022*. — Constrained decoding aplicado a generación de código. Análogo al uso de ACAI para generación de representaciones formales.

MONOGRÁFICO FORMATIVO

Fundamentos de IA Conversacional Neuro-Simbólica

«The bridge between language and logic is not a single span but a spectrum of crossings, each trading flexibility for guarantee.»

Eje B — El paradigma neuro-simbólico

Unidad 7 de 12

Research Sprint ACAI · Proyecto $(AI)^2$:TDG · 2026

Eje B — El paradigma neuro-simbólico

Prerequisitos: UD5, UD6 · Eje B — El paradigma neuro-simbólico

UNIDAD DIDÁCTICA 7

Técnicas neuro-simbólicas para el procesamiento del lenguaje natural

Contenidos de la unidad

7.1.1 El espectro de restricción 4
7.2.1 Semantic parsing: la tradición y su transformación 5
7.2.2 Structured extraction con LLMs 6
7.2.3 El espectro de restricción en la interpretación 8
Restricción gramatical: constrained decoding para NLU 8
Verificación post-generación: el reasoning loop para NLU 9
7.2.4 Dialogue state tracking como tarea neuro-simbólica 10
7.3.1 El problema de la fidelidad 11
7.3.2 Técnicas de generación fiel 11
Prompting con especificación de fidelidad 11
Template-guided generation 12
Constrained decoding para NLG 12
Verificación post-generación 13
7.3.3 Políticas de generación como restricciones formalizadas 13
7.4.1 La compilación como text-to-code 14
7.4.2 El pipeline de compilación 14
7.4.3 La triple restricción como desafío técnico 16
7.5.1 Síntesis del espectro 17
7.5.2 Restricción dinámica 18
7.6.1 Mapeo de técnicas a componentes 19
7.6.2 La UD7 como bisagra 19

¿Por qué necesito esto para ACAI?

La UD 5 explicó por qué la integración neuro-simbólica es necesaria. La UD 6 mostró con qué patrones arquitectónicos se implementa. Esta unidad responde la pregunta restante: *con qué técnicas concretas*. El investigador necesita un repertorio preciso de herramientas para cruzar la frontera entre lenguaje natural y representaciones formales — en ambas direcciones y en los distintos momentos del ciclo de vida del sistema. Sin estas técnicas, la arquitectura ACAI es un esquema; con ellas, es un plan implementable. Esta es la unidad más extensa del monográfico porque es la más directamente operativa: lo que aquí se presenta es lo que el investigador usará durante el sprint.

Objetivos de aprendizaje

Al completar esta unidad, el investigador será capaz de:

1. Formular el problema central de ACAI como un problema de cruce entre lenguaje natural y representaciones formales, con cuatro flujos diferenciados.
2. Seleccionar y aplicar técnicas de semantic parsing y structured extraction adecuadas para la interpretación del input en el formalismo ACAI.
3. Aplicar el espectro de restricción — desde prompting hasta constrained decoding con verificación — evaluando el balance entre flexibilidad y garantía en cada punto.
4. Diseñar un pipeline de generación fiel que garantice correspondencia entre las decisiones del motor y el texto generado.
5. Diseñar un pipeline de compilación (design time) que transforme descripciones en lenguaje natural en representaciones formales verificadas.
6. Evaluar las técnicas contra la triple restricción (legibilidad, procesabilidad, generabilidad) y sus tensiones.

7.1 El problema del puente

Sede principal de este concepto.

Toda la maquinaria técnica de ACAI puede entenderse como un conjunto de soluciones a un problema único: cómo cruzar la frontera entre dos mundos con propiedades radicalmente diferentes. El lenguaje natural es inherentemente ambiguo — una misma frase puede tener múltiples interpretaciones legítimas. Es contextual — el significado

El problema técnico central de ACAI es cruzar la frontera entre lenguaje natural y representaciones formales — en ambas direcciones y en distintos momentos del ciclo de vida.

depende de la historia del diálogo, del conocimiento compartido, de las expectativas pragmáticas. Es gradual — los conceptos tienen fronteras difusas, las implicaturas son más o menos fuertes, la certeza del hablante es una cuestión de grado. Y es creativo — un hablante puede expresar la misma idea de un número impredecible de formas.

Las representaciones formales, por el contrario, son precisas — cada símbolo tiene una semántica definida que no depende de la interpretación. Son descontextualizadas — una regla significa lo mismo independientemente del turno en que se invoque. Son discretas — un estado se satisface o no, una condición es verdadera o falsa. Y son cerradas — solo se pueden expresar los conceptos previstos por el vocabulario y la gramática del formalismo.

Cruzar esta frontera implica una pérdida inevitable en ambas direcciones. De lenguaje natural a formal, se pierde la riqueza pragmática que el formalismo no captura: el matiz emocional, la fuerza ilocutiva exacta, las implicaturas conversacionales sutiles. De formal a lenguaje natural, se pierde la precisión: el texto generado es siempre una aproximación del significado formal, susceptible de malinterpretación. La calidad de un sistema neuro-simbólico se mide en buena medida por cuánto se pierde en cada cruce.

Para ACAI, este cruce se produce en cuatro flujos diferenciados que organizan toda esta unidad. El primero es la *interpretación* (NL → Formal en runtime): mapear lo que dice el usuario a términos del modelo formal. El segundo es la *generación* (Formal → NL en runtime): expresar las decisiones del motor en lenguaje natural fiel. El tercero es la *compilación* (NL → Formal en design time): construir el modelo formal a partir de descripciones en lenguaje natural. El cuarto es el *aprendizaje* (Formal → análisis → Formal en learning time): analizar las trazas de operación para proponer extensiones del modelo.

7.1.1 El espectro de restricción

Transversal a los cuatro flujos, un eje organiza las técnicas según el nivel de garantía formal que ofrecen. Este **espectro de restricción** va desde las técnicas más flexibles y menos garantizadas hasta las más rígidas y más garantizadas.

En un extremo, la *restricción blanda* (prompting): el LLM recibe instrucciones en lenguaje natural sobre qué producir y en qué formato. La calidad depende de que el LLM siga las instrucciones, lo que ocurre con alta probabilidad pero sin garantía.

En el segundo nivel, la *restricción estructural* (schemas): el LLM debe producir su salida en un formato estructurado que se ajusta a un esquema predefinido (JSON Schema, function calling). La corrección sin-

táctica del formato está garantizada por la API del modelo; la corrección semántica del contenido no.

En el tercer nivel, la *restricción gramatical* (constrained decoding): la generación del LLM se fuerza, token a token, a producir secuencias compatibles con una gramática formal. La corrección sintáctica respecto a la gramática está garantizada por construcción; la corrección semántica depende de que la gramática capture las restricciones relevantes.

En el cuarto nivel, la *verificación post-generación* (reasoning loop): la salida del LLM se evalúa contra el modelo formal completo. Si la verificación falla, el LLM regenera condicionado por el feedback del verificador. La corrección se busca iterativamente con garantía de detección de errores (aunque no de corrección en tiempo acotado).

El espectro de restricción no es un ranking: más restricción no es siempre mejor. Es un mapa de trade-offs.

Cada nivel del espectro ofrece un balance diferente entre flexibilidad (la capacidad del LLM para manejar situaciones imprevistas), garantía (la probabilidad de que el resultado sea formalmente correcto), latencia (el tiempo de procesamiento), y complejidad (el esfuerzo de implementación). La elección del nivel adecuado no es uniforme: depende de la consecuencia de un error en cada componente. Para ACAI, el motor simbólico puede determinar dinámicamente el nivel de restricción según el contexto del diálogo y la criticidad de la operación — una forma de gobernanza que conecta el espectro de restricción con el information state.

7.2 De lenguaje a estructura: el cruce NL→Formal en runtime

Sede principal de este concepto.

La interpretación del input del usuario en términos del modelo formal es la tarea más crítica del pipeline de ACAI. Si la interpretación es incorrecta, la decisión del motor será correcta respecto a una premisa falsa — un tipo de error particularmente difícil de detectar y particularmente costoso en términos de accountability. Las técnicas de esta sección abordan cómo minimizar el error de interpretación con garantías crecientes.

7.2.1 Semantic parsing: la tradición y su transformación

Kamath & Das (2019) — survey de semantic parsing, de CCG a modelos neuronales.

El **semantic parsing** — la tarea de mapear lenguaje natural a representaciones formales ejecutables — es la tarea de NLP más directamente relevante para ACAI. Su historia traza una evolución que recapitula, en miniatura, la tensión fundacional analizada en la **UD 5**.

Los primeros parsers semánticos operaban sobre gramáticas forma-

les diseñadas manualmente. Zettlemoyer y Collins (2005, 2007), en trabajos fundacionales, usaron gramáticas categóricas combinatorias (CCG) para aprender mapeos de lenguaje natural a expresiones en lambda calculus. El enfoque producía representaciones formalmente correctas por construcción — la gramática garantizaba la buena formación — pero era frágil ante la variabilidad lingüística y requería gramáticas diseñadas para cada dominio.

La siguiente generación adoptó modelos neuronales de secuencia a secuencia. Jia y Liang (2016) y Dong y Lapata (2016) demostraron que redes neuronales podían aprender el mapeo NL → Formal directamente de pares de entrenamiento, sin gramáticas explícitas. El enfoque ganaba robustez ante la variabilidad (la red generaliza a formulaciones no vistas) pero perdía la garantía de corrección formal: la red podía generar expresiones sintácticamente malformadas.

Zettlemoyer & Collins (2005): semantic parsing con CCG — corrección garantizada, fragilidad ante variabilidad.

La generación actual usa LLMs de propósito general. Shin et al. (2021), con PICARD, demostraron que un LLM condicionado con constrained decoding produce SQL a partir de lenguaje natural con calidad superior a los modelos especializados, y con la garantía de que el SQL generado es siempre sintácticamente válido. Rajkumar et al. (2022) extendieron este resultado a otros formalismos. La idea central de PICARD — usar un LLM potente y restringir su generación para que satisfaga la gramática del formalismo de destino — es directamente el patrón que ACAI necesita.

📖 Shin et al. (2021) — PICARD: LLM con constrained decoding para semantic parsing.

Para ACAI, el semantic parsing tiene una especificidad que lo distingue de los benchmarks académicos. El formalismo de destino no es SQL (cuya gramática es bien conocida y estable) ni lambda calculus (cuya semántica es canónica), sino el lenguaje de representación propio de ACAI, diseñado bajo la triple restricción. Esto significa que el LLM no puede apoyarse en el conocimiento extenso de SQL que posee de su entrenamiento sino que debe aprender el formalismo ACAI mediante in-context learning, few-shot examples, y fine-tuning selectivo. Las técnicas de adaptación a formalismos nuevos son, por tanto, especialmente relevantes.

7.2.2 Structured extraction con LLMs

La técnica más directamente aplicable en la Phase 1 de ACAI es la **structured extraction**: usar el LLM para extraer información estructurada del input del usuario en un formato predefinido. A diferencia del semantic parsing completo (que produce una representación formal ejecutable), la structured extraction produce una estructura intermedia que el motor simbólico procesa para derivar la representación formal completa.

La structured extraction de ACAI no es una extracción de intents y slots como la que la **UD 2** analizó y criticó. Opera sobre una estructura significativamente más rica que captura múltiples dimensiones del acto comunicativo.

La structured extraction de ACAI captura cinco componentes: actos comunicativos, entidades con mapeo ontológico, señales pragmáticas, relaciones entre actos, y nivel de certeza.

El primer componente es el *acto comunicativo*, que va más allá del intent simple. Un acto comunicativo tiene tipo (solicitar, informar, confirmar, rechazar, preguntar), objeto (la entidad sobre la que opera), condiciones (bajo qué circunstancias), y fuerza ilocutiva (directa, indirecta, tentativa). «Quiero cancelar mi seguro» es un acto directo de tipo solicitar con objeto cancelación. «No sé si me conviene seguir con el seguro» es un acto indirecto de tipo explorar-cancelación con fuerza tentativa. La diferencia tiene consecuencias para el motor: el primero activa el protocolo de cancelación; el segundo activa un protocolo de consulta.

El segundo componente es la *extracción de entidades con mapeo ontológico*. No basta con reconocer que «seguro de hogar» es una entidad de tipo producto; es necesario mapearla a la entidad formal correspondiente en el modelo de conocimiento: el producto específico `seguro_hogar_HG-2019-4421` del cliente identificado. Este mapeo requiere que el LLM tenga acceso al contexto del information state (qué pólizas tiene el cliente) como parte de su prompt estructurado.

El tercer componente son las *señales pragmáticas*: indicadores de grounding («entiendo», «vale»), señales de reparación («no, quiero decir...»), marcadores de incertidumbre («creo que...», «no estoy seguro»), y actos de gestión del diálogo («espere un momento», «¿puede repetir?»). Estas señales corresponden a las dimensiones 2–5 de la conversación analizadas en la **UD 2** y son las que los sistemas basados en intents ignoran sistemáticamente.

El cuarto componente son las *relaciones entre actos* cuando el input contiene múltiples actos. «Quiero cancelar, pero antes dígame si pierdo la bonificación» contiene dos actos (solicitar cancelación, solicitar información sobre bonificación) con una relación de precedencia (el segundo debe resolverse antes de ejecutar el primero). Capturar esta estructura requiere que la extracción produzca no una lista plana de actos sino un grafo con relaciones.

El quinto componente es el *nivel de certeza* de la extracción: una señal que indica al motor cuán confiable es la interpretación. Una extracción con alta certeza puede procesarse directamente; una con baja certeza puede activar el reasoning loop o una solicitud de clarificación.

La implementación práctica de la structured extraction usa las técnicas de in-context learning documentadas por Brown et al. (2020) y analizadas en profundidad por Min et al. (2022). El prompt del LLM incluye: el esquema de la estructura de salida (JSON Schema), ejemplos

anotados de inputs y sus extracciones (few-shot examples), el estado relevante del diálogo (information state parcial), y las entidades del dominio disponibles. Los trabajos recientes sobre la calidad del in-context learning (Wei et al., 2023; Dong et al., 2024) muestran que la selección de los ejemplos y su ordenación tienen impacto significativo en la calidad de la extracción.

📖 Min et al. (2022) — análisis de qué aprenden los LLMs del in-context learning.

7.2.3 El espectro de restricción en la interpretación

Las técnicas de structured extraction descritas operan en los dos primeros niveles del espectro de restricción (prompting y schemas). Para situaciones que requieren mayor garantía, ACAI dispone de técnicas en los niveles superiores del espectro.

Restricción gramatical: constrained decoding para NLU

El constrained decoding, descrito arquitectónicamente en la **UD 6**, se implementa técnicamente mediante un autómata que se ejecuta en paralelo con la generación del LLM. En cada paso de la generación autoregresiva, el autómata determina qué tokens son compatibles con la gramática del formalismo de destino dado el prefijo generado hasta ese punto. Los tokens incompatibles reciben probabilidad cero en la distribución del LLM, lo que garantiza que la secuencia completa satisface la gramática.

📖 Willard & Louf (2023) — Outlines: la referencia técnica para constrained generation con gramáticas.

Willard y Louf (2023), en su implementación Outlines, demostraron que este mecanismo es práctico y eficiente. Su enfoque convierte la gramática (que puede ser una gramática context-free, una expresión regular, o un JSON Schema) en un índice de tokens permitidos para cada estado del autómata, lo que reduce el overhead a una consulta en tabla por paso de generación. El impacto en la latencia es modesto (típicamente un 10–20 % de overhead) y en la calidad es frecuentemente positivo: la eliminación de tokens degenerados mejora la coherencia de la generación.

Para ACAI, el constrained decoding en NLU garantiza que la interpretación producida por el LLM es siempre una expresión sintácticamente válida del formalismo de actos comunicativos. Si el formalismo define que un acto tiene tipo, objeto, condiciones y fuerza, el constrained decoding garantiza que la salida del LLM contendrá exactamente esos campos con valores del vocabulario permitido. La corrección sintáctica queda garantizada por construcción; la corrección semántica (¿es este el acto correcto?) sigue dependiendo de la capacidad del LLM.

Scholak et al. (2021) aplicaron esta idea específicamente al semantic parsing con su sistema PICARD, que demostró ganancias significativas

en la tarea de text-to-SQL al combinar un LLM potente (T5) con un parser incremental que verifica la validez del SQL en cada paso de generación. La extensión de este enfoque al formalismo ACAI requiere definir la gramática del formalismo como un autómata compatible con las implementaciones disponibles — un requisito que condiciona el diseño del lenguaje de representación y que la **UD 9** analizará en detalle.

Scholak et al. (2021): PICARD — constrained decoding específico para semantic parsing.

Verificación post-generación: el reasoning loop para NLU

El cuarto nivel del espectro añade una verificación semántica que el constrained decoding no puede proporcionar. Una interpretación puede ser sintácticamente válida (es una expresión bien formada del formalismo) y semánticamente incorrecta (atribuye al usuario un acto que no realizó, o mapea una entidad incorrectamente).

La verificación semántica opera evaluando la interpretación contra el information state y las restricciones del modelo formal. Si el LLM interpreta que el usuario solicita cancelar una póliza que no existe en su cartera, la verificación detecta la inconsistencia. Si interpreta que el usuario confirma una operación que nunca se le propuso, la verificación detecta la incoherencia temporal. En cada caso, el feedback estructurado al LLM especifica la naturaleza del error y las restricciones que la nueva interpretación debe satisfacer.

Espectro de restricción en acción

Input del usuario: «A ver, lo que quiero es que no me cobren más pero sin perder lo que llevo acumulado, ¿se puede?»
Nivel 1 (prompting): El LLM interpreta libremente → acto: `solicitar_información` sobre opciones de reducción de coste. Plausible pero impreciso: no captura la condición sobre lo acumulado.
Nivel 2 (schema): El LLM produce un JSON con campos obligatorios → acto_principal: `solicitar_cambio`, condición: `preservar_bonificación_acumulada`, objeto: `coste_prima`. Mejor: la estructura fuerza a explicitar la condición.
Nivel 3 (constrained decoding): La gramática del formalismo fuerza a seleccionar entre actos y condiciones del vocabulario formal → acto: `explorar_alternativa`(`reducción_prima`), restricción: `mantener`(`bonificación_no_siniestralidad`), modalidad: `consulta_viabilidad`. Sintácticamente garantizado.
Nivel 4 (reasoning loop): El motor evalúa: el cliente tiene bonificación por 3 años sin siniestros. Las opciones compatibles son: suspensión temporal (mantiene bonificación), cambio de cobertura (mantiene bonificación), cancelación (pierde bonificación). El feedback al LLM incluye estas opciones concretas → el LLM refina: el usuario explora alternativas a la cancelación que preserven su historial, compatible con

```
suspensión_temporal o cambio_cobertura.
```

7.2.4 Dialogue state tracking como tarea neuro-simbólica

El **dialogue state tracking** (DST) — la tarea de mantener una representación actualizada del estado del diálogo — conecta la interpretación turno a turno con el mantenimiento del information state a lo largo de la conversación. En los sistemas de diálogo clásicos, el DST era una tarea separada con modelos especializados. En ACAI, es una función distribuida entre el LLM (que proporciona la interpretación de cada turno) y el motor simbólico (que actualiza el information state según las reglas formales).

La tradición de DST es extensa. Los DSTC challenges (Henderson et al., 2014) establecieron benchmarks que impulsaron el desarrollo de modelos progresivamente más sofisticados: de modelos estadísticos con features manuales a modelos neuronales especializados (Zhong et al., 2018; Wu et al., 2019), y más recientemente a LLMs con in-context learning (Feng et al., 2023; Hudecek y Dusek, 2023). La tendencia muestra una mejora constante en la capacidad de tracking pero una persistente dificultad con los cambios de estado complejos: correcciones del usuario, cambios de opinión, retracciones de compromisos.

Feng et al. (2023) — DST con LLMs: estado del arte y limitaciones.

Para ACAI, las lecciones del DST son directamente operativas. La primera es que la representación del estado debe ser suficientemente rica para capturar no solo los valores actuales de las variables del diálogo sino también su *historial de cambios*: un dato que fue proporcionado, luego corregido, tiene un estatus diferente al de un dato proporcionado una sola vez. El information state de ACAI incorpora esta dimensión temporal.

La segunda lección es que el DST con LLMs funciona significativamente mejor cuando el LLM recibe el estado anterior como parte de su contexto, en lugar de inferirlo de toda la historia del diálogo. Esto justifica la decisión arquitectónica de ACAI de mantener un information state explícito que se proporciona al LLM en cada turno como contexto estructurado.

La tercera lección es que los errores de DST se acumulan: un error en el turno 3 persiste y se amplifica en los turnos siguientes si no se corrige. En ACAI, el motor simbólico funciona como mecanismo de corrección: al evaluar la interpretación del LLM contra las reglas formales y el information state, puede detectar inconsistencias que un DST puramente neuronal propagaría.

7.3 De estructura a lenguaje: el cruce Formal→NL en runtime

Sede principal de este concepto.

7.3.1 El problema de la fidelidad

La generación de lenguaje natural a partir de representaciones formales en ACAI tiene un problema específico que no es el que los LLMs resuelven bien (la naturalidad) sino uno que no suelen abordar: la **fidelidad**. La fidelidad es la correspondencia estricta entre lo que el motor simbólico determinó y lo que el texto generado expresa. Una respuesta infiel puede ser perfectamente natural y completamente incorrecta desde la perspectiva de la accountability.

El problema de la generación en ACAI no es la naturalidad — es la fidelidad: que el texto exprese exactamente lo que el motor determinó, sin más y sin menos.

La infidelidad puede manifestarse de tres formas. La *adición*: el LLM añade información que el motor no autorizó. Si el motor determina informar sobre la bonificación y el LLM añade «y probablemente le convenga esperar a fin de año», ha producido un consejo financiero no autorizado. La *omisión*: el LLM omite información que el motor requiere comunicar. Si el motor determina informar sobre la penalización y la bonificación, y el LLM solo menciona la bonificación, ha omitido información legalmente relevante. La *distorsión*: el LLM modifica las condiciones formales. Si la bonificación es válida durante 24 meses y el LLM genera «la bonificación se mantiene durante un tiempo», ha introducido una imprecisión con consecuencias contractuales.

Maynez et al. (2020) propusieron una taxonomía de infidelidades en la generación abstractiva que es directamente aplicable: infidelidades intrínsecas (que contradicen la fuente) y extrínsecas (que añaden información no presente en la fuente). Honovich et al. (2022) desarrollaron métodos automáticos de detección de inconsistencias factuales usando modelos de *natural language inference* (NLI). Estos métodos son adaptables al contexto de ACAI, donde la "fuente" no es un documento sino la especificación formal del motor.

Maynez et al. (2020) — taxonomía de infidelidades en la generación de texto.

7.3.2 Técnicas de generación fiel

El espectro de restricción se aplica a la generación con las mismas cuatro gradaciones que a la interpretación, cada una con sus propiedades específicas.

Prompting con especificación de fidelidad

En el nivel más básico del espectro, el LLM recibe la decisión del motor como parte del prompt con instrucciones explícitas de fidelidad:

«Genera una respuesta que comunique exactamente la siguiente información al usuario: [especificación del motor]. No añadas información que no esté en la especificación. Incluye todos los datos numéricos tal como aparecen. Si la especificación incluye condiciones, comunícalas completamente.» Esta técnica funciona razonablemente bien en la mayoría de los casos pero sin garantía formal. Liu et al. (2023) documentaron que la adherencia a las instrucciones del prompt se degrada con la longitud del contexto y la complejidad de las instrucciones.

Template-guided generation

Una técnica intermedia que es especialmente prometedora para ACAI es la **template-guided generation**: el motor proporciona un template parcial donde los elementos críticos (datos numéricos, condiciones formales, compromisos) están fijados como texto literal, y el LLM completa los espacios intermedios con transiciones naturales, expresiones de cortesía, y adaptación al tono del diálogo.

La template-guided generation combina la fidelidad de los templates con la naturalidad de la generación libre.

Por ejemplo, el motor genera el template: «Su bonificación por ausencia de siniestros asciende a {monto} y se mantiene durante {período}. [TRANSICIÓN] Si cancela antes de {fecha}, se aplicará una penalización de {penalización}. [CIERRE]». Los valores entre llaves son literales del modelo formal; las etiquetas [TRANSICIÓN] y [CIERRE] son espacios que el LLM completa con texto natural adaptado al contexto.

Esta técnica ofrece fidelidad estricta para los elementos críticos (los datos literales no pueden ser distorsionados porque no los genera el LLM) con naturalidad para los elementos expresivos (el LLM aporta la fluidez que un template rígido no tendría). El trade-off es que requiere que el motor produzca templates, lo que añade complejidad al modelo formal.

Constrained decoding para NLG

El constrained decoding puede aplicarse a la generación para garantizar la inclusión de elementos obligatorios y la exclusión de patrones prohibidos. Los trabajos sobre *lexically constrained decoding* son directamente relevantes. Hokamp y Liu (2017) propusieron Grid Beam Search, que modifica la búsqueda beam para garantizar que ciertos tokens o secuencias aparecen en la salida. Post y Vilar (2018) propusieron Dynamic Beam Allocation para hacer este proceso más eficiente. Lu et al. (2022) extendieron estas técnicas con NeuroLogic Decoding, que permite especificar restricciones complejas (conjunciones, disyunciones, negaciones) sobre los tokens de la salida.

Para ACAI, las restricciones de generación pueden derivarse del motor simbólico: los tokens obligatorios (el número de la póliza, el monto

📖 Lu et al. (2022) — NeuroLogic Decoding: restricciones complejas sobre la generación.

de la bonificación, el período), los patrones prohibidos (asesoramiento financiero, promesas no autorizadas, datos personales de otros clientes), y las restricciones de estructura (la penalización debe mencionarse *después* de las opciones alternativas, no antes, según la política de retención).

Verificación post-generación

El nivel más alto del espectro añade una verificación de la respuesta generada contra la especificación del motor. Esta verificación puede usar técnicas de NLI (*Natural Language Inference*) para comprobar que cada proposición de la respuesta es consecuencia de la especificación (Laban et al., 2022), o puede usar un segundo LLM como verificador que evalúa la fidelidad con una rúbrica formal (Tang et al., 2024). En el contexto de ACAI, la verificación puede ser parcialmente simbólica: extraer los datos numéricos de la respuesta generada y compararlos con los datos del motor, verificar que las condiciones mencionadas corresponden a las condiciones formales, comprobar que no se han añadido compromisos no autorizados.

Tang et al. (2024): verificación de fidelidad con rúbricas formales — adaptable al contexto ACAI.

7.3.3 Políticas de generación como restricciones formalizadas

> **Políticas de generación en el modelo formal**
>
> Las políticas de generación — tono, nivel de detalle, estrategia comunicativa, formato — pueden formalizarse como parámetros que el motor simbólico determina y que condicionan la generación del LLM. Por ejemplo, el motor puede determinar que el turno actual requiere *tono empático* (porque el contexto es una cancelación), *nivel de detalle alto* (porque hay datos numéricos que comunicar), y *estrategia de retención* (porque la política comercial indica ofrecer alternativas antes de ejecutar la cancelación).
> Estos parámetros se inyectan en el prompt del LLM como parte de la especificación de generación, junto con la decisión operativa. La formalización de las políticas en el modelo de conocimiento permite que las decisiones de *cómo* comunicar sean tan auditables como las decisiones de *qué* comunicar: la traza registra no solo la información proporcionada sino la política de comunicación aplicada.

7.4 De lenguaje a formalismo: el cruce NL→Formal en design time

Sede principal de este concepto.

7.4.1 La compilación como text-to-code

La transformación de descripciones en lenguaje natural en representaciones formales — lo que ACAI denomina *compilación* — es estructuralmente análoga al problema de text-to-code que ha sido extensamente estudiado.

La compilación de ACAI es estructuralmente análoga al text-to-code, pero con un formalismo de destino nuevo y con requisitos de verificación específicos del dominio.

Chen et al. (2021) — Codex: LLM para generación de código.

Chen et al. (2021), con Codex, demostraron que un LLM entrenado sobre código puede generar programas correctos a partir de docstrings en lenguaje natural con una precisión notable: pass@1 del 28.8 % y pass@100 del 72.3 % en el benchmark HumanEval. Estos números tienen dos lecturas complementarias. Optimista: en el 72 % de los casos, al menos una de las muestras es correcta, lo que es útil en un pipeline con verificación. Pesimista: en el 71 % de los casos, la primera muestra no es correcta, lo que hace imprescindible un mecanismo de verificación.

Trabajos posteriores han mejorado significativamente estos números. Li et al. (2022) con AlphaCode y Rozière et al. (2024) con Code Llama demostraron que modelos especializados en código alcanzan pass@1 superiores al 50 % en benchmarks estándar. Más relevante para ACAI, los trabajos sobre generación de código en lenguajes de dominio específico (DSLs) — SQL, SPARQL, expresiones regulares — muestran que los LLMs pueden adaptarse a formalismos nuevos mediante few-shot learning, aunque la precisión depende de la complejidad y regularidad del formalismo.

La analogía text-to-code es precisa pero con un límite: el formalismo ACAI no está en los datos de entrenamiento del LLM.

La analogía tiene un límite importante: los LLMs conocen Python, SQL y JavaScript por su presencia masiva en los datos de entrenamiento. El formalismo ACAI es nuevo y no está en esos datos. Esto significa que la compilación de ACAI depende enteramente del in-context learning: la capacidad del LLM para aprender el formalismo a partir de la descripción del esquema y los ejemplos proporcionados en el prompt. Esta dependencia refuerza la importancia de la tercera restricción (generabilidad): el formalismo debe diseñarse para ser aprendible por el LLM a partir de pocos ejemplos.

7.4.2 El pipeline de compilación

Phase 0 / Phase 1
La compilación es la operación central de la formalización asistida.

El pipeline de compilación opera en cinco pasos, cada uno con técnicas específicas.

Paso 1: Elicitación estructurada. El LLM asiste al experto de dominio mediante un diálogo de elicitación que sigue un esquema predefinido. El esquema determina qué información es necesaria para producir una

formalización completa: descripción de la regla, condiciones de aplicación, excepciones, relaciones con otras reglas, datos requeridos, acciones resultantes. El LLM formula preguntas de seguimiento cuando la descripción del experto es incompleta o ambigua. La elicitación no es un formulario rígido sino un diálogo guiado donde el LLM usa su capacidad lingüística para interpretar descripciones informales y traducirlas a los campos del esquema.

Paso 2: Generación de primera formalización. El LLM genera una representación formal a partir de la descripción elicitada, condicionado por tres elementos: el esquema del formalismo ACAI (la gramática y las restricciones de buena formación), ejemplos de reglas existentes en el modelo (few-shot examples que muestran el estilo y las convenciones del formalismo), y el modelo de conocimiento actual (para garantizar compatibilidad con las reglas existentes). La generación puede usar structured output (nivel 2 del espectro) o constrained decoding (nivel 3) según la complejidad de la regla.

Paso 3: Validación simbólica. Un validador comprueba la representación generada en tres dimensiones. La *corrección sintáctica*: ¿es una expresión válida de la gramática del formalismo? La *consistencia con el modelo*: ¿contradice alguna regla existente? ¿Usa entidades y relaciones definidas en la ontología? La *completitud local*: ¿faltan condiciones que las reglas relacionadas tienen? ¿Están todos los campos obligatorios presentes?

Paso 4: Corrección iterativa. Si la validación detecta errores, el feedback se estructura y se devuelve al LLM para regeneración. El feedback es específico: no dice «la regla es incorrecta» sino «la condición `antigüedad > 30` referencia la entidad `antigüedad` que no está definida en la ontología; las entidades disponibles son `antigüedad_póliza`, `antigüedad_cliente`, `antigüedad_siniestro`». El LLM regenera condicionado por el error concreto y la corrección esperada. El ciclo continúa hasta que la validación no detecta errores o se alcanza un límite de iteraciones.

Paso 5: Revisión humana. La formalización validada se presenta al experto de dominio para su aprobación. La presentación incluye la representación formal y una *verbalización* generada por el LLM que traduce la regla formal de vuelta a lenguaje natural, permitiendo al experto verificar que la formalización captura su intención. Si el experto detecta discrepancias, el ciclo vuelve al paso 2 con las correcciones como input adicional.

Pipeline de compilación

Descripción del experto: «Cuando un cliente con más de dos años de antigüedad cancela, si no ha tenido siniestros, la bonificación se congela durante dos años por si quiere volver.»
Paso 1 (elicitación): LLM pregunta: ¿la congelación aplica a todos los productos o solo a hogar? ¿Qué pasa si el cliente vuelve después de los dos años? ¿La ausencia de siniestros se refiere a toda la vida de la póliza o a un período? Experto clarifica: solo hogar, después de dos años se pierde, se refiere a los últimos 3 años.
Paso 2 (generación): LLM genera la regla formal en el formalismo ACAI con las condiciones, restricciones de producto, período de referencia para siniestros, período de congelación, y acción resultante.
Paso 3 (validación): El validador detecta: la regla usa `producto = hogar` pero la ontología define `producto = seguro_hogar`. Error de vocabulario.
Paso 4 (corrección): Feedback al LLM: «Vocabulario incorrecto: `hogar` → `seguro_hogar`». LLM corrige. Segunda validación: correcta.
Paso 5 (revisión): Verbalización: «Cuando un cliente con más de 2 años de antigüedad cancela su seguro de hogar, y no ha tenido siniestros en los últimos 3 años, su bonificación por no-siniestralidad se congela durante 24 meses. Si contrata una nueva póliza de seguro de hogar dentro de ese período, recupera la bonificación acumulada. Pasados los 24 meses, la bonificación se pierde.» Experto aprueba.

7.4.3 La triple restricción como desafío técnico

La UD 5 introdujo la triple restricción (I1) como especificidad conceptual de ACAI. Aquí se concreta como un conjunto de requisitos técnicos que condicionan tanto el diseño del formalismo como las técnicas de compilación.

La **restricción de legibilidad** exige que un experto de dominio no técnico pueda leer y comprender las reglas formalizadas. Esto condiciona la sintaxis del formalismo: no puede usar notación lógica abstracta (el experto no lee $\forall x.P(x) \rightarrow Q(x)$) ni convenciones de programación opacas (el experto no lee `lambda x: x.policy.age > 30`). Los formalismos legibles tienden a ser verbosos, cercanos al lenguaje natural, con palabras clave inteligibles.

Las tres restricciones están en tensión: la legibilidad favorece formalismos verbosos, la procesabilidad favorece formalismos formales, y la generabilidad favorece formalismos regulares.

La **restricción de procesabilidad** exige que el motor simbólico pueda ejecutar las reglas de forma determinista y eficiente. Esto condiciona la semántica del formalismo: debe tener semántica operacional definida, sin ambigüedades de interpretación. Los formalismos procesables tienden a ser compactos, precisos, con una gramática formal estricta.

La **restricción de generabilidad** exige que el LLM pueda producir las reglas con fiabilidad a partir de descripciones en lenguaje natural.

Esto condiciona la regularidad del formalismo: las reglas deben seguir patrones predecibles que el LLM pueda aprender con pocos ejemplos. Los formalismos generables tienden a ser regulares, con estructuras repetitivas y un vocabulario controlado.

Las tres restricciones están en tensión porque favorecen propiedades diferentes del formalismo. Un formalismo óptimo para la legibilidad (verbose, cercano al NL) puede ser difícil de procesar (ambiguo) y de generar (irregular). Un formalismo óptimo para la procesabilidad (compacto, formal) puede ser difícil de leer (opaco) y de generar (el LLM no lo ha visto en sus datos de entrenamiento). Un formalismo óptimo para la generabilidad (regular, predecible) puede ser artificialmente rígido para el lector humano y poco expresivo para el motor.

Referencia contextual. Este concepto se desarrolla en profundidad en **UD 9** y **UD 11**. La evaluación sistemática de formalismos contra la triple restricción y el análisis detallado de sus tensiones y resoluciones se desarrollan en profundidad en el Eje C.

La resolución de estas tensiones no es un compromiso único sino un espacio de diseño que el sprint debe explorar. Los experimentos de la Phase 0 (directamente apoyados por la **UD 10**) consisten precisamente en probar diferentes puntos de este espacio: generar reglas con varios formalismos candidatos, evaluar la legibilidad con expertos, la procesabilidad con prototipos de motor, y la generabilidad con métricas de corrección del LLM.

> **La triple restricción como programa experimental**
>
> La triple restricción convierte el diseño del formalismo en un problema experimental, no solo teórico. No se puede determinar a priori qué formalismo satisface mejor las tres restricciones porque la generabilidad depende de propiedades empíricas del LLM que solo se conocen experimentando. Esto es una diferencia fundamental con el diseño clásico de lenguajes formales, donde los criterios de evaluación (expresividad, decidibilidad, complejidad) son formales. ACAI añade un criterio empírico — la generabilidad — que requiere experimentación.

7.5 El espectro de restricción como principio unificador

7.5.1 Síntesis del espectro

El espectro de restricción recorre toda esta unidad como eje transversal. Cada flujo (NLU, NLG, compilación) tiene sus cuatro niveles de restricción, y en cada flujo la elección del nivel produce un balance

diferente entre flexibilidad y garantía. La síntesis muestra que todas las técnicas presentadas son instancias de un principio común: *el grado de control simbólico sobre la computación neuronal es un parámetro de diseño continuo, no una decisión binaria*.

Esta formulación generaliza la taxonomía de Kautz dentro de cada nivel. Un sistema de nivel 2 puede operar con restricciones blandas (prompting simple) o con restricciones estructurales (schemas), y la diferencia en propiedades de accountability es significativa. Un sistema de nivel 3 puede operar con constrained decoding solo o con constrained decoding más reasoning loop, y de nuevo la diferencia importa. El espectro de restricción es un eje de variación *dentro* de cada nivel de la taxonomía de Kautz.

7.5.2 Restricción dinámica

Restricción dinámica

La **restricción dinámica** es el principio según el cual el nivel de restricción aplicado en cada interacción neuronal-simbólica se determina en función del contexto del diálogo y la criticidad de la operación, en lugar de ser fijo para todo el sistema.

La restricción dinámica es una forma de gobernanza sofisticada donde el motor simbólico no solo determina *qué* debe hacer el LLM sino *con cuánta libertad* puede hacerlo. En un turno donde el usuario pide información general sobre sus pólizas, el motor puede aplicar restricción blanda (prompting): la consecuencia de un error menor es baja. En un turno donde el sistema va a ejecutar una cancelación irreversible, el motor aplica restricción máxima (constrained decoding + reasoning loop + verificación post-generación): la consecuencia de un error es alta.

La restricción dinámica conecta el espectro de restricción con el information state y las políticas de riesgo del dominio.

Esta idea conecta tres elementos de la arquitectura ACAI: el espectro de restricción (las técnicas disponibles), el information state (el contexto que determina la criticidad), y las políticas del dominio (las reglas que definen qué nivel de error es aceptable en cada situación). La restricción dinámica es un ejemplo de cómo la gobernanza simbólica se extiende no solo a las decisiones operativas sino al *modo* en que se ejecutan las competencias delegadas al LLM.

7.6 Síntesis: el repertorio técnico de ACAI

7.6.1 Mapeo de técnicas a componentes

Las técnicas presentadas se mapean a los componentes de la arquitectura ACAI y a las fases del sprint de la forma siguiente.

Para la **interpretación** (NLU), la Phase 1 usa structured extraction con schemas (nivel 2) como mecanismo principal, con el information state como contexto estructurado. La Phase 2 añade constrained decoding para garantizar corrección sintáctica y reasoning loops para manejar ambigüedad, con restricción dinámica según la criticidad del turno.

Para la **generación** (NLG), la Phase 1 usa prompting con especificación de fidelidad y template-guided generation para los elementos críticos (nivel 2). La Phase 2 añade lexically constrained decoding para garantizar la inclusión de elementos obligatorios y verificación post-generación para los turnos de alta consecuencia.

Para la **compilación** (design time), ambas fases usan el pipeline de cinco pasos con reasoning loop iterativo. La Phase 1 opera con schemas (nivel 2) para la generación de formalizaciones. La Phase 2 explora constrained decoding (nivel 3) para reducir los errores de formalización y acelerar la convergencia del reasoning loop.

Para el **aprendizaje** (learning time), la Phase 2 introduce el análisis de trazas con LLM para detectar lagunas en el modelo formal, usando las técnicas de structured extraction sobre los logs conversacionales en lugar de sobre los inputs del usuario.

7.6.2 La UD7 como bisagra

> **Conexión Eje B → Eje C**
>
> Esta unidad cierra el Eje B (el paradigma neuro-simbólico) y abre el camino al Eje C (representación y formalización). Las técnicas aquí presentadas condicionan directamente qué formalismos son viables: un formalismo cuya gramática no se puede expresar como autómata para constrained decoding es incompatible con las técnicas de nivel 3; un formalismo demasiado irregular para el in-context learning del LLM no satisface la restricción de generabilidad; un formalismo demasiado opaco para el experto de dominio no satisface la restricción de legibilidad. La **UD 9** evaluará los formalismos candidatos contra estos criterios técnicos; la **UD 10** implementará el pipeline de compilación; la **UD 11** analizará las tensiones de la triple restricción. Las técnicas de la UD7 son el puente que conecta el paradigma con la representación.

Conexiones con otras unidades

→ **Prerequisito: UD 5** (Fundamentos neuro-simbólicos) para la taxonomía de Kautz y las fortalezas complementarias. **UD 6** (Arquitecturas y patrones) para los patrones que aquí se llenan con técnicas concretas.

⇄ **Complementa: UD 2** (La conversación como fenómeno) definió las cinco dimensiones conversacionales que la structured extraction de esta unidad debe capturar. UD7 proporciona las técnicas para implementar esa captura.

⇄ **Complementa: UD 3** (Ecosistema operativo) definió el concepto de RAG gobernado, capacidad agéntica gobernada, y alineamiento formalizado. Las técnicas de esta unidad (structured extraction, constrained decoding, template-guided generation) son los mecanismos concretos que implementan esa gobernanza.

Aplicación: UD 9 (Formalismos candidatos) evalúa los formalismos contra criterios que incluyen la compatibilidad con constrained decoding, la generabilidad por LLM con in-context learning, y la legibilidad.

Aplicación: UD 10 (Formalización asistida) implementa directamente el pipeline de compilación descrito en la sección 7.4.

Aplicación: UD 11 (La triple restricción) analiza en profundidad las tensiones entre las tres restricciones que la sección 7.4.3 introduce técnicamente.

Ejercicios

1. [Implementación] Diseñe el prompt completo (system prompt + few-shot examples) para la structured extraction de ACAI en el escenario de la aseguradora. El prompt debe incluir: (a) la descripción del esquema de extracción (acto comunicativo, entidades, señales pragmáticas, relaciones, certeza), (b) al menos tres ejemplos anotados de inputs con sus extracciones, y (c) las instrucciones de formato. Pruebe el prompt con un LLM disponible sobre los siguientes inputs: «Quiero cancelar mi seguro de hogar», «Bueno, no sé, ¿qué me recomienda?», y «Sí pero espere, antes de cancelar, ¿qué pasa con la bonificación?». Evalúe la calidad de las extracciones.

2. [Espectro de restricción] Para el input «Me gustaría cambiar lo del seguro», produzca cuatro interpretaciones correspondientes a los cuatro niveles del espectro de restricción. Para cada nivel,

especifique: (a) qué técnica se usa, (b) qué garantías ofrece, (c) qué errores son posibles, y (d) qué latencia añade. Evalúe cuál sería el nivel apropiado si el contexto es una primera consulta informativa vs. si el contexto es la confirmación de una operación irreversible.

3. [Generación fiel] Dado que el motor simbólico ha determinado la siguiente acción: «Informar al usuario: bonificación acumulada = 135 €, período de validez = 24 meses, condición = ausencia de siniestros en últimos 3 años. Política: tono empático, ofrecer alternativas a cancelación», diseñe: (a) un template para template-guided generation con espacios libres y elementos fijados, (b) las restricciones para un constrained decoding (tokens obligatorios, patrones prohibidos), y (c) una rúbrica de verificación post-generación con 5 criterios de fidelidad.

4. [Pipeline de compilación] Un experto de dominio describe la siguiente regla: «Si el cliente llama para quejarse de una subida de prima y lleva más de 5 años con nosotros, le ofrecemos mantener la prima anterior durante un año como gesto comercial, pero solo si no ha tenido más de un siniestro en los últimos dos años.» Ejecute los pasos 1–4 del pipeline de compilación: (a) formule al menos 3 preguntas de elicitación que clarificarían la regla, (b) genere una formalización en pseudocódigo estructurado, (c) identifique al menos 2 errores o ambigüedades que un validador detectaría, y (d) especifique el feedback estructurado para la corrección.

5. [Triple restricción] Considere dos formalismos candidatos para representar la regla del ejercicio anterior. El primero es un formalismo cercano al lenguaje natural: `WHEN client.tenure > 5 years AND client.claims(last 2 years) <= 1 AND call.reason = premium_increase THEN OFFER maintain_premium FOR 1 year`. El segundo es una notación lógica: `tenure(X,T), T>5, claims(X,2y,N), N=<1, reason(call,premium_up) => offer(X, maintain_premium, 12m)`. Evalúe ambos contra las tres restricciones: (a) legibilidad (¿lo entiende un experto no técnico?), (b) procesabilidad (¿se puede ejecutar determinísticamente?), y (c) generabilidad (¿puede un LLM producirlo fiablemente con few-shot learning?). Argumente cuál preferiría y por qué.

Referencias y lecturas

Lectura de entrada

Shin, R., Leblond, C. H., Thomson, S. & Song, D. (2021). Constrained Language Models Yield Few-Shot Semantic Parsers. *EMNLP 2021*. PICARD: la demostración más influyente de que un LLM con constrained decoding supera a los modelos especializados en semantic parsing. Directamente relevante para la estrategia de NLU de ACAI. Lectura de entrada que integra semantic parsing, constrained decoding, y LLMs en una sola técnica.

Lecturas esenciales

- Willard, B. T. & Louf, R. (2023). Efficient Guided Generation for Large Language Models. *ArXiv preprint arXiv:2307.09702*. — Outlines: la referencia técnica para constrained generation con gramáticas. Ya citado en UD6; aquí esencial como implementación.
- Kamath, A. & Das, R. (2019). A Survey on Semantic Parsing. *AKBC 2019*. — Survey comprensivo de semantic parsing, desde gramáticas categóricas hasta modelos neuronales. Proporciona el contexto histórico.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H., Kaplan, J. et al. (2021). Evaluating Large Language Models Trained on Code. *ArXiv preprint arXiv:2107.03374*. — Codex. Referencia central para la compilación text-to-code.
- Maynez, J., Narayan, S., Bohnet, B. & McDonald, R. (2020). On Faithfulness and Factuality in Abstractive Summarization. *ACL 2020*. — Taxonomía de infidelidades en generación. Base para el análisis de fidelidad en NLG.

Lecturas complementarias

- Zettlemoyer, L. S. & Collins, M. (2005). Learning to Map Sentences to Logical Form: Structured Classification with Probabilistic Categorial Grammars. *UAI 2005*. — Semantic parsing con CCG. Fundacional.
- Zettlemoyer, L. S. & Collins, M. (2007). Online Learning of Relaxed CCG Grammars for Parsing to Logical Form. *EMNLP-CoNLL 2007*. — Extensión del trabajo anterior. Fundacional.
- Jia, R. & Liang, P. (2016). Data Recombination for Neural Semantic Parsing. *ACL 2016*. — Semantic parsing neuronal. Transición de gramáticas a redes.
- Dong, L. & Lapata, M. (2016). Language to Logical Form with Neural Attention. *ACL 2016*. — Modelos de atención para semantic parsing.
- Rajkumar, N., Li, R. & Baber, D. (2022). Evaluating the Text-to-SQL Capabilities of Large Language Models. *ArXiv preprint*. — LLMs para semantic parsing: estado del arte.
- Scholak, T., Schucher, N. & Bahdanau, D. (2021). PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. *EMNLP 2021*. — Implementación técnica de constrained decoding para semantic parsing.
- Brown, T. B. et al. (2020). Language Models are Few-Shot Learners. *NeurIPS 2020*. — GPT-3 y few-shot learning. Base para la structured extraction con

in-context learning.

- Min, S., Lyu, X., Holtzman, A., Arber, M., Lewis, M., Hajishirzi, H. & Zettlemoyer, L. (2022). Rethinking the Role of Demonstrations: What Makes In-Context Learning Work? *EMNLP 2022*. — Análisis de qué aprenden los LLMs del in-context learning.
- Hokamp, C. & Liu, Q. (2017). Lexically Constrained Decoding for Sequence Generation Using Grid Beam Search. *ACL 2017*. — Constrained decoding léxico. Fundacional para NLG restringida.
- Post, M. & Vilar, D. (2018). Fast Lexically Constrained Decoding with Dynamic Beam Allocation. *NAACL 2018*. — Mejora de eficiencia sobre Grid Beam Search.
- Lu, X., West, P., Zellers, R., Le Bras, R., Bhagavatula, C. & Choi, Y. (2022). NeuroLogic A*esque Decoding: Constrained Text Generation with Lookahead Heuristics. *NAACL 2022*. — Restricciones complejas sobre la generación. Extensión sofisticada.
- Honovich, O., Aharoni, R., Herzig, J., Taitelbaum, H., Kuber, D., Choi, V., Szpektor, I. & Levy, O. (2022). TRUE: Re-evaluating Factual Consistency Evaluation. *NAACL 2022*. — Evaluación de consistencia factual. Adaptable a verificación de fidelidad.
- Laban, P., Schnabel, T., Bennett, P. N. & Hearst, M. A. (2022). SummaC: Re-Visiting NLI-based Models for Inconsistency Detection in Summarization. *TACL*, 10, 163–177. — NLI para detección de inconsistencias. Adaptable a verificación post-generación.
- Tang, L., Sun, Z., Patel, N., Obeid, J. & Jiang, X. (2024). Evaluating Large Language Models on Medical Evidence Summarization. *NPJ Digital Medicine*, 6, 158. — Verificación de fidelidad con rúbricas en dominio médico. Paradigma transferible.
- Henderson, M., Thomson, B. & Williams, J. D. (2014). The Second Dialog State Tracking Challenge. *SIGDIAL 2014*. — DSTC2: benchmark fundacional de dialogue state tracking.
- Feng, Y., Lipani, A., Ye, F., Zhang, Q. & Yilmaz, E. (2023). A Survey on Dialogue State Tracking: Models, Datasets, and Challenges. *ACM Computing Surveys*, 55(7). — Survey actualizado de DST.
- Hudecek, V. & Dusek, O. (2023). Are LLMs All You Need for Task-Oriented Dialogue? *SIGDIAL 2023*. — Evaluación de LLMs para DST y gestión de diálogo.
- Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R. et al. (2022). Competition-Level Code Generation with AlphaCode. *Science*, 378, 1092–1097. — AlphaCode.
- Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. et al. (2024). Code Llama: Open Foundation Models for Code. *ArXiv preprint arXiv:2308.12950*. — Code Llama.
- Liu, N. F. et al. (2023). Lost in the Middle: How Language Models Use

Long Contexts. *Transactions of the ACL*. — Degradación del seguimiento de instrucciones en contextos largos.

- Mernik, M., Heering, J. & Sloane, A. M. (2005). When and How to Develop Domain-Specific Languages. *ACM Computing Surveys*, 37(4), 316–344. — Diseño de DSLs. Relevante para la triple restricción.